IPA 情報処理技術者試験 ／ レベル3

# 応用情報 技術者試験

## 図解でわかる 学習ノート

Applied Information Technology Engineer Examination ／ 午前・午後 全分野対応




ゼロから読める基礎解説　全分野を網羅　午後の解き方つき

### Applied Information Technology Engineer Examination

# 図解でわかる 学習ノート

午前（テクノロジ・マネジメント・ストラテジ）＋ 午後（記述式）対応 ／ 全18章

**この資料の特徴**

- 「カルノー図とは？」「論理変数とは？」のように、**用語の意味そのものから**説明します。前提知識がなくても読み進められます。
- 各章の冒頭に図解を置き、まず「絵」で全体像をつかんでから細部に進む構成です。
- 計算問題は「計算例」で手順を1ステップずつ示します。公式は丸暗記ではなく「なぜそうなるか」から説明します。
- IPA公開の過去問題（主に令和7年度春期・秋期）で実際に問われたテーマを「過去問ではこう出る」で紹介します（問題文そのものは掲載していません）。
- 最終章で午後試験（記述式）の分野別の解き方と選択戦略を、巻末に直前チェック用の総まとめをまとめています。

# 目次


| | | |
|---|---|---|
| **はじめに ― 試験の全体像と本書の使い方** | 試験の仕組み・出題分野・合格ラインと、本書を使った学習の進め方 | **4** |
| **第1章　基礎理論** | 2進数・補数・論理演算・カルノー図・確率統計・待ち行列・符号・AI。すべての土台になる「数学と理論」 | **7** |
| **第2章　アルゴリズムとプログラミング** | 変数・配列・リスト・スタック・キュー・木構造と、探索・整列・計算量・再帰。午後問3の土台 | **18** |
| **第3章　コンピュータ構成要素** | CPU・命令実行・パイプライン・メモリ階層・キャッシュ・補助記憶・入出力・組込みハードウェア | **25** |
| **第4章　システム構成要素** | 処理形態・クラウド・仮想化・IaC・稼働率（直列／並列）・MTBF／MTTR・性能評価・キャパシティ | **33** |
| **第5章　ソフトウェア（OS）** | OSの役割・タスク管理・スケジューリング・排他制御・セマフォ・デッドロック・仮想記憶・ページ置換・メモリ管理・ファイル・OSS | **39** |
| **第6章　ヒューマンインタフェースとマルチメディア** | UI設計の原則・アクセシビリティ・Webデザインと、画像・音声・動画のデータ量計算・圧縮・VR/AR | **45** |
| **第7章　データベース** | 関係モデル・E-R図・キー・正規化・SQL（結合／集約／副問合せ）・トランザクション・排他制御・障害回復・NoSQL | **48** |
| **第8章　ネットワーク** | OSI参照モデルとTCP/IP・LAN機器・IPアドレスとサブネット計算・ルーティング・DNS/DHCP/NAT・HTTP/メール・無線LAN・伝送時間 | **54** |
| **第9章　情報セキュリティ** | CIA・脅威と攻撃手法・暗号（共通鍵／公開鍵）・ハッシュ・デジタル署名・PKI・認証・ネットワーク／Webセキュリティ・ISMS。午後問1（必須）の土台 | **60** |
| **第10章　システム開発技術** | 開発プロセス（共通フレーム）・要件定義・UML・モジュール設計（結合度／強度）・テスト技法・レビュー・アジャイル・DevOps・見積り | **68** |
| **第11章　プロジェクトマネジメント** | PMBOK・WBS・アローダイアグラムとクリティカルパス・日程短縮・EVM・リスク・品質・調達・コミュニケーション | **75** |
| **第12章　サービスマネジメント** | ITIL・JIS Q 20000・SLA・インシデント／問題／変更／リリース管理・可用性とキャパシティ・サービスデスク・ファシリティ | **80** |
| **第13章　システム監査** | システム監査の目的と流れ・監査人の独立性・監査証拠と監査技法・内部統制・ITガバナンス | **84** |
| **第14章　システム戦略とシステム企画** | 情報システム戦略・EA・BPR／BPM・ソリューション（クラウド／SaaS／RPA）・DX・システム化計画・要件定義・調達（RFI／RFP） | **87** |
| **第15章　経営戦略・技術戦略・ビジネスインダストリ** | SWOT・PPM・3C・5フォース・競争戦略・バランススコアカード・マーケティング（4P／STP）・技術経営（MOT）・e-ビジネス・IoT／Fintech | **91** |
| **第16章　企業活動と法務** | 経営組織・OR／IE（線形計画・在庫・QC七つ道具）・会計（損益分岐点・財務諸表・減価償却・NPV）・知的財産・セキュリティ関連法規・労働／取引関連法規・標準化 | **97** |
| **第17章　午後試験の全体像と解き方** | 記述式の基本ルール・時間配分・選択問題の選び方・11分野それぞれの「解き方の型」と頻出テーマ | **106** |

**総まとめ ― 試験直前チェック** 必須の公式・計算パターンと、まぎらわしい用語の区別を一覧で総復習 **112**

# はじめに ― 試験の全体像と本書の使い方

応用情報技術者試験（AP：Applied Information Technology Engineer Examination）は、IPA（独立行政法人 情報処理推進機構）が実施する国家試験で、「ITエンジニアとして独り立ちできる応用的な知識・技能」を問う試験です。基本情報技術者試験（FE）の一つ上、高度試験の一つ下にあたる「レベル3」に位置付けられています。まずは試験の輪郭をつかみましょう。

**試験の全体像 ― 応用情報技術者試験**

午前と午後の両方で 60点以上 を取れば合格

| 午前試験 | 午後試験 | 合格ライン | 合格率の目安 |
|---|---|---|---|
| **80問** | **5問** | **60点** | **約25%** |
| 四肢択一・150分<br>全問必須 | 記述式・150分<br>問1必須＋4問選択 | 午前・午後とも<br>100点満点中 | 4人に1人程度<br>（年度により変動） |

**合格のコツ**　午前は「用語の意味」と「定番計算」を確実に。午後は「問題文の中に答えがある」ことを意識し、本文の言葉を使って答える練習を。午前の知識は午後の土台です。

**用語メモ　試験方式について**

本書は令和7年度（2025年度）までの筆記方式の出題形式を前提に説明しています。IPAは応用情報技術者試験のCBT方式（コンピュータで受験する方式）への移行を進めており、実施時期・時間・出題数などが変わる可能性があります。受験前には必ずIPAの公式サイトで最新の「試験要綱」を確認してください。

## 午前試験の出題範囲

午前試験の80問は、大きく3つの系統（テクノロジ系・マネジメント系・ストラテジ系）から出題されます。問題番号の並びもほぼ固定で、前半が技術、後半が経営・法律という順番です。

| 系統 | 問題番号の目安 | 問題数 | 主な分野 | 本書の章 |
|---|---|---|---|---|
| テクノロジ系 | 問1〜問50 | 約50問 | 基礎理論、アルゴリズム、コンピュータ、OS、DB、ネットワーク、セキュリティ、開発技術 | 第1〜10章 |
| マネジメント系 | 問51〜問60 | 約10問 | プロジェクトマネジメント、サービスマネジメント、システム監査 | 第11〜13章 |
| ストラテジ系 | 問61〜問80 | 約20問 | システム戦略・企画、経営戦略、技術戦略、企業活動、法務 | 第14〜16章 |

午前は**全問1問1.25点**（80問×1.25＝100点）なので、60点に必要な正解数は**48問**です。苦手分野があっても、得意分野でカバーできる配点です。

## 午後試験の出題範囲

午後試験は11問の中から**問1（情報セキュリティ）は必須**、残りの問2〜問11から**4問を選択**して、合計5問に解答します。1問20点です。

| 問 | 分野 | 区分 | 必要な午前知識（本書の章） |
|---|---|---|---|
| 問1 | 情報セキュリティ | 必須 | 第9章 |
| 問2 | 経営戦略 | 選択 | 第14・15・16章 |
| 問3 | プログラミング（アルゴリズム） | 選択 | 第2章 |
| 問4 | システムアーキテクチャ | 選択 | 第3・4章 |
| 問5 | ネットワーク | 選択 | 第8章 |
| 問6 | データベース | 選択 | 第7章 |
| 問7 | 組込みシステム開発 | 選択 | 第3・5章 |
| 問8 | 情報システム開発 | 選択 | 第10章 |
| 問9 | プロジェクトマネジメント | 選択 | 第11章 |
| 問10 | サービスマネジメント | 選択 | 第12章 |
| 問11 | システム監査 | 選択 | 第13章 |

## 学習ロードマップ

本書は全18章で構成されています。前の章の知識を使って次の章が理解できるよう並べているので、原則として順番に進めることをおすすめします。特に第1章（基礎理論）と第2章（アルゴリズム）は、その後のすべての章の土台です。

## 学習ロードマップ — 出題分野マップ（全18章）

テクノロジ系 → マネジメント系 → ストラテジ系 → 午後対策 の順に進む


**1 基礎理論**
2進数・論理演算・確率・AI

**2 アルゴリズム**
データ構造・探索・整列

**3 コンピュータ構成**
CPU・メモリ・入出力

**4 システム構成**
信頼性・性能・クラウド

**5 ソフトウェア**
OS・タスク・仮想記憶

**6 UI・マルチメディア**
画像・音声・圧縮

**7 データベース**
正規化・SQL・トランザクション

**8 ネットワーク**
TCP/IP・IPアドレス

**9 セキュリティ**
暗号・認証・攻撃と対策

**10 開発技術**
設計・テスト・アジャイル

**11 プロジェクト管理**
日程・EVM・リスク

**12 サービス管理**
ITIL・SLA・運用

**13 システム監査**
監査・内部統制

**14 システム戦略**
EA・企画・調達

**15 経営・技術戦略**
SWOT・PPM・マーケ

**16 企業活動・法務**
会計・OR・法律

**17 午後の解き方**
分野別の攻略法

**総まとめ**
直前チェック


**ポイント　本書の使い方**

各章は「①図解で全体像 → ②用語の意味から解説 → ③計算例・具体例 → ④過去問ではこう出る → ⑤この章のまとめ」の順に構成されています。1周目はわからない所があっても止まらずに最後まで読み、2周目で「計算例」を自分の手で解き直すと知識が定着します。その後、午前クイズ・過去問演習に進みましょう。

## ■ おすすめの学習スケジュール（3か月の例）

| 期間 | やること | 目安 |
|---|---|---|
| 1か月目 | 本書の第1〜10章（テクノロジ系）を通読し、計算例を手で解く | 1日1〜1.5時間 |
| 2か月目 | 第11〜16章（マネジメント・ストラテジ系）を通読。並行して午前過去問を1日20問 | 1日1.5時間 |
| 3か月目 | 第17章を読み、午後の選択分野を決めて過去問演習（1日1問）。直前は総まとめで復習 | 1日2時間 |

## ■ 過去問題の入手と利用について

IPAの公式サイト「過去問題」のページでは、過去の試験の問題冊子・解答例・採点講評がPDFで無料公開されています。本書の「過去問ではこう出る」では、令和7年度（2025年度）春期・秋期を中心に、実際にどんなテーマが問われたかを紹介しています（問題文そのものは掲載していません）。実際の問題は必ずIPAのサイトで確認し、本書で学んだ知識を使って解いてみてください。

**用語メモ　出典**

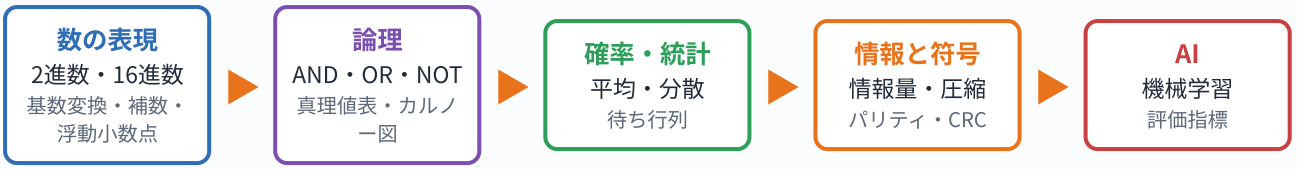
本書で言及している過去問題は、独立行政法人 情報処理推進機構（IPA）が公開している「応用情報技術者試験」の過去問題（例：令和7年度秋期 午前 問1）です。表記は「R7秋 午前問1」のように略します。

# 第1章 基礎理論

コンピュータは、電気が「流れている／流れていない」という2つの状態だけで、計算も画像も音楽も扱っています。この章では、その「2つの状態」を使って数や論理を表す方法から始め、確率・統計、待ち行列、誤り検出、AIまで、試験の問1〜問4あたりで毎回出題される基礎理論をまとめて学びます。最初は抽象的に感じても、具体例で手を動かせば必ず理解できます。

**第1章① この章で学ぶこと**

「数の表し方」→「論理」→「確率・統計」→「情報の伝え方」→「AI」




**出題傾向**　午前の問1〜問4は、ほぼ毎回この章の範囲です。計算問題が多いので、手順を覚えれば確実に得点源にできます。

## 1. 数の表し方 — 2進数・16進数

### ■ 基数（きすう）とは

私たちが普段使っている数は **10進数** です。0〜9の10種類の数字を使い、9の次で桁が1つ上がります（桁上がり）。この「何種類の数字を使うか」を **基数** といいます。

- **10進数**：基数10。0〜9を使う。
- **2進数**：基数2。0と1だけを使う。1の次で桁上がりして「10」になる。コンピュータ内部はすべて2進数。
- **16進数**：基数16。0〜9とA〜F（A=10, B=11, …, F=15）を使う。2進数4桁をまとめて1桁で書けるので、人間が2進数を読みやすくするために使う。

> **用語メモ　ビットとバイト**
> 2進数の1桁を **ビット（bit）** といいます。8ビットをまとめて **1バイト（byte）** と呼びます。nビットで表せるパターンは **$2^n$ 通り** です（1ビット＝2通り、8ビット＝256通り）。この「nビットで$2^n$通り」は試験全体で何度も使う基本中の基本です。

### ■ 桁の重み

10進数の「352」は、3×100＋5×10＋2×1 という意味です。100, 10, 1 はそれぞれ $10^2$, $10^1$, $10^0$ で、これを **桁の重み** といいます。2進数では重みが $2^n$ になります。

| 桁 | $2^7$ | $2^6$ | $2^5$ | $2^4$ | $2^3$ | $2^2$ | $2^1$ | $2^0$ | . | $2^{-1}$ | $2^{-2}$ | $2^{-3}$ |
|---|---|---|---|---|---|---|---|---|---|---|---|---|
| 重み | 128 | 64 | 32 | 16 | 8 | 4 | 2 | 1 | . | 0.5 | 0.25 | 0.125 |

### ■ 基数変換（2進数 ⇔ 10進数 ⇔ 16進数）

**計算例　2進数 → 10進数（重みを足す）**

2進数 1011.01 を10進数にする。
1が立っている桁の重みを足すだけ：8＋2＋1＋0.25 ＝ **11.25**

**計算例　10進数 → 2進数（整数部は2で割る、小数部は2を掛ける）**

**整数部 13**：2で割り続け、余りを下から読む。13÷2=6余り1、6÷2=3余り0、3÷2=1余り1、1÷2=0余り1 → 下から読んで **1101**
**小数部 0.375**：2を掛け、整数部に出た数字を上から読む。0.375×2=**0**.75、0.75×2=**1**.5、0.5×2=**1**.0 → **0.011**

**計算例　2進数 ⇔ 16進数（4桁ずつ区切る）**

2進数 1011 1110 → 1011＝11＝B、1110＝14＝E → 16進数 **BE**
逆に16進数 3F → 3＝0011、F＝1111 → **0011 1111**
8進数の場合は3桁ずつ区切ります。

**ひっかけ注意　10進数の小数は2進数で正確に表せないことがある**

10進数の 0.1 を2進数にすると 0.000110011… と無限に続きます（循環小数）。つまり「0.1を10回足しても正確に1にならない」ことがあります。分母が2のべき乗（1/2, 1/4, 1/8…）の組合せで表せる小数だけが、2進数で有限桁になります。「有限桁で表せるのはどれか」という問題では、0.5, 0.25, 0.125, 0.375 などを選びます。

## 2. 負の数の表し方 ― 補数

### ■ 補数とは

コンピュータには「−（マイナス）」の記号がありません。そこで、負の数を **補数** という形で表します。8ビットの場合、**2の補数** は次の手順で作ります。

- **手順①** ビットをすべて反転する（0⇔1）。これを **1の補数** という。
- **手順②** 1を足す。これが **2の補数**。

**計算例　−5 を8ビットの2の補数で表す**

＋5 ＝ 0000 0101 → 反転 1111 1010 → ＋1 → **1111 1011**（これが −5）
確かめ：0000 0101 ＋ 1111 1011 ＝ 1 0000 0000 となり、8ビットからはみ出た1を捨てると0。つまり「足すと0になる数」＝マイナスの数として正しく働きます。

- 2の補数を使うと、**引き算を足し算で計算できる**（A − B ＝ A ＋ Bの2の補数）。回路が1種類で済むので、コンピュータは2の補数を使う。
- 先頭ビット（最上位ビット：MSB）が **0なら正、1なら負**（符号ビット）。
- nビットで表せる範囲：**$-2^{n-1}$ ～ $2^{n-1}-1$**。8ビットなら −128～＋127。負の方が1つ多い点に注意。

# 3. 小数の表し方と誤差

## ■ 固定小数点数と浮動小数点数

- **固定小数点数**：小数点の位置を固定する方式。整数を表すのによく使う。
- **浮動小数点数**：数を「仮数 × 基数$^{指数}$」の形で表す方式。例えば 123.45 ＝ 1.2345 × $10^2$。非常に大きな数や小さな数を少ないビット数で表せる。

**浮動小数点数（IEEE 754 単精度 32ビット）の構造**

| 符号部 | 指数部 | 仮数部 |
|---|---|---|
| 1ビット<br>0:正 1:負 | 8ビット<br>2の何乗か（＋127のげたばき） | 23ビット<br>1.xxxx の「xxxx」部分だけを格納（先頭の1は省略） |

値 ＝ $(-1)^{符号} \times 1.仮数 \times 2^{指数-127}$

> **用語メモ　正規化**
>
> 仮数部の先頭に余計な0が並ばないように、小数点の位置を調整することを **正規化** といいます。例：$0.00101 \times 2^0$ → $1.01 \times 2^{-3}$。正規化すると有効桁数を最大限使えるので精度が上がります。

## ■ 誤差の種類

コンピュータで扱う桁数は有限なので、計算結果にはどうしても誤差が生まれます。試験では次の4種類の名前と意味が問われます。

| 誤差 | 意味 | 例 |
|---|---|---|
| **丸め誤差** | 表せる桁数を超えた部分を四捨五入・切捨てしたことで生じる誤差 | 0.1 を有限ビットで表す |
| **打切り誤差** | 無限に続く計算を途中で打ち切ったことで生じる誤差 | 級数計算を10項で止める |
| **桁落ち** | **値がほぼ等しい数どうしを引き算**して、有効数字が急に少なくなる | 1.2345 − 1.2344 ＝ 0.0001（有効数字1桁に） |
| **情報落ち** | **絶対値が大きく異なる数を足し引き**して、小さい方の数が結果に反映されない | 10000000 ＋ 0.0001 |

> **ポイント　桁落ちと情報落ちの区別**
>
> 「近い数の **引き算**」＝桁落ち、「大きさが **大きく違う** 数の足し算」＝情報落ち。情報落ちを防ぐには、**絶対値の小さい数から順に足す** のが定番の対策です。

## ■ シフト演算

2進数のビット列を左右にずらす演算を **シフト** といいます。**左に1ビットずらすと2倍、右に1ビットずらすと1/2** になります（10進数で0を付け足すと10倍になるのと同じ）。

- **論理シフト**：空いたビットに常に0を入れる。符号を考えない。
- **算術シフト**：符号ビットを固定したままずらす。右シフトでは空いたところに符号ビットと同じ値を入れる。負の数の2倍・1/2に使う。

> **計算例　シフトと足し算で「×10」を作る**
> x を10倍したい → 10x ＝ 8x ＋ 2x ＝（xを3ビット左シフト）＋（xを1ビット左シフト）。掛け算回路を使わずに高速に計算できるため、試験でもよく問われます。

## 4. 論理演算 — 論理変数と真理値表

### ■ 論理変数とは

**論理変数** とは、**「真（1）」か「偽（0）」のどちらか一方の値だけをとる変数** のことです。「ドアが開いている」「スイッチがONである」のような、YES/NOで答えられる状態を表します。例えば A＝「雨が降っている」とすると、A は 1（降っている）か 0（降っていない）のどちらかです。

論理変数どうしを組み合わせる演算を **論理演算** といい、その結果を表で全パターン書き出したものを **真理値表** といいます。

**第1章② 基本の論理演算と真理値表**

| A | B | A・B（AND：論理積） | A＋B（OR：論理和） | A ⊕ B（XOR：排他的論理和） | NAND | NOR |
|---|---|---|---|---|---|---|
| 0 | 0 | 0 | 0 | 0 | 1 | 1 |
| 0 | 1 | 0 | 1 | 1 | 1 | 0 |
| 1 | 0 | 0 | 1 | 1 | 1 | 0 |
| 1 | 1 | 1 | 1 | 0 | 0 | 0 |

**AND（・ ∧）**
両方1のときだけ1。「かつ」

**OR（＋ ∨）**
どちらかが1なら1。「または」

**NOT（¯¬）**
0と1を反転。Ā と書く

**XOR（⊕）**
2つが異なるときだけ1

- **NAND** ＝ NOT（AND）、**NOR** ＝ NOT（OR）。NANDだけ（またはNORだけ）を組み合わせれば、すべての論理回路を作れる。
- **XOR** は「同じなら0、違えば1」。足し算の1桁目（半加算器の和）や、誤り検出（パリティ）、暗号で使われる。
- 記号の書き方は問題によって異なる（・と∧、＋と∨など）。**問題文の注記を必ず確認** する。

### ■ 含意（→）

「P → Q」は「PならばQ」という意味の論理演算です。**「真 → 偽」のときだけ偽、それ以外はすべて真** になります。「Pが偽のときは、Qが何であっても真」という点が直感に反するので要注意です。

| P | Q | P → Q |
|---|---|---|
| 0 | 0 | 1 |
| 0 | 1 | 1 |
| 1 | 0 | **0** |
| 1 | 1 | 1 |

**用語メモ　含意の書き換え**

P → Q ＝ P̄ ＋ Q（「Pでない、またはQ」）と同じです。真理値表を書けば一致することを確かめられます。含意が出てきたら、この形に書き換えると計算が楽になります。

### ■ 論理演算の法則（ド・モルガンの法則など）

| 法則 | 式 |
|---|---|
| ド・モルガンの法則 | $\overline{A \cdot B} = \bar{A} + \bar{B}$ ／ $\overline{A+B} = \bar{A} \cdot \bar{B}$ |
| 分配法則 | A・(B＋C) ＝ A・B ＋ A・C ／ A＋(B・C) ＝ (A＋B)・(A＋C) |
| 吸収法則 | A ＋ A・B ＝ A ／ A・(A＋B) ＝ A |
| 補元 | A ＋ Ā ＝ 1 ／ A・Ā ＝ 0 |
| べき等 | A ＋ A ＝ A ／ A・A ＝ A |

**ポイント　ド・モルガンは「線を切ったら記号をひっくり返す」**

全体にかかった否定（上の線）を分けるときは、線を各変数に配り、・と＋を入れ替えます。午前問題でも午後のプログラミング問題（条件の否定）でも頻出です。

**過去問ではこう出る　R7春 午前問1**

論理式 P, Q がいずれも真であるとき、R の値にかかわらず真になる式を選ぶ問題。含意（→）の定義「真→偽のときだけ偽」を使い、P＝1, Q＝1 を代入して、R＝0 と R＝1 の両方で真になる式を探します。代入して1つずつ確かめるのが確実な解き方です。

## 5. カルノー図 — 論理式を簡単にする道具

### ■ カルノー図とは

**カルノー図** とは、**真理値表を2次元の表の形に並べ替えて、論理式を簡単にする（簡単化する）ための図** です。真理値表のままだと、どの項をまとめられるかが見えにくいのですが、カルノー図にすると「隣り合う1のかたまり」として目で見て見つけられます。

カルノー図の最大の特徴は、行と列の並び順が **00 → 01 → 11 → 10** になっていることです（2進数の順番 00, 01, 10, 11 ではありません）。この並びは **隣どうしで1ビットしか変化しない**（グレイコード）ので、隣り合うマスは「1つの変数だけが違う」関係になります。

**第1章③ カルノー図の読み方（4変数 A, B, C, D）**

行は AB、列は CD。マスの中は、その組合せのときの論理式の値

| AB＼CD | 00 | 01 | 11 | 10 |
|---|---|---|---|---|
| **00** | **1** | 0 | 0 | **1** |
| **01** | 0 | 0 | 0 | 0 |
| **11** | 0 | **1** | **1** | 0 |
| **10** | **1** | 0 | 0 | **1** |

オレンジ：四隅の4マス ／ 青：横に並んだ2マス

**ルール**

- 1のマスを、**1, 2, 4, 8個**（2のべき乗）の長方形でまとめる
- できるだけ **大きく**、できるだけ **少ない** グループで全部の1を囲む
- **表の端どうしもつながっている**（左端と右端、上端と下端、四隅も）
- グループ内で **値が変わらない変数だけ** を残す

**計算例　上のカルノー図から論理式を求める**

**①オレンジ（四隅）**：AB が 00 と 10 → A は0と1で変化、**B は常に0**。CD が 00 と 10 → C は変化、**D は常に0**。変化しない B＝0, D＝0 が残るので **$\bar{B}\cdot\bar{D}$**。

**②青（AB＝11, CD＝01と11）**：A＝1, B＝1 で固定。CD が 01 と 11 → C は変化、**D＝1** で固定。よって **$A\cdot B\cdot D$**。

**③2つを＋（OR）でつなぐ**：$f = \bar{B}\cdot\bar{D} + A\cdot B\cdot D$

1が6個ある真理値表をそのまま式にすると6個の項になりますが、カルノー図を使えば2項まで簡単にできました。

- 0のマスを無視して1だけを見る。「1のある場所の条件」を OR でつないだ式になる。
- 2マスのグループ → 変数が1つ消える。4マス → 2つ消える。8マス → 3つ消える。
- 選択肢から選ぶ問題では、**選択肢の式に具体的な値を代入して、カルノー図のマスと一致するか確かめる** 方法も有効（例：A=0,B=0,C=0,D=0 で1になるか？）。

**過去問ではこう出る　R7秋 午前問1**

4変数 A, B, C, D のカルノー図が与えられ、等価な論理式を選ぶ問題。上と同じ手順で「隣り合う1をまとめ、変化しない変数を残す」ことで解けます。四隅や上下の端がつながる点を見落とさないことがポイントです。

### ■ 論理回路

論理演算を電子回路で実現したものを **論理回路** といいます。午前ではMIL記号（回路図の記号）で書かれた回路の出力を求める問題が出ます。

- **半加算器**：1ビットどうしの足し算。和 S ＝ A ⊕ B（XOR）、桁上がり C ＝ A・B（AND）。
- **全加算器**：下の桁からの桁上がりも含めて足す回路。半加算器2つとORで作る。
- **フリップフロップ**：1ビットを記憶できる回路。SRAMやレジスタに使われる（第3章）。

## 6. 集合とベン図

**集合** とは「ものの集まり」です。集合どうしの関係を円で表した図を **ベン図** といいます。論理演算と1対1で対応しているので、セットで覚えましょう。

| 集合演算 | 記号 | 意味 | 論理演算 |
|---|---|---|---|
| 積集合（共通部分） | A ∩ B | AとBの両方に入っている | AND |
| 和集合 | A ∪ B | AかBの少なくとも一方に入っている | OR |
| 補集合 | $\bar{A}$ | Aに入っていない | NOT |
| 差集合 | A − B | Aに入っていてBに入っていない | $A \cdot \bar{B}$ |
| 対称差 | A △ B | どちらか一方だけに入っている | XOR |

## 7. 確率と統計

### ■ 場合の数（順列と組合せ）

- **順列** ${}_nP_r$：n個から r個を選んで **並べる**（順番を区別する）方法の数。${}_nP_r = n \times (n-1) \times \cdots$（r個掛ける）。例：${}_5P_2 = 5 \times 4 = 20$。
- **組合せ** ${}_nC_r$：n個から r個を **選ぶだけ**（順番を区別しない）の方法の数。${}_nC_r = {}_nP_r \div r!$。例：${}_5C_2 = 20 \div 2 = 10$。

### ■ 確率

- **確率** ＝（その事象が起こる場合の数）÷（起こりうるすべての場合の数）
- **AかつB**（独立なとき）：P(A) × P(B)　／　**AまたはB**（同時に起こらないとき）：P(A) ＋ P(B)
- **少なくとも1回起こる確率** ＝ 1 −（1回も起こらない確率）。「余事象」を使うと計算が楽。

> **計算例　少なくとも1つ故障する確率**
> 故障率 0.1 の装置が2台ある。少なくとも1台が故障する確率は？
> 1台も故障しない確率 ＝ 0.9 × 0.9 ＝ 0.81 → 求める確率 ＝ 1 − 0.81 ＝ **0.19**

### ■ 統計の基本用語

| 用語 | 意味 |
|---|---|
| 平均（算術平均） | データの合計 ÷ データの個数 |
| メジアン（中央値） | 小さい順に並べたときの真ん中の値 |
| モード（最頻値） | 最も多く現れる値 |
| 分散 | 「各データ − 平均」を2乗して平均したもの。ばらつきの大きさ |
| 標準偏差 | 分散の平方根。元のデータと同じ単位でばらつきを表す |
| 正規分布 | 平均を中心とした左右対称の釣鐘型の分布。平均±1σ（標準偏差）に約68%、±2σに約95%、±3σに約99.7%が入る |
| 相関係数 | 2つの変数の関係の強さ（−1〜1）。1に近いほど正の相関、0なら相関なし。相関は因果関係を意味しない |
| 回帰分析 | ある変数から別の変数を予測する式（回帰直線など）を求める手法 |

## ■ 待ち行列理論（M/M/1）

**待ち行列** とは、銀行のATMやレジのように「サービスを受けるために並ぶ列」のことです。コンピュータでは「処理を待つデータやジョブの列」にあたります。**M/M/1モデル** は、待ち行列の最も基本的なモデルで、次の条件を満たすものです。

- 1つ目のM：客が **ランダムに到着**（ポアソン分布に従う到着）
- 2つ目のM：処理時間が **ランダム**（指数分布に従うサービス時間）
- 1：窓口（サーバ）が **1つ**
- 待ち行列の長さに制限はなく、先着順に処理する

**用語メモ　ポアソン分布と指数分布**

「ポアソン分布に従って到着する」「指数分布に従う処理時間」は、どちらも「到着や処理がランダムに起こる」という意味だと理解すれば十分です。試験でこの言葉が出てきたら「M/M/1の公式が使えるサイン」と読み取りましょう。

### 第1章④ M/M/1 待ち行列の公式

まず利用率 ρ（ロー）を求める → 公式に当てはめる

**利用率 ρ**

ρ ＝ 平均到着率 × 平均サービス時間
＝ 平均サービス時間 ÷ 平均到着間隔
（窓口が忙しい割合。0〜1未満）

**平均待ち時間 W**

W ＝ ρ ÷ (1 − ρ) × 平均サービス時間
（列に並んでいる時間だけ）

**平均応答時間**

＝ 平均待ち時間 ＋ 平均サービス時間
＝ 1 ÷ (1 − ρ) × 平均サービス時間

**イメージ**　利用率が 0.5 → 待ち時間はサービス時間の1倍、0.8 → 4倍、0.9 → 9倍。利用率が1に近づくと待ち時間は急激に長くなります。

**計算例　待ち時間の計算**

1件の処理に平均4秒かかる窓口に、1分間に平均12件のデータが到着する。平均待ち時間は？

①**利用率**：平均到着間隔 ＝ 60秒 ÷ 12件 ＝ 5秒。ρ ＝ 4 ÷ 5 ＝ **0.8**

②**平均待ち時間**：0.8 ÷ (1 − 0.8) × 4秒 ＝ 4 × 4 ＝ **16秒**

③**平均応答時間**：16 ＋ 4 ＝ **20秒**

**過去問ではこう出る　R7秋 午前問2**

M/M/1 モデルで「平均待ち時間が平均処理時間 T 以上になるのは、利用率が何%以上のときか」を問う問題。W ＝ ρ/(1−ρ) × T ≧ T より ρ/(1−ρ) ≧ 1 を解けば ρ ≧ 0.5 となります。公式を「待ち時間はサービス時間の ρ/(1−ρ) 倍」と覚えておけば、一瞬で解けます。

# 8. 情報量・符号化・誤り検出

## ■ 情報量とデータ圧縮

- **情報量** の単位はビット。起こる確率が P の出来事の情報量は **$-\log_2 P$** ビット。めったに起こらないこと（Pが小さい）ほど情報量が大きい。
- **ハフマン符号**：よく出てくる文字には短い符号、めったに出ない文字には長い符号を割り当てて、全体のデータ量を小さくする圧縮方式。
- **ランレングス符号**：同じ値が続く部分を「値と繰返し回数」で表す圧縮方式（例：AAAAABB → A5B2）。

> **計算例　ハフマン符号の平均符号長**
>
> 文字 A, B, C, D の出現確率が 0.5, 0.25, 0.125, 0.125 で、符号を A=0, B=10, C=110, D=111 とする。
>
> 平均符号長 ＝ 0.5×1 ＋ 0.25×2 ＋ 0.125×3 ＋ 0.125×3 ＝ **1.75ビット**（固定長なら2ビット必要）

## ■ 誤り検出・訂正

通信や記録の途中でビットが反転してしまう誤りを見つけたり、直したりする仕組みです。

| 方式 | しくみ | 能力 |
|---|---|---|
| **パリティチェック** | データの1の個数が偶数（または奇数）になるように1ビット（パリティビット）を付ける | 1ビットの誤りを **検出** できる。訂正はできない。2ビット誤りは見逃す |
| **ハミング符号** | 複数の検査ビットを付け、どの位置が誤ったかを特定する | 1ビットの誤りを **訂正**、2ビットの誤りを **検出** できる。ECCメモリで使用 |
| **CRC（巡回冗長検査）** | データを **生成多項式** で割り算し、その余りを検査用データとして付けて送る | 連続した誤り（バースト誤り）の検出に強い。通信で広く使われる |
| **チェックサム** | データを一定単位で足し合わせた値を付ける | 簡易な誤り検出 |

> **過去問ではこう出る　R7秋 午前問4**
>
> CRC に関する正しい記述を選ぶ問題。CRC は「**送信側**が生成多項式を使ってデータから検査用データ（余り）を作り、データに付けて送る。**受信側**も **同じ生成多項式** で割り算し、余りが0なら誤りなしと判断する」仕組みです。「検査用データは1ビット」「送信側と受信側で異なる多項式」などは誤りです。

# 9. オートマトン・形式言語・逆ポーランド記法

- **有限オートマトン**：「状態」と「入力による状態の移り変わり（状態遷移）」でモデル化したもの。状態遷移図や状態遷移表で表す。入力文字列を最後まで読んだときに「受理状態」にいれば、その文字列を受理する。
- **BNF（バッカス・ナウア記法）**：プログラム言語の文法を定義する記法。`<数字> ::= 0|1|2|…|9` のように書く。「::=」は「定義する」、「|」は「または」。
- **逆ポーランド記法（後置記法）**：演算子をオペランド（数）の後ろに書く記法。(A＋B)×C → **AB＋C×**。かっこが不要で、スタックを使えば左から順に計算できるのでコンピュータ向き。

**計算例**　**逆ポーランド記法の計算（スタックを使う）**

「3 4 ＋ 2 ×」を計算する。数はスタックに積み、演算子が来たら2つ取り出して計算し、結果を積む。

3を積む → 4を積む → ＋：4と3を取り出し 7 を積む → 2を積む → ×：2と7を取り出し **14** を積む

## 10. 数値計算 — 二分法

**二分法** とは、方程式 f(x)＝0 の解（グラフがx軸と交わる点）を、範囲を半分ずつ絞り込んで近似的に求める方法です。解がある範囲 $[x_0, x_1]$ の真ん中を調べ、解が左右どちらにあるかを判定して、範囲を半分にすることを繰り返します。

- 1回の繰返しで範囲は 1/2 になる → n 回で $1/2^n$ になる。
- 範囲 1 を 0.001 未満まで絞るには、$1/2^n < 0.001 \rightarrow 2^n > 1000 \rightarrow$ **n ＝ 10**（$2^{10} = 1024$）。

**ポイント**　**2<sup>10</sup> ≒ 1000 は必ず覚える**

$2^{10} = 1024 \fallingdotseq 10^3$（キロ）、$2^{20} \fallingdotseq 10^6$（メガ）、$2^{30} \fallingdotseq 10^9$（ギガ）。二分法、二分探索の比較回数、アドレス空間の計算など、あらゆる場面で使います。

**過去問ではこう出る**　**R7春 午前問2**

単調増加の関数について、二分法の手順のうち中点を求める処理が何回実行されるかを問う問題。区間の幅 1 が 0.001 より小さくなるまでの回数なので、上の計算から約10回と見当を付けられます。

## 11. AI（人工知能）と機械学習

### ■ 機械学習の種類

**機械学習** とは、コンピュータがデータからルール（パターン）を自動的に見つけ出す技術です。

| 種類 | 学習のしかた | 用途の例 |
|---|---|---|
| **教師あり学習** | 正解（ラベル）付きのデータで学習する | 迷惑メール判定、画像分類、売上予測 |
| **教師なし学習** | 正解のないデータから、似たもの同士のグループや構造を見つける | 顧客のクラスタリング（グループ分け） |
| **強化学習** | 試行錯誤し、「報酬」が最大になる行動を学習する | 囲碁・将棋AI、ロボット制御 |

- **ディープラーニング（深層学習）**：人間の脳の神経回路をまねた **ニューラルネットワーク** を何層にも重ねた手法。画像認識・音声認識・生成AIの中核技術。
- **過学習**：学習データに合わせすぎて、未知のデータに対する予測精度が下がること。
- **生成AI**：文章・画像・音声などを新たに生成するAI。**大規模言語モデル（LLM）** はその代表。もっともらしい誤情報を生成する **ハルシネーション** に注意が必要。
- **転移学習**：ある課題で学習済みのモデルを、別の課題に流用する手法。

## ■ 分類モデルの評価指標（混同行列）

「正しい／間違い」の2つに分類するAIの性能は、予測と実際の組合せ4パターン（**混同行列**）で評価します。

| | 予測：陽性（正と予測） | 予測：陰性（誤と予測） |
|---|---|---|
| **実際：陽性** | TP（真陽性）：正しく陽性 | FN（偽陰性）：見逃し |
| **実際：陰性** | FP（偽陽性）：誤って陽性 | TN（真陰性）：正しく陰性 |

| 指標 | 式 | 意味 |
|---|---|---|
| 正解率 | (TP＋TN) ÷ 全体 | 全体のうち正しく予測できた割合 |
| 適合率（精度） | TP ÷ (TP＋FP) | 陽性と予測したもののうち、本当に陽性だった割合 |
| 再現率（真陽性率） | TP ÷ (TP＋FN) | 実際に陽性のもののうち、陽性と予測できた割合 |
| 偽陽性率 | FP ÷ (FP＋TN) | **実際に陰性のもの** のうち、**誤って陽性と予測した** 割合 |
| F値 | 適合率と再現率の調和平均 | 2つのバランスを見る指標 |

**用語メモ　ROC曲線とAUC**

**ROC曲線** は、横軸に偽陽性率、縦軸に真陽性率（再現率）をとり、判定のしきい値を変えながら描いた曲線です。曲線の下側の面積 **AUC** が1に近いほど性能が良いモデルです。

**過去問ではこう出る　R7秋 午前問3**

ROC曲線で使う **偽陽性率** の説明を選ぶ問題。「実際に“間違い”（陰性）であるデータに対し、誤って“正しい”（陽性）と予測した割合」が正解です。分母が「実際の陰性」であることがポイントです。

### この章のまとめ

- nビットで$2^n$通り。2進⇔16進は4桁区切り。負の数は2の補数（反転して＋1）。
- 誤差：近い数の引き算＝桁落ち、大小差のある足し算＝情報落ち（小さい数から足す）。
- 論理変数は0か1だけをとる変数。含意 P→Q は「1→0」のときだけ0。ド・モルガンは記号を入れ替える。
- カルノー図は 00,01,11,10 の順。隣り合う1を2のべき乗個でまとめ、変化しない変数を残す。端はつながる。
- M/M/1：ρ＝到着率×サービス時間、待ち時間＝ρ/(1−ρ)×サービス時間。
- パリティは検出のみ、ハミングは1ビット訂正、CRCは生成多項式の余り。
- 混同行列の各指標は「分母が何か」で区別する。偽陽性率の分母は実際の陰性。

# 第2章 アルゴリズムとプログラミング

**アルゴリズム** とは「問題を解くための手順」のことです。料理のレシピのように、誰がやっても同じ結果になるよう、手順を明確に書いたものです。この章では、データを入れておく「入れ物（データ構造）」と、データを探したり並べ替えたりする代表的な手順を学びます。午前で毎回数問出題されるほか、午後の問3（プログラミング）は丸ごとこの章の知識で解きます。

**第2章① データ構造とアルゴリズムの関係**

「どう入れるか（データ構造）」と「どう処理するか（アルゴリズム）」はセット

| 配列 | リスト | スタック／キュー | 木（ツリー） |
|---|---|---|---|
| 番号（添字）で直接アクセス。<br>挿入・削除は遅い | ポインタでつなぐ。<br>挿入・削除が速い | 後入れ先出し／先入れ先出し | 親子関係。探索・整列に活用 |

**探索**
目的のデータを探す
線形・二分・ハッシュ

＋

**整列**
データを並べ替える
バブル・選択・挿入・クイック・マージ・ヒープ

＝

**計算量**
速さの目安
$O(n)$、$O(\log n)$、$O(n^2)$…

## 1. プログラムの基本要素

### ■ 変数・代入・データ型

- **変数**：データを入れておく名前付きの箱。「x ← 5」は「xに5を入れる（代入する）」という意味（試験の擬似言語では ← を使う）。
- **データ型**：変数に入るデータの種類。整数型、実数型、文字型、論理型（true/false）など。
- **定数**：値を変えない名前付きのデータ。

### ■ 制御構造（構造化プログラミング）

どんなアルゴリズムも、次の3つの組合せで書けます（**構造化定理**）。

**3つの基本構造**

| 順次 | 選択（分岐） | 繰返し（ループ） |
|---|---|---|
| 上から順に1つずつ実行する | 条件によって実行する処理を変える（if ～ else） | 条件を満たす間、同じ処理を繰り返す（while, for） |

**用語メモ　擬似言語の読み方**

試験では特定のプログラム言語ではなく、IPA独自の **擬似言語** で書かれたプログラムが出ます。「if ～ elseif ～ else ～ endif」「while (条件) ～ endwhile」「for (i を 1 から n まで 1 ずつ増やす) ～ endfor」のような日本語混じりの書き方です。配列の添字は **1から始まる** 場合が多いので、問題文の注記を必ず確認しましょう。

### ■ トレース（追跡）

プログラムを1行ずつ実行したつもりで、変数の値の変化を表に書き出していく作業を **トレース** といいます。午後のプログラミング問題では、トレースが最も確実な武器です。

**計算例　合計を求めるプログラムをトレースする**

```
sum ← 0
for (i を 1 から 4 まで 1 ずつ増やす)
    sum ← sum + i
endfor
```

| 繰返し | i | sum（実行後） |
|---|---|---|
| 1回目 | 1 | 1 |
| 2回目 | 2 | 3 |
| 3回目 | 3 | 6 |
| 4回目 | 4 | 10 |

## 2. 基本的なデータ構造

### ■ 配列

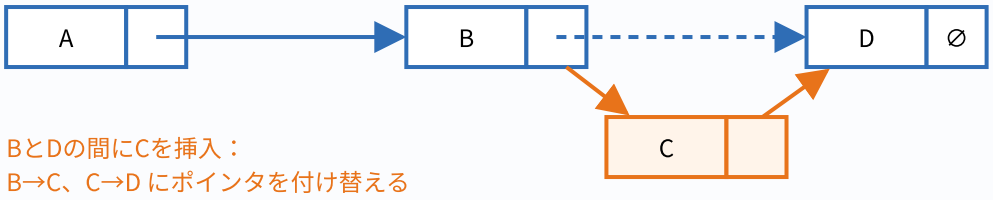
**配列** は、同じ型のデータを連続した領域に並べたものです。**添字（インデックス）** で何番目の要素かを指定して、直接アクセスできます。

- 長所：添字を指定すれば、何番目の要素でも **すぐに読み書きできる**。
- 短所：途中に要素を挿入・削除すると、**後ろの要素を全部ずらす** 必要があり遅い。
- **2次元配列**：行と列で表す表のような配列。メモリ上では1列に並べて格納する（行ごとに並べる「行優先」など）。

### ■ リスト（連結リスト）

**リスト** は、各要素（ノード）が「データ」と「次の要素の場所（**ポインタ**）」を持ち、ポインタで数珠つなぎにしたデータ構造です。

**連結リストの挿入 — ポインタを付け替えるだけ**




- **単方向リスト**：次の要素へのポインタだけを持つ。
- **双方向リスト**：次と前の両方のポインタを持つ。逆向きにもたどれる。
- **循環リスト**：最後の要素が先頭を指す。
- 長所：挿入・削除は **ポインタの付け替えだけ** で済み、データの移動が不要。短所：n番目の要素を見るには **先頭から順にたどる** 必要がある。

## ■ スタックとキュー

**第2章② スタックとキュー**

**スタック（LIFO：後入れ先出し）**
最後に入れたものを最初に取り出す。積み重ねた皿のイメージ。
入れる操作：**PUSH**、取り出す操作：**POP**
用途：関数呼出しの戻り先の記憶、再帰、逆ポーランド記法の計算、深さ優先探索

**キュー（FIFO：先入れ先出し）**
最初に入れたものを最初に取り出す。レジの行列のイメージ。
入れる操作：**ENQUEUE**、取り出す操作：**DEQUEUE**
用途：プリンタの印刷待ち、待ち行列、幅優先探索

## ■ 木構造

**木（ツリー）** は、データを親子関係で階層的に表すデータ構造です。一番上を **根（ルート）**、子を持たない末端を **葉（リーフ）**、各要素を **節（ノード）** と呼びます。

- **2分木**：各ノードの子が最大2つの木。
- **完全2分木**：葉以外のすべての段が埋まっていて、最下段は左から詰まっている2分木。
- **2分探索木**：「左の子孫 ＜ 親 ＜ 右の子孫」というルールを満たす2分木。探索が速い。
- **ヒープ**：「親 ≧ 子」（または「親 ≦ 子」）のルールを満たす完全2分木。根が常に最大（最小）値になる。優先度付きキューやヒープソートに使う。
- **バランス木（B木など）**：木の高さが偏らないように調整する木。データベースの索引（インデックス）に使われる。

> **用語メモ　木のなぞり方（走査）**
> **行きがけ順（先行順）**：親 → 左 → 右　／　**通りがけ順（中間順）**：左 → 親 → 右　／　**帰りがけ順（後行順）**：左 → 右 → 親。2分探索木を通りがけ順でなぞると、データが **小さい順** に取り出せます。数式の木を帰りがけ順でなぞると逆ポーランド記法になります。

### ■ グラフ

点（**頂点・ノード**）と、それらを結ぶ線（**辺・エッジ**）で表したデータ構造です。道路網やネットワークの経路をモデル化します。辺に向きがあるものを **有向グラフ**、重み（距離・コスト）があるものを **重み付きグラフ** といいます。最短経路を求める **ダイクストラ法** が代表的なアルゴリズムです。

## 3. 探索アルゴリズム

**探索** とは、たくさんのデータの中から目的のデータを探し出すことです。

| 方式 | 方法 | 前提条件 | 平均比較回数 | 計算量 |
|---|---|---|---|---|
| **線形探索** | 先頭から順に1つずつ比べる | なし | 約 n/2 回 | O(n) |
| **二分探索** | 真ん中と比べ、探す範囲を半分ずつに絞る | **データが整列済み** | 約 $\log_2 n$ 回 | O(log n) |
| **ハッシュ探索** | ハッシュ関数でデータの格納位置を計算し、直接アクセスする | ハッシュ表に格納済み | 原則1回 | O(1) |

**計算例**　**二分探索の最大比較回数**

1,000個の整列済みデータを二分探索するとき、最大何回比べれば見つかる？

1回比べるごとに範囲が半分 → $2^{10} = 1024 \geqq 1000$ なので、**最大約10回**（正確には $\lfloor \log_2 n \rfloor + 1$ 回）。線形探索なら最大1,000回なので、圧倒的に速い。

**用語メモ**　**ハッシュの衝突（シノニム）**

異なるデータから同じハッシュ値（格納位置）が計算されてしまうことを **衝突（コリジョン）** といい、衝突したデータどうしを **シノニム** といいます。対策には、次の空き場所に入れる **オープンアドレス法** と、同じ位置のデータをリストでつなぐ **チェイン法** があります。

**過去問ではこう出る**　**R7秋 午前問6**

整列済みの表を m 個ずつのブロックに分け、まず各ブロックの最後尾だけを線形探索してブロックを特定し、次にブロック内を線形探索する方式の平均比較回数を問う問題。ブロック数は n/m 個なので1段目の平均は約 (n/m)/2 回、2段目はブロック内 m 個の平均で約 m/2 回、合計で **m/2 ＋ n/(2m)** となります。「線形探索の平均比較回数は個数の半分」を2回使うだけです。

## 4. 整列（ソート）アルゴリズム

**整列（ソート）** とは、データを小さい順（昇順）や大きい順（降順）に並べ替えることです。

### 第2章③ 代表的な整列アルゴリズム

上の3つは単純だが遅い $O(n^2)$、下の3つは速い $O(n \log n)$

| 名前 | しくみ | 平均計算量 | 最悪 |
|---|---|---|---|
| **バブルソート** | 隣どうしを比べて、逆順なら交換する。大きい値が泡のように後ろへ浮かぶ | $O(n^2)$ | $O(n^2)$ |
| **選択ソート** | 未整列部分から最小値を **選んで** 先頭と交換する | $O(n^2)$ | $O(n^2)$ |
| **挿入ソート** | 未整列部分の先頭を、整列済み部分の適切な位置に **挿入** する。ほぼ整列済みのデータに強い | $O(n^2)$ | $O(n^2)$ |
| **クイックソート** | 基準値（ピボット）より小さいグループと大きいグループに分け、それぞれを再帰的に整列する。平均的に最速 | $O(n \log n)$ | $O(n^2)$ |
| **マージソート** | データを半分ずつに分割し続け、整列しながら併合（マージ）する | $O(n \log n)$ | $O(n \log n)$ |
| **ヒープソート** | データをヒープにし、根（最大値）を取り出すことを繰り返す | $O(n \log n)$ | $O(n \log n)$ |
| **シェルソート** | 一定間隔ごとに挿入ソートを行い、間隔を狭めていく（挿入ソートの改良） | 約 $O(n^{1.25})$ | $O(n^2)$ |

**計算例　バブルソートを1周だけトレースする**

[5, 3, 8, 1] を昇順に。隣どうしを左から比べて、左が大きければ交換する。

(5,3)交換 → [3,5,8,1]　(5,8)そのまま → [3,5,8,1]　(8,1)交換 → [3,5,1,**8**]

1周目で最大値8が一番右に確定しました。これを残りの要素で繰り返します。

## 5. 計算量（オーダー記法）

**計算量** は、データ量 n が増えたときに、処理時間がどのくらいの勢いで増えるかを表す目安です。**O（オーダー）記法** で書き、最も影響の大きい項だけを残し、係数は無視します（例：$3n^2 + 5n \rightarrow O(n^2)$）。

| 計算量 | n が 2倍になると | 例 |
|---|---|---|
| $O(1)$ | 変わらない | ハッシュ探索、配列の添字アクセス |
| $O(\log n)$ | 1回分増えるだけ | 二分探索 |
| $O(n)$ | 2倍 | 線形探索 |
| $O(n \log n)$ | 2倍強 | クイックソート、マージソート |
| $O(n^2)$ | 4倍 | バブル・選択・挿入ソート（二重ループ） |
| $O(2^n)$ | 爆発的に増える | 全組合せの総当たり |

**ポイント　ループの形で計算量がわかる**

1重ループ → O(n)、二重ループ → O($n^2$)、毎回半分にする → O(log n)。午後問題で「この処理の計算量は？」と聞かれたら、まずループの入れ子の深さを見ましょう。

## 6. 再帰

**再帰** とは、関数（手続）の中で **自分自身を呼び出す** ことです。大きな問題を「同じ形の小さな問題」に分けて解くときに使います。必ず「これ以上呼び出さずに答えを返す条件（終了条件）」を書きます。

**計算例　階乗を再帰で計算する**

```
○整数型: fact(整数型: n)
  if (n = 0)
    return 1                // 終了条件
  else
    return n × fact(n − 1)  // 自分自身を呼び出す
  endif
```

fact(3) ＝ 3 × fact(2) ＝ 3 × 2 × fact(1) ＝ 3 × 2 × 1 × fact(0) ＝ 3 × 2 × 1 × 1 ＝ **6**
呼出しのたびに戻り先がスタックに積まれ、終了条件に達してから逆順に計算が戻ってきます。

- **再入可能（リエントラント）**：複数のプロセスから同時に呼ばれても正しく動くプログラム。データを呼出しごとに別の領域（スタックなど）に持つ。
- **再帰的**：自分自身を呼び出せるプログラム。
- **再使用可能（リユーザブル）**：一度実行した後、ロードし直さずに再実行できるプログラム。

## 7. プログラム言語とマークアップ言語

### ■ 言語処理の方式

- **コンパイラ**：ソースコード全体をまとめて機械語に翻訳してから実行する。実行が速い（C、C++、Goなど）。
- **インタプリタ**：ソースコードを1行ずつ解釈しながら実行する。手軽だが遅め（Python、JavaScriptなど）。
- **JIT（実行時）コンパイル**：中間コードを実行時に機械語に変換して高速化する（Java、C#など）。

## ■ プログラム言語の特徴（頻出）

| 言語 | 特徴 |
|---|---|
| **Java** | オブジェクト指向。中間コード（バイトコード）をJVM上で実行するので、OSに依存しない |
| **Python** | 動的型付けのインタプリタ言語。AI・データ分析で広く利用 |
| **JavaScript** | Webブラウザ上で動くスクリプト言語。Node.jsでサーバ側でも動く |
| **Scala** | **オブジェクト指向と関数型の両方** のプログラミングが可能。JVM上で動作し、静的型付け |
| **Kotlin** | JVM上で動作。Androidアプリ開発の公式言語 |
| **R** | 統計解析・グラフ描画向けの言語 |
| **Rust** | メモリ安全性を重視したシステムプログラミング言語 |
| **C** | OSや組込み機器の開発に使われる手続き型言語 |

> **過去問ではこう出る　R7秋 午前問7**
>
> OSS として公開されている言語 Scala の特徴を問う問題。「オブジェクト指向プログラミングと関数型プログラミングの両方が可能」が正解です。多重継承や動的型付けではない点に注意します。

## ■ マークアップ言語・データ記述言語

- **HTML**：Webページの構造を記述する。
- **XML**：タグを自由に定義できる。データ交換に使う。
- **JSON**：JavaScriptのオブジェクト表記を基にした、軽量なデータ記述形式。`{"name": "山田", "age": 30}` のように書く。Web API で標準的に使われる。
- **YAML**：インデントで構造を表す、人間が読みやすいデータ記述形式。設定ファイルに多用される。

### この章のまとめ

- 構造化定理：順次・選択・繰返しで何でも書ける。午後問3はトレースが最強の武器。
- 配列は直接アクセスが速く挿入削除が遅い。リストはその逆。
- スタック＝LIFO（再帰・関数呼出し）、キュー＝FIFO（待ち行列）。
- 二分探索は整列済みが前提で O(log n)。ハッシュは O(1)、衝突対策も押さえる。
- 単純ソートは $O(n^2)$、クイック・マージ・ヒープは O(n log n)。クイックの最悪は $O(n^2)$。
- 再帰には必ず終了条件。呼出しのたびにスタックを使う。

# 第3章 コンピュータ構成要素

コンピュータの中身は、大きく **5つの装置**（制御・演算・記憶・入力・出力）でできています。この章では、その中心である CPU（プロセッサ）がどのように命令を実行し、どうやって高速化しているのか、そしてメモリやディスクとの間でどのようにデータをやり取りしているのかを学びます。計算問題（CPI、MIPS、キャッシュの実効アクセス時間など）が頻出です。




## 1. CPU の構成と命令の実行

### ■ CPU の中身

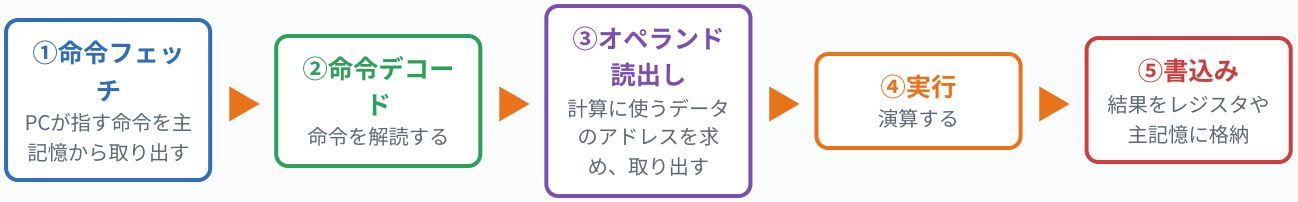
- **制御装置**：主記憶から命令を取り出し、解読して、各装置に指示を出す。
- **演算装置（ALU）**：足し算・論理演算などの計算をする。
- **レジスタ**：CPU内部の超高速な小さい記憶場所。主なものは次のとおり。

| レジスタ | 役割 |
|---|---|
| プログラムカウンタ（PC） | **次に実行する命令のアドレス** を保持する |
| 命令レジスタ | 取り出した命令そのものを保持する |
| 汎用レジスタ／アキュムレータ | 演算対象のデータや結果を保持する |
| インデックスレジスタ | アドレス計算に使う値（添字）を保持する |
| フラグレジスタ（状態レジスタ） | 演算結果がゼロ、負、桁あふれなどの状態を保持する |

### ■ 命令実行の流れ

CPUは1つの命令を、次のステージに分けて実行します。

**命令実行サイクル**




### ■ アドレス指定方式

命令の中の「アドレス部」の値から、実際にデータがある場所（**実効アドレス**）を求める方法です。

| 方式 | 実効アドレスの求め方 |
|---|---|
| 即値アドレス指定 | アドレス部の値そのものがデータ（アドレスではない） |
| 直接アドレス指定 | アドレス部の値 ＝ 実効アドレス |
| 間接アドレス指定 | アドレス部が指す場所に書かれている値 ＝ 実効アドレス |
| インデックス（指標）アドレス指定 | アドレス部 ＋ インデックスレジスタの値 |
| ベースアドレス指定 | アドレス部 ＋ ベースレジスタの値 |
| 相対アドレス指定 | アドレス部 ＋ プログラムカウンタの値 |

## 2. CPU の性能指標

- **クロック周波数**：CPUが動作するリズムの速さ。1GHz なら1秒間に10億回刻む。1クロックの時間（**クロックサイクル時間**）は 1 ÷ 周波数。1GHz なら 1ナノ秒。
- **CPI（Cycles Per Instruction）**：1命令の実行に必要な平均クロック数。
- **MIPS（Million Instructions Per Second）**：1秒間に何百万命令を実行できるか。
- **FLOPS**：1秒間に何回の浮動小数点演算ができるか。スーパーコンピュータの性能指標。

> **計算例　MIPS の計算**
>
> クロック周波数 2GHz、平均 CPI が 4 の CPU の性能は何 MIPS？
> 1秒間のクロック数 ＝ 2,000,000,000 回。1命令 ＝ 4 クロックなので、1秒間の命令数 ＝ 2,000,000,000 ÷ 4 ＝ 500,000,000 ＝ **500 MIPS**
> 1命令の平均実行時間 ＝ 1 ÷ 5億 ＝ **2ナノ秒**

> **計算例　命令ミックスから平均 CPI を求める**
>
> 命令Aが CPI 2 で出現率60%、命令Bが CPI 5 で出現率40%。
> 平均 CPI ＝ 2 × 0.6 ＋ 5 × 0.4 ＝ 1.2 ＋ 2.0 ＝ **3.2**

**過去問ではこう出る　R7秋 午前問11**

HPC（高性能計算）マシンの構成を更新したときのノード数と総理論ピーク演算性能を求める問題。「1コアの性能 × 1ノードのコア数 × ノード数 ＝ 総性能」という掛け算の関係を整理すれば解けます。単位（GFLOPS → TFLOPS は1,000倍）にも注意します。

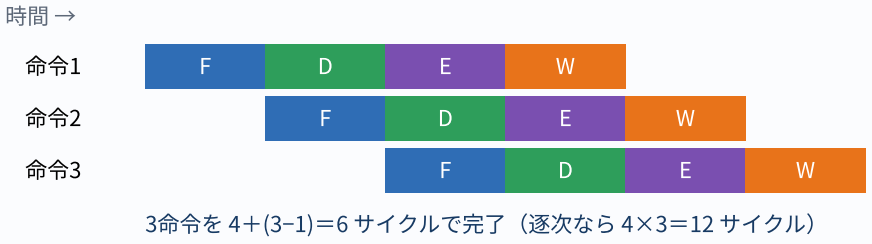
## 3. CPU の高速化技術

### ■ パイプライン

**パイプライン** とは、命令実行の各ステージを流れ作業のように重ねて、複数の命令を少しずつずらして同時に処理する方式です。工場のベルトコンベアと同じ考え方です。

**第3章② パイプライン処理のイメージ（4ステージ）**

F:フェッチ D:デコード E:実行 W:書込み




3命令を 4＋(3−1)＝6 サイクルで完了（逐次なら 4×3＝12 サイクル）

**計算例　パイプラインの実行時間**

ステージ数 k、命令数 n のとき、所要サイクル数 ＝ **k ＋ (n − 1)**。
5ステージで10命令なら 5 ＋ 9 ＝ 14 サイクル（パイプラインなしなら 50 サイクル）。

**用語メモ　パイプラインハザード**

パイプラインの流れが乱れて性能が落ちることを **ハザード** といいます。
**構造ハザード**：同じ装置を同時に使おうとして衝突する。**データハザード**：前の命令の結果を次の命令が使うため待たされる。**制御ハザード**：分岐命令で次に実行する命令が決まらず、先読みした命令が無駄になる。対策として **分岐予測** や **投機実行** があります。

## ■ その他の高速化技術

| 技術 | 内容 |
|---|---|
| **スーパースカラ** | パイプラインを複数本持ち、同時に複数の命令を実行する |
| **スーパーパイプライン** | パイプラインのステージをさらに細かく分割し、クロックを速くする |
| **VLIW** | **コンパイルの段階で**、同時に実行できる複数の命令を1つの長い命令にまとめる |
| **マルチコア** | 1つのCPUの中に複数のコア（演算の中核）を持ち、並列処理する |
| **SIMD** | 1つの命令で複数のデータを同時に処理する（画像処理、GPU） |
| **MIMD** | 複数のプロセッサがそれぞれ別の命令で別のデータを処理する |

**過去問ではこう出る　R7秋 午前問8**

「同時に実行可能な複数の動作を、コンパイルの段階でまとめて一つの複合命令とする方式」を選ぶ問題。キーワード「コンパイルの段階でまとめる」→ **VLIW**（Very Long Instruction Word）です。

## ■ RISC と CISC

| | RISC（縮小命令セット） | CISC（複合命令セット） |
|---|---|---|
| 命令の種類 | 少なく単純 | 多く複雑 |
| 命令の長さ | 固定長 | 可変長 |
| 制御方式 | **ワイヤードロジック**（回路で直接実行） | **マイクロプログラム** |
| 特徴 | パイプラインと相性が良い。スマホのARMなど | 1命令で複雑な処理ができる。x86など |

## ■ マルチプロセッサとアムダールの法則

- **密結合マルチプロセッサ**：複数のプロセッサが主記憶を共有する。
- **疎結合マルチプロセッサ**：プロセッサごとに主記憶を持ち、通信で連携する。
- **アムダールの法則**：プログラムのうち並列化できない部分があると、プロセッサを増やしても性能向上には限界がある、という法則。

**計算例　アムダールの法則**

処理の80%が並列化可能。プロセッサを4台にすると何倍速くなる？

所要時間 ＝ 並列化できない20% ＋ 並列化できる80%÷4 ＝ 0.2 ＋ 0.2 ＝ 0.4 → 速度は 1 ÷ 0.4 ＝ **2.5倍**（4倍にはならない）

# 4. メモリ

## ■ 半導体メモリの種類

| 種類 | 特徴 | 用途 |
|---|---|---|
| **DRAM** | コンデンサに電荷をためて記憶。**リフレッシュ（再書込み）が必要**。安価で大容量 | 主記憶 |
| **SRAM** | **フリップフロップ** 回路で記憶。リフレッシュ不要。高速だが高価 | キャッシュメモリ |
| **フラッシュメモリ** | 電源を切っても内容が消えない（**不揮発性**）。ブロック単位で消去 | SSD、USBメモリ |
| **ROM** | 読出し専用。不揮発性 | ファームウェア |

- **揮発性**：電源を切ると内容が消える（DRAM, SRAM）。**不揮発性**：消えない（フラッシュ, ROM）。

## ■ キャッシュメモリ

**キャッシュメモリ** は、CPUと主記憶の速度差を埋めるために間に置く、高速・小容量のメモリです。よく使うデータをキャッシュに置いておき、CPUはまずキャッシュを探します。キャッシュにデータがあることを **ヒット**、ないことを **ミス** といい、ヒットする割合を **ヒット率** といいます。

> **計算例　実効アクセス時間**
>
> キャッシュのアクセス時間 10ns、主記憶のアクセス時間 60ns、ヒット率 90% のとき、平均（実効）アクセス時間は？
> 実効アクセス時間 ＝ キャッシュ時間 × ヒット率 ＋ 主記憶時間 × (1 − ヒット率)
> ＝ 10 × 0.9 ＋ 60 × 0.1 ＝ 9 ＋ 6 ＝ **15ns**

- **ライトスルー方式**：書込み時、キャッシュと主記憶の **両方に同時に** 書く。一貫性は保ちやすいが遅い。
- **ライトバック方式**：書込み時はキャッシュだけに書き、キャッシュから追い出すときに主記憶へ書き戻す。速いが制御が複雑。
- **マッピング方式**：主記憶のどのブロックをキャッシュのどこに置くか。**ダイレクトマッピング**（置き場所が1か所に決まる）、**フルアソシアティブ**（どこにでも置ける）、**セットアソシアティブ**（決まったグループ内ならどこでもよい）。
- **局所性**：プログラムは「最近使ったデータをまた使う（時間的局所性）」「近くのアドレスを続けて使う（空間的局所性）」傾向がある。これがキャッシュが効く理由。

> **用語メモ　メモリインタリーブ**
>
> 主記憶を複数のバンク（区画）に分け、連続したアドレスを別々のバンクに割り当てて、並行してアクセスすることで高速化する技術です。

### ■ ユニファイドメモリ

**ユニファイドメモリ方式** とは、CPUとGPU（画像処理装置）が **主記憶を共有する** 方式です。画面表示用の領域（フレームバッファ）も主記憶の一部を使います。専用のビデオメモリを持たないので安価・省電力ですが、主記憶の帯域を分け合います。

> **過去問ではこう出る　R7秋 午前問9**
> フレームバッファがユニファイドメモリ方式であるシステムの特徴を問う問題。「主記憶の一部を表示領域として使用する」が正解です。

## 5. 補助記憶装置

### ■ HDD（ハードディスク）

磁気ディスク（円盤）を回転させ、磁気ヘッドでデータを読み書きします。

- **アクセス時間 ＝ 平均シーク時間 ＋ 平均回転待ち時間 ＋ データ転送時間**
- **シーク時間**：ヘッドを目的のトラック（同心円）まで動かす時間。
- **平均回転待ち時間**：目的のデータがヘッドの下に来るまで待つ時間。平均は **1回転の半分**。

> **計算例　平均回転待ち時間**
> 6,000回転/分のHDDの平均回転待ち時間は？
> 1秒間に 6,000 ÷ 60 ＝ 100回転 → 1回転 ＝ 10ミリ秒 → 平均回転待ち時間 ＝ その半分 ＝ **5ミリ秒**

### ■ SSD

フラッシュメモリを使った補助記憶装置です。機械的な動作がないので高速・低消費電力・衝撃に強い反面、書換え回数に上限があります。同じ場所への書込みが集中しないように分散させる **ウェアレベリング** という技術を使います。

### ■ RAID

複数のディスクを組み合わせて、速度や信頼性を高める技術です。

| レベル | 方式 | 特徴 |
|---|---|---|
| RAID 0 | **ストライピング**：データを複数ディスクに分散して書く | 高速。冗長性なし（1台壊れると全データ消失） |
| RAID 1 | **ミラーリング**：同じデータを2台に書く | 信頼性が高い。容量効率50% |
| RAID 5 | ストライピング＋**パリティ**を分散して記録 | 1台が故障しても復元可能。3台以上必要 |
| RAID 6 | パリティを2種類記録 | 2台同時故障まで耐える |
| RAID 1+0 | ミラーリングしたものをストライピング | 高速かつ高信頼 |

# 6. 入出力インタフェースと入出力制御

| インタフェース | 特徴 |
|---|---|
| USB | シリアル。ホットプラグ（電源を入れたまま抜き差し）可能。バスパワーで給電 |
| HDMI | 映像と音声をデジタルで1本のケーブルで伝送 |
| Bluetooth | 近距離無線。キーボード、イヤホンなど |
| NFC | 10cm程度の超近距離無線。ICカード、スマホ決済 |
| $I^2C$ | 組込み機器で、センサーなど複数のデバイスを2本の信号線（バス）で接続する |
| PCI Express | 拡張カードやSSD（NVMe）を接続する高速なシリアルインタフェース |

- **DMA（直接メモリアクセス）**：CPUを介さずに、入出力装置と主記憶の間で直接データを転送する方式。CPUの負担が減る。
- **割込み**：実行中の処理を一時中断して、緊急の処理を優先させる仕組み。**外部割込み**（入出力完了、タイマ、電源異常など）と **内部割込み**（ゼロ除算、ページフォールト、ソフトウェア割込み＝スーパバイザコールなど）がある。

# 7. ハードウェアと組込みシステム

## ■ 電子部品と論理回路

- **FPGA**：製造後に利用者が回路の構成を **書き換えられる** 集積回路。試作や少量生産に向く。
- **ASIC**：特定の用途向けに設計・製造された専用の集積回路。大量生産で安価・高性能。
- **SoC（System on a Chip）**：CPU、メモリ、周辺回路など、システムに必要な機能を1つのチップにまとめたもの。
- **A/D変換**：アナログ信号をデジタル値に変換する。サンプリング（標本化）→ 量子化 → 符号化 の順。
- **PWM制御**：パルスのON時間の割合（**デューティ比**）を変えて、モーターの回転速度やLEDの明るさを制御する方式。

## ■ センサーとアクチュエータ

**センサー** は温度・光・加速度などの物理量を電気信号に変える部品、**アクチュエータ** は電気信号を動き（モーターの回転など）に変える部品です。組込みシステムは「センサーで測る → CPUで判断 → アクチュエータで動かす」という流れで動きます。

> **計算例　回転数をパルスから求める**
>
> 1MHz（1秒間に100万カウント）のタイマで、センサーのパルス1周期を測ったら 10,000 カウントだった。1回転で2パルス出るとき、回転数は毎秒何回転？
>
> 1パルスの周期 ＝ 10,000 ÷ 1,000,000 ＝ 0.01秒 → 1回転 ＝ 2パルス ＝ 0.02秒 → 1秒間に 1 ÷ 0.02 ＝ **50回転**
>
> （R7秋 午前問10 で問われた考え方です）

## ■ 電力と省エネ

- **DVFS**：処理の負荷に応じて、電圧とクロック周波数を動的に変えて消費電力を下げる技術。
- **パワーゲーティング**：使っていない回路への電源供給を止める。
- **クロックゲーティング**：使っていない回路へのクロック供給を止める。

### この章のまとめ

- PCは次の命令のアドレス。命令はフェッチ→デコード→オペランド読出し→実行→書込み。
- MIPS ＝ クロック周波数 ÷ CPI ÷ $10^6$。平均CPIは出現率で重み付け平均。
- パイプライン所要サイクル ＝ k ＋ (n−1)。VLIWは「コンパイル時にまとめる」。
- DRAMはリフレッシュ必要で主記憶、SRAMはフリップフロップでキャッシュ。
- 実効アクセス時間 ＝ キャッシュ×ヒット率 ＋ 主記憶×ミス率。
- HDDの平均回転待ちは半回転分。RAID5はパリティ分散で1台故障まで耐える。
- DMAはCPUを介さない転送。割込みは外部と内部に分かれる。

# 第4章 システム構成要素

第3章では「1台のコンピュータの中身」を学びました。この章では視点を広げて、**複数のコンピュータを組み合わせたシステム**をどう構成するか、そしてそのシステムが「どれくらい止まらないか（信頼性）」「どれくらい速いか（性能）」をどう評価するかを学びます。稼働率の計算は午前でほぼ毎回出題され、午後の問4（システムアーキテクチャ）でも中心テーマです。

### 第4章① システムを評価する3つの視点

**構成（どう組むか）**
集中／分散、クライアントサーバ、クラウド、仮想化、冗長化（デュアル・デュプレックス・クラスタ）

**信頼性（止まらないか）**
RASIS、MTBF・MTTR、稼働率、直列・並列の計算、フォールトトレランス

**性能（速いか）**
レスポンスタイム、スループット、ベンチマーク、キャパシティプランニング、スケールアップ／アウト

## 1. システムの処理形態と構成

### ■ 処理形態

- **集中処理**：1台の大型コンピュータ（ホスト）ですべてを処理する。管理しやすいが、ホストが止まると全体が止まる。
- **分散処理**：複数のコンピュータで処理を分担する。一部が止まっても全体は動き続けられるが、管理が複雑。
- **クライアントサーバシステム**：サービスを要求する側（クライアント）と提供する側（サーバ）に役割を分ける。
- **3層クライアントサーバ**：**プレゼンテーション層**（画面）、**ファンクション層**（業務処理）、**データ層**（データベース）の3つに分ける。各層を独立して変更しやすい。
- **ピアツーピア（P2P）**：対等なコンピュータどうしが直接やり取りする。

### ■ 冗長化構成（高信頼化）

| 構成 | しくみ |
|---|---|
| **デュアルシステム** | 2系統のシステムで **同じ処理を同時に** 行い、結果を照合する。片方が故障してもすぐ処理を続けられる |
| **デュプレックスシステム** | 主系（現用系）と従系（待機系）を用意し、主系が故障したら従系に切り替える |
| ホットスタンバイ | 待機系を起動した状態で待たせる。切替えが速い |
| コールドスタンバイ | 待機系を停止した状態で待たせる。切替えに時間がかかるが安価 |
| **クラスタ** | 複数のサーバを連携させて、1台のように見せる。負荷分散や高可用性を実現 |
| **ロードバランサ** | 複数のサーバに処理要求を振り分ける装置 |

## ■ クラウドコンピューティング

インターネット経由で、サーバやソフトウェアを「サービス」として利用する形態です。利用者は必要な分だけ借りて、使った分だけ支払います。

**第4章② クラウドのサービスモデル ― 利用者が管理する範囲**

| IaaS | PaaS | SaaS |
|---|---|---|
| Infrastructure as a Service<br>サーバ・ストレージ・ネットワークなどの**インフラ**を提供。OS以上は利用者が管理 | Platform as a Service<br>OS・ミドルウェア・実行環境（**プラットフォーム**）まで提供。利用者はアプリだけ作る | Software as a Service<br>**アプリケーション**まで提供。利用者はそのまま使う（Webメールなど） |

**覚え方**　IaaS → PaaS → SaaS の順に、事業者が面倒を見る範囲が広がり、利用者の自由度は下がります。

- **パブリッククラウド**：不特定多数で共有する。**プライベートクラウド**：自社専用。**ハイブリッドクラウド**：両者の組合せ。
- **オンプレミス**：自社内に機器を設置して運用する従来の形態。
- **エッジコンピューティング**：データの発生源（端末）の近くで処理し、通信の遅延や量を減らす。IoTで重要。

## ■ 仮想化

**仮想化** とは、1台の物理的なコンピュータの上で、複数の論理的な（仮想の）コンピュータを動かす技術です。

| 方式 | しくみ |
|---|---|
| **ホスト型** | ホストOSの上に仮想化ソフトを入れ、その上でゲストOSを動かす |
| **ハイパーバイザ型** | ハードウェアの上に直接ハイパーバイザ（仮想化ソフト）を置き、その上でゲストOSを動かす。オーバーヘッドが小さい |
| **コンテナ型** | 1つのOSの上で、アプリケーションの実行環境を隔離した「コンテナ」を複数動かす。ゲストOSが不要なので軽量・高速に起動する（Dockerなど） |

- **ライブマイグレーション**：仮想マシンを **停止させずに** 別の物理サーバへ移動させる技術。
- **VDI（デスクトップ仮想化）**：利用者のデスクトップ環境をサーバ上で動かし、端末には画面だけを転送する。端末にデータが残らないので情報漏えい対策になる。

## ■ IaC（Infrastructure as Code）

**IaC** とは、サーバやネットワークなどの **インフラの構成・設定をプログラム（コード）として記述し、専用のソフトウェアがその内容に従って自動的にシステムに適用する** 考え方です。手作業による設定ミスを防ぎ、同じ環境を何度でも再現できます。コードなのでバージョン管理やレビューも可能です。

**過去問ではこう出る　R7秋 午前問12**

IaC の説明を選ぶ問題。「システムの構成、設定などをプログラムとして記述し、専用のソフトウェアがその内容に従って自動的にシステムに適用する」が正解。「手動で」という言葉が入った選択肢は誤りです。

## 2. 信頼性 — RASIS と稼働率

### ■ RASIS（ラシス）

システムの信頼性を評価する5つの指標の頭文字です。

| 指標 | 意味 | 代表的な尺度 |
|---|---|---|
| **R**eliability（信頼性） | 故障しにくいこと | MTBF |
| **A**vailability（可用性） | 使いたいときに使えること | 稼働率 |
| **S**erviceability（保守性） | 故障してもすぐ直せること | MTTR |
| **I**ntegrity（完全性） | データが壊れたり矛盾したりしないこと | |
| **S**ecurity（安全性） | 不正アクセスから守られていること | |

### ■ MTBF・MTTR・稼働率

**第4章③ MTBF と MTTR のイメージ**

システムは「動く→故障→修理→動く…」を繰り返す




MTBF（平均故障間隔）＝ 稼働時間の平均　　MTTR（平均修理時間）＝ 修理時間の平均

**稼働率 ＝ MTBF ÷ (MTBF ＋ MTTR)**　（全体の時間のうち、動いている時間の割合）

- **MTBF（Mean Time Between Failures：平均故障間隔）**：故障が直ってから次に故障するまでの平均時間。**長いほど良い**（信頼性）。
- **MTTR（Mean Time To Repair：平均修理時間）**：故障してから直るまでの平均時間。**短いほど良い**（保守性）。
- **故障率** ＝ 1 ÷ MTBF（単位時間あたりに故障する回数）。

**計算例　稼働率の計算**

MTBF が 480時間、MTTR が 20時間のシステムの稼働率は？

稼働率 ＝ 480 ÷ (480 ＋ 20) ＝ 480 ÷ 500 ＝ **0.96**（96%）

**過去問ではこう出る　R7秋 午前問13**

信頼性に関する記述として適切なものを選ぶ問題。「MTBF/(MTBF＋MTTR) はシステムが稼働している時間の割合を表す」が正解です。MTBF は「運用開始から初めて故障するまで」ではなく「故障間隔の平均」、MTTR は「修理にかかる時間の平均」です。

## ■ 直列システムと並列システム

### 第4章④ 直列・並列システムの稼働率

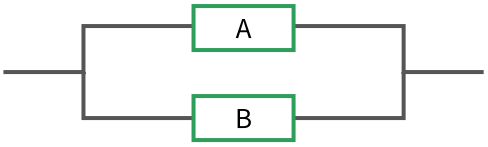
装置Aの稼働率を a、装置Bの稼働率を b とする

**直列システム（両方動かないとダメ）**




稼働率 ＝ **a × b**
例：0.9 × 0.9 ＝ 0.81（個々より下がる）

**並列システム（どちらか動けばOK）**




稼働率 ＝ **1 − (1−a)(1−b)**
例：1 − 0.1 × 0.1 ＝ 0.99（個々より上がる）

**考え方**　並列は「両方とも止まる確率」を1から引く。止まる確率 (1−a) を掛け合わせるのがコツです。

**計算例　直列と並列の組合せ**

稼働率 0.9 の装置2台を並列にしたものと、稼働率 0.95 の装置1台を直列につないだシステムの稼働率は？
**①並列部分**：1 − (1 − 0.9) × (1 − 0.9) ＝ 1 − 0.01 ＝ 0.99
**②直列**：0.99 × 0.95 ＝ **0.9405**
複雑な構成も「内側の並列を1つの装置とみなす → 直列で掛ける」と分解すれば解けます。

**用語メモ　n台中 k台以上が動けばよいシステム**

「3台中2台以上動けばよい」システムの稼働率は、「3台とも動く確率」＋「ちょうど2台動く確率」です。稼働率 a のとき $a^3 + 3a^2(1-a) = 3a^2 - 2a^3$。組合せ（第1章）の考え方を使います。

## ■ 信頼性を高める設計の考え方

| 用語 | 意味 |
|---|---|
| **フォールトトレランス** | 故障が起きても、冗長化などによってシステム全体としては機能を維持する |
| **フェールセーフ** | 故障時には **安全側** に動作させる（信号機の故障時は全部赤にする） |
| **フェールソフト** | 故障した部分を切り離して、機能を縮小してでも動き続ける（縮退運転） |
| **フォールトアボイダンス** | 高品質の部品を使うなどして、そもそも故障しないようにする |
| **フールプルーフ** | 人が誤った操作をしても、危険な状態にならないようにする（ドアが閉まらないと動かない電子レンジ） |

> **ポイント　フェールセーフとフェールソフトの区別**
> フェール「セーフ」は **安全** 優先（止めてもよい）、フェール「ソフト」は **継続** 優先（性能を落としてでも動かす）。選択肢で頻繁に入れ替えて出題されます。

### ■ バスタブ曲線

装置の故障率は時間とともに「**初期故障期**（製造不良などで故障率が高い）→ **偶発故障期**（低く安定）→ **摩耗故障期**（劣化で上昇）」と変化し、グラフがお風呂の形になるので **バスタブ曲線** と呼ばれます。

## 3. 性能評価

### ■ 性能の指標

- **レスポンスタイム（応答時間）**：要求を出してから、最初の応答が返ってくるまでの時間。
- **ターンアラウンドタイム**：ジョブを投入してから、すべての結果を受け取るまでの時間。
- **スループット**：単位時間あたりに処理できる仕事の量。

### ■ 性能評価の手法

| 手法 | 内容 |
|---|---|
| **ベンチマーク** | 標準的なプログラムを実際に実行して、性能を測定・比較する（SPECなど） |
| **モニタリング** | 稼働中のシステムの資源使用状況や応答時間を測定する |
| **シミュレーション** | モデルを作って性能を予測する |
| TPC | トランザクション処理（OLTP）性能のベンチマーク |

### ■ キャパシティプランニング

**キャパシティプランニング** とは、将来の利用者数やデータ量の増加を予測し、**将来にわたって必要なサービスレベル（応答時間など）を維持できるように**、システム資源の容量を計画することです。「現状を最大限引き出す」チューニングとは目的が違います。

> **過去問ではこう出る　R7秋 午前問14**
> キャパシティプランニングの目的に関する記述を選ぶ問題。「現在の応答時間を調査し、長期的に監視することで、**将来を含めて** 応答時間を維持する」が最も適切です。「現状の処理能力を最大限に引き出す」はチューニングの説明です。

### ■ スケールアップとスケールアウト

- **スケールアップ**：サーバ1台の性能を上げる（CPUやメモリの増強）。
- **スケールアウト**：サーバの **台数を増やして** 処理を分散する。Webサーバやクラウドで多用。
- **スケールダウン／スケールイン**：それぞれの逆（性能を下げる／台数を減らす）。

### この章のまとめ

- IaaS（インフラ）→ PaaS（実行環境）→ SaaS（アプリ）の順に事業者の管理範囲が広がる。
- コンテナはゲストOS不要で軽量。IaC は構成をコードで書き、自動で適用する。
- 稼働率 ＝ MTBF ÷ (MTBF＋MTTR)。直列は掛け算、並列は 1 − (1−a)(1−b)。
- フェールセーフ＝安全側、フェールソフト＝縮退して継続、フールプルーフ＝誤操作対策。
- キャパシティプランニングは「将来の需要」に対する容量計画。スケールアウトは台数増。

# 第5章 ソフトウェア（OS）

**OS（オペレーティングシステム）** は、ハードウェアとアプリケーションの間に立って、CPU・メモリ・ファイルなどの資源を管理する基本ソフトウェアです。複数のプログラムを同時に動かしているように見せたり、メモリが足りなくても大きなプログラムを動かしたりできるのは、OSの働きです。この章の内容は、午前に加えて午後の問7（組込みシステム開発）でも重要です。

### 第5章① ソフトウェアの階層とOSの主な機能

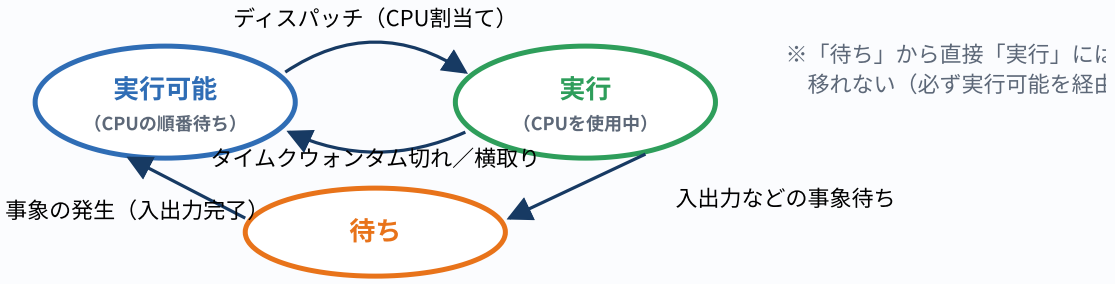
| | |
|---|---|
| **応用ソフトウェア** | ワープロ、表計算、業務システム、Webブラウザ など |
| **ミドルウェア** | DBMS（データベース管理システム）、Webサーバ、運用管理ツール など |
| **基本ソフトウェア（OS）** | タスク管理・記憶管理・ファイル管理・入出力管理・ユーザ管理 |
| **ハードウェア** | CPU・メモリ・ディスク・入出力装置 |

## 1. タスク（プロセス）管理

### ■ タスクとは

**タスク（プロセス）** とは、OSから見た「実行中のプログラム」の単位です。OSは複数のタスクに CPU を少しずつ順番に割り当てることで、同時に動いているように見せます（**マルチタスク**）。

### 第5章② タスクの3つの状態と遷移




### ■ スケジューリング方式

どのタスクに次に CPU を割り当てるかを決める方式です。

| 方式 | 内容 |
|---|---|
| **到着順（FCFS）** | 先に到着したタスクから順に実行する |
| **処理時間順（SPT）** | 処理時間が **短い** タスクから実行する。平均待ち時間が最小になる |
| **優先度順** | 優先度の高いタスクから実行する。低優先度のタスクがいつまでも実行されない「飢餓状態」に注意 |
| **ラウンドロビン** | 一定時間（**タイムクウォンタム**）ずつ順番に CPU を割り当て、時間切れになったら列の最後に戻す |
| **プリエンプティブ** | 実行中のタスクから強制的に CPU を奪える（横取りあり）方式 |

**計算例　処理時間順スケジューリングのターンアラウンドタイム**

| ジョブ | 到着時刻 | 処理時間 |
|---|---|---|
| A | 0 | 3 |
| B | 1 | 4 |
| C | 2 | 1 |
| 多重度1、横取りなしの処理時間順。時刻0ではAしかないのでAを実行（0〜3）。時刻3の時点でBとCが待っており、短いCを先に実行（3〜4）、次にB（4〜8）。 | | |
| Bのターンアラウンドタイム ＝ 終了時刻 8 − 到着時刻 1 ＝ **7** | | |
| **「その時点で到着済みのジョブの中から選ぶ」** のがポイントです。 | | |

**過去問ではこう出る　R7秋 午前問15**

5つのジョブを処理時間順で実行したときの、あるジョブのターンアラウンドタイムを求める問題。上の計算例と同じく、時刻を1つずつ進めながら「到着済みで最も短いジョブ」を選んでいけば解けます。

## ■ スレッドとリアルタイムOS

- **スレッド**：1つのタスクの中で、さらに並行して動く処理の単位。同じタスク内のスレッドはメモリを共有するので、切替えが軽い。
- **スレッドセーフ**：複数のスレッドから同時に（並列に）呼び出されても、問題が生じない（正しく動く）こと。
- **リアルタイムOS（RTOS）**：決められた時間内に必ず処理を終えること（**リアルタイム性**）を重視したOS。組込み機器で使われる。優先度に基づく **イベントドリブン型プリエンプティブ** スケジューリングを使うことが多い。

**過去問ではこう出る　R7秋 午前問17**

スレッドセーフの説明を選ぶ問題。「アプリケーションを複数のスレッドで並列に実行しても、問題が生じない」が正解です。

## 2. 排他制御・セマフォ・デッドロック

### ■ 排他制御とは

複数のタスクが同じ資源（データ、プリンタなど）を同時に書き換えると、データが壊れることがあります。これを防ぐため、**ある資源を1つのタスクが使っている間は、他のタスクに使わせない** ようにすることを **排他制御** といいます。

### ■ セマフォ

**セマフォ** は、排他制御や同期を実現する代表的な仕組みです。資源の空き数を表すカウンタを持ち、次の2つの操作で制御します。

- **P操作**：資源を使う前に行う。カウンタを1減らす。0なら（空きがなければ）待ち状態になる。
- **V操作**：資源を使い終わったら行う。カウンタを1増やし、待っているタスクがあれば起こす。
- カウンタの初期値を1にすると、同時に1つのタスクしか使えない（**バイナリセマフォ**・ミューテックス）。

> **過去問ではこう出る**　**R7秋 午前問16**
>
> 1つの $I^2C$ バスにつながった2つのセンサーの値を読む2つのタスクを、排他的に制御したい。リアルタイム OS のどの機能を使うか？ → **セマフォ**。「排他的に制御」という言葉が出たら、セマフォ（またはミューテックス）を思い出しましょう。

### ■ デッドロック

**デッドロック** とは、2つ以上のタスクが、互いに相手の持っている資源の解放を待ち合って、**どちらも永久に先に進めなくなる** 状態です。

**第5章③ デッドロックが起きる仕組みと防ぎ方**

**起きる例（獲得順序が逆）**
タスク1：資源X を確保 → 資源Y を要求
タスク2：資源Y を確保 → 資源X を要求
→ 互いに相手の解放を待ち続ける

**防ぐ方法**
すべてのタスクで **資源を獲得する順序を同じにする**
（例：必ず X → Y の順に確保）
→ 循環待ちが起きなくなる

> **過去問ではこう出る**　**R7秋 午前問18**
>
> 2つのタスクが2つの資源を排他的に使うとき、デッドロックを防ぐ方法を選ぶ問題。「資源獲得の順序を両方のタスクで同じにする」が正解。優先度を変えても防げません。

## 3. 記憶管理

### ■ 実記憶管理（主記憶の割当て）

主記憶の空き領域にプログラムを割り当てる方式として、次の3つがよく出ます。

| 方式 | 割当て方 | 特徴 |
|---|---|---|
| **ファーストフィット** | 先頭から探し、**最初に見つかった** 十分な大きさの空き領域に割り当てる | 探索が速い |
| **ベストフィット** | 要求された大きさを満たす **最小の** 空き領域に割り当てる | 大きな空き領域が残るが、**小さな空き領域が多数残る** 傾向 |
| **ワーストフィット** | **最大の** 空き領域に割り当てる | 残る空き領域が大きくなる |

- **フラグメンテーション（断片化）**：割当てと解放を繰り返すうちに、空き領域が細切れになり、合計は十分なのに大きなプログラムを置けなくなる現象。
- **コンパクション（メモリの再配置）**：使用中の領域を移動して、細切れの空き領域を1つにまとめること。
- **ガーベジコレクション**：プログラムが使い終わった（どこからも参照されなくなった）メモリ領域を自動的に回収すること。
- **メモリリーク**：確保したメモリを解放し忘れて、使える領域がだんだん減っていく不具合。

> **過去問ではこう出る　R7秋 午前問5**
>
> ベストフィット方式の特徴を選ぶ問題。「要求された大きさを満たす最小の空きブロックを割り当てるので、最終的には小さな空きブロックが多数残る傾向にある」が正解です。

## ■ 仮想記憶

**仮想記憶** とは、補助記憶（ディスク）の一部を使って、**実際の主記憶よりも大きな記憶空間があるように見せる** 仕組みです。プログラムは仮想的なアドレス（**仮想アドレス**）を使い、OSとハードウェア（MMU）が実際のアドレス（**実アドレス**）に変換します。

- **ページング方式**：記憶領域を固定長の「**ページ**」に分けて管理する。必要なページだけを主記憶に置く。
- **セグメント方式**：プログラムの論理的なまとまり（可変長のセグメント）単位で管理する。
- **ページフォールト**：必要なページが主記憶にないときに発生する割込み。OSがディスクからページを読み込む（**ページイン**）。主記憶に空きがなければ、どれかのページをディスクに追い出す（**ページアウト**）。
- **スラッシング**：主記憶が足りず、ページイン／ページアウトが頻発して、処理がほとんど進まなくなる状態。
- **TLB**：仮想アドレスから実アドレスへの変換結果を記憶しておく高速なバッファ。

## ■ ページ置換えアルゴリズム

主記憶がいっぱいのとき、どのページを追い出すかを決める方式です。

| 方式 | 追い出すページ |
|---|---|
| **FIFO** | **最も昔に** 主記憶に読み込まれたページ |
| **LRU**（Least Recently Used） | **最後に参照されてから最も時間が経っている**（最も長く使われていない）ページ |
| **LFU**（Least Frequently Used） | **参照された回数が最も少ない** ページ |

**計算例　LRU のページフォールト回数**

ページ枠3つ、ページ参照順「1, 2, 3, 1, 4, 2」、最初は空。

1:フォールト[1] → 2:フォールト[1,2] → 3:フォールト[1,2,3] → 1:ヒット（1が最近使われた）→ 4:フォールト。最も長く使われていないのは2なので2を追い出す[1,3,4] → 2:フォールト。最も長く使われていない3を追い出す[1,4,2]

ページフォールトは **5回**。

## 4. ファイル管理

- **ディレクトリ**：ファイルを分類して整理するための入れ物（フォルダ）。階層構造（木構造）になっている。
- **絶対パス**：最上位（ルート）からの道筋で表す（例：/home/user/a.txt）。
- **相対パス**：現在いるディレクトリ（カレントディレクトリ）からの道筋で表す。「..」は1つ上（親）のディレクトリ、「.」は現在のディレクトリ。

### ■ バックアップの方式

| 方式 | バックアップする範囲 | 復元に必要なもの |
|---|---|---|
| **フルバックアップ** | すべてのデータ | フルバックアップだけ |
| **差分バックアップ** | **前回のフルバックアップ以降** に変更されたデータ | フル ＋ 最新の差分 |
| **増分バックアップ** | **前回のバックアップ（種類を問わず）以降** に変更されたデータ | フル ＋ それ以降のすべての増分 |

## 5. 開発ツールと OSS

### ■ 言語処理の流れ

ソースプログラム →（**コンパイラ**）→ 目的プログラム →（**リンカ（連係編集）**）→ ロードモジュール →（**ローダ**）→ 主記憶に読み込んで実行。

- **静的リンク**：実行前にライブラリを組み込む。**動的リンク**：実行時に必要なライブラリを呼び出す（DLLなど）。
- **プリプロセッサ**：コンパイルの前に、マクロの展開などの前処理を行う。
- **最適化**：コンパイラが、実行速度やサイズが良くなるように目的プログラムを改善すること。

### ■ OSS（オープンソースソフトウェア）

ソースコードが公開され、誰でも利用・改変・再配布できるソフトウェアです。ライセンスの条件を守る必要があります。

- **GPL**：改変して配布する場合、そのソースコードも同じ GPL で公開しなければならない（**コピーレフト**）。
- **MIT ライセンス・BSD ライセンス**：著作権表示などを残せば、改変後のソースを公開しなくてよい。商用製品に組み込みやすい。
- 代表的な OSS：Linux（OS）、Apache・Nginx（Webサーバ）、MySQL・PostgreSQL（DBMS）、Git（バージョン管理）。

### この章のまとめ

- タスクは 実行可能・実行・待ち の3状態。待ちから直接実行には移らない。
- 処理時間順は「到着済みの中で最短」を選ぶ。ラウンドロビンはタイムクウォンタムで交代。
- 排他制御はセマフォ（P操作で確保、V操作で解放）。デッドロックは資源獲得順序をそろえて防ぐ。
- ベストフィットは最小の空きを使い、小さな空きが多数残る。
- 仮想記憶：ページフォールトでページイン。LRU は最も長く参照されていないページを追い出す。
- 差分は「前回フル以降」、増分は「前回バックアップ以降」。GPL はコピーレフト。

# 第6章 ヒューマンインタフェースとマルチメディア

どんなに高性能なシステムでも、使いにくければ価値が下がります。この章では、人とコンピュータの接点である **ヒューマンインタフェース** の設計の考え方と、画像・音声・動画といった **マルチメディア** をデジタルで扱う仕組みを学びます。出題数は1〜2問と少なめですが、データ量の計算は確実に得点したいところです。

### 第6章① この章の全体像

**ヒューマンインタフェース**
GUI部品の使い分け／ユーザビリティ評価／ユニバーサルデザイン・アクセシビリティ／UX

**マルチメディア**
画素と色深度／サンプリングと量子化／圧縮方式（可逆・非可逆）／VR・AR・MR

## 1. ヒューマンインタフェース

### ■ GUI 部品の使い分け

| 部品 | 用途 |
|---|---|
| ラジオボタン | 複数の選択肢から **1つだけ** 選ばせる |
| チェックボックス | 複数の選択肢から **いくつでも** 選ばせる（ON/OFF） |
| プルダウンメニュー（リストボックス） | 選択肢が多いときに、一覧から選ばせる。画面を節約できる |
| テキストボックス | 自由に文字を入力させる |
| スライダー | 連続した値（音量など）を直感的に指定させる |

### ■ 画面設計・帳票設計の原則

- 入力の順序は、元になる伝票や業務の流れに合わせる。
- 関連する項目はまとめて配置し、画面間で操作方法や配色を **統一** する。
- 入力ミスを防ぐため、コード入力には選択式やチェックディジットを使い、エラーメッセージは何をどう直せばよいかを具体的に示す。
- 頻繁に使う機能にはショートカットキーを用意し、熟練者の操作を効率化する。

### ■ ユーザビリティとアクセシビリティ

- **ユーザビリティ**：特定の利用者が、特定の目的を、有効に・効率よく・満足して達成できる度合い（使いやすさ）。
- **ヒューリスティック評価**：専門家が経験則（ガイドライン）に照らしてUIの問題点を洗い出す評価手法。
- **ユーザビリティテスト**：実際の利用者に操作してもらい、観察して問題点を見つける手法。**思考発話法**（考えていることを声に出しながら操作してもらう）がよく使われる。
- **アクセシビリティ**：高齢者や障害者を含め、**誰もが** 情報やサービスを利用できること。Webでは **JIS X 8341** や **WCAG** というガイドラインがある（画像に代替テキストを付ける、色だけで情報を伝えない、など）。

- **ユニバーサルデザイン**：年齢・能力・国籍などにかかわらず、**最初からすべての人が使いやすい** ように設計する考え方。
- **UX（ユーザエクスペリエンス）**：製品やサービスを使う前・中・後を通じて利用者が得る体験全体。
- **ペルソナ**：典型的な利用者像を、名前・年齢・行動などを具体的に設定して描いた架空の人物。設計の判断基準にする。

## 2. マルチメディア

### ■ 画像のデジタル表現

- **画素（ピクセル）**：画像を構成する小さな点。画素の数が多いほど細かい画像になる（**解像度**）。
- **色深度**：1画素の色を何ビットで表すか。24ビット（R・G・B 各8ビット）なら約1,677万色（**フルカラー**）。
- **光の三原色 RGB**：画面表示で使う。混ぜるほど明るくなり、全部混ぜると白（**加法混色**）。
- **色の三原色 CMY（＋K）**：印刷で使う。混ぜるほど暗くなる（**減法混色**）。K は黒。

**計算例　画像のデータ量**

横 1,920 × 縦 1,080 画素、24ビットフルカラーの画像1枚のデータ量は？

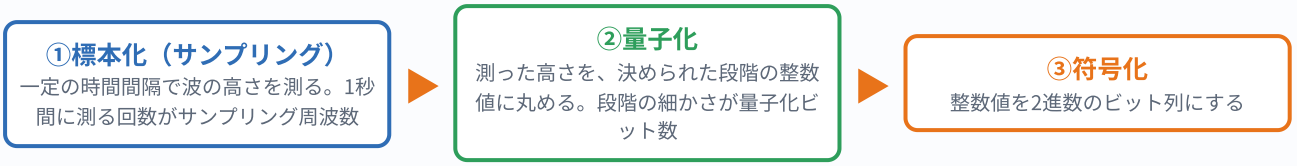
画素数 ＝ 1,920 × 1,080 ＝ 2,073,600 画素。1画素 ＝ 24ビット ＝ 3バイト。

データ量 ＝ 2,073,600 × 3 ＝ 6,220,800 バイト ≒ **約6.2Mバイト**

### ■ 音声のデジタル表現（PCM）

アナログの音声をデジタルにするには、次の3ステップを行います（**PCM：パルス符号変調**）。

**第6章② アナログ → デジタル変換の3ステップ**




**標本化定理**　元の信号に含まれる最高周波数の **2倍以上** の周波数でサンプリングすれば、元の信号を再現できます。人の耳に聞こえる約20kHzの2倍強で、音楽CDは44.1kHz。

**計算例　音声のデータ量**

サンプリング周波数 44.1kHz、量子化ビット数 16ビット、ステレオ（2チャネル）で1分間録音したデータ量は？

1秒間 ＝ 44,100 回 × 16ビット × 2チャネル ＝ 1,411,200 ビット ＝ 176,400 バイト

1分間 ＝ 176,400 × 60 ＝ 10,584,000 バイト ≒ **約10.6Mバイト**

## ■ 圧縮方式

| 分類 | 意味 | 例 |
|---|---|---|
| **可逆圧縮** | 圧縮したデータを **完全に元に戻せる** | ZIP、PNG、GIF、FLAC |
| **非可逆圧縮** | 人が気づきにくい情報を捨てて高い圧縮率を実現。完全には戻せない | JPEG（静止画）、MPEG（動画）、MP3・AAC（音声） |

- **MPEG** は、前後のフレーム（コマ）との **差分** だけを記録することで動画を大きく圧縮する。H.264、H.265などの規格がある。
- **ビットレート**：1秒あたりのデータ量（bps）。動画のデータ量 ＝ ビットレート × 時間 ÷ 8 でバイトに換算。

## ■ VR・AR・その他の技術

| 技術 | 内容 |
|---|---|
| **VR**（仮想現実） | コンピュータが作った仮想空間に入り込んだような体験をさせる |
| **AR**（拡張現実） | 現実の映像に、デジタル情報を **重ねて** 表示する |
| **MR**（複合現実） | 現実と仮想を融合し、仮想物体に触れるなど相互作用できる |
| **メタバース** | インターネット上の3次元仮想空間で、アバターを通じて交流・経済活動を行う |
| **CG のレンダリング** | 3次元のデータから画像を生成すること。**テクスチャマッピング**（表面に模様を貼る）、**レイトレーシング**（光線を追跡して反射・屈折を計算）など |
| **モーションキャプチャ** | 人の動きをセンサーで計測し、CGキャラクターに反映する |

### この章のまとめ

- ラジオボタンは1つだけ、チェックボックスは複数選択。アクセシビリティ＝誰もが利用可能。
- 画像のデータ量 ＝ 画素数 × 色深度。音声 ＝ サンプリング周波数 × 量子化ビット数 × チャネル数 × 秒。
- 標本化定理：最高周波数の2倍以上でサンプリング。
- 可逆（ZIP・PNG）と非可逆（JPEG・MPEG・MP3）。AR は現実に重ねる、VR は仮想空間に入る。

# 第7章 データベース

**データベース（DB）** とは、データを一元的に整理して保存し、複数の利用者やプログラムが共有できるようにしたものです。DBを管理するソフトウェアを **DBMS（データベース管理システム）** といいます。午前では毎回4〜5問、午後では問6として丸ごと1問出題されます。SQLと正規化を中心に、しっかり身につけましょう。

### 第7章① 関係データベースの基本用語

データを「表（テーブル）」の形で管理する

**社員表（関係・リレーション）**

| 社員番号 🔑 | 氏名 | 部署コード | 給与 |
|---|---|---|---|
| 1001 | 山田 | D01 | 300 |
| 1002 | 佐藤 | D02 | 280 |
| 1003 | 鈴木 | D01 | 320 |

**表（関係）**
テーブル全体

**行（タプル）**
1件分のデータ（オレンジの行）。レコードとも

**列（属性）**
項目。カラムとも

**主キー**
行を一意に特定する列（🔑）

## 1. データベースの基本

### ■ データモデル

- **関係データモデル（リレーショナルモデル）**：データを表で表し、表どうしを共通の値で関連付ける。現在の主流。
- **階層型モデル**：データを木構造（親子関係）で表す。
- **ネットワーク型モデル**：データを網目状につなぐ。

### ■ 3層スキーマ

DBの構造を3つの視点で定義し、利用者の見え方と物理的な格納方法を切り離す考え方です。

| スキーマ | 内容 |
|---|---|
| **外部スキーマ** | 利用者やプログラムから見たデータの形（**ビュー**） |
| **概念スキーマ** | DB全体の論理的な構造（表の定義） |
| **内部スキーマ** | ディスク上の物理的な格納方法 |

### ■ キーの種類

| キー | 意味 |
|---|---|
| **候補キー** | 行を一意に特定できる、必要最小限の列の組合せ。複数ある場合もある |
| **主キー** | 候補キーの中から1つ選んだもの。**重複不可・NULL不可**（一意性制約・非NULL制約） |
| **外部キー** | 他の表の主キーを参照する列。参照先に存在しない値は入れられない（**参照制約**） |
| **複合キー（連結キー）** | 複数の列を組み合わせたキー |

> **用語メモ　NULL（ナル）とは**
> 「値が存在しない・不明」を表す特別な状態です。0や空文字とは違います。SQLで NULL かどうかを調べるときは「= NULL」ではなく「**IS NULL**」と書きます。

## 2. E-R 図（概念データモデル）

**E-R図** は、データベースを設計する前に、管理対象（**エンティティ**：実体）と、それらの関係（**リレーションシップ**）を図で表したものです。

- **エンティティ**：管理したい「もの」や「こと」（社員、部署、受注など）。長方形で表す。
- **属性**：エンティティが持つ項目（社員番号、氏名など）。
- **カーディナリティ（多重度）**：関係の数の対応。**1対1、1対多、多対多**。

> **ポイント　多対多は連関エンティティで分解する**
> 「学生」と「科目」は多対多（1人の学生が多くの科目を履修し、1つの科目を多くの学生が履修）です。関係データベースでは多対多を直接表せないので、間に「**履修**」という表（連関エンティティ）を作り、「学生 1対多 履修 多対1 科目」に分解します。午後問6で頻出の考え方です。

## 3. 正規化

### ■ 正規化とは

**正規化** とは、表を適切に分割して、**データの重複をなくし、更新時の矛盾（更新時異状）を防ぐ** ことです。例えば、受注表の各行に顧客の住所を書いておくと、住所が変わったとき全行を直さなければならず、直し漏れで矛盾が起きます。

### ■ 関数従属

**関数従属** とは、「Aの値が決まれば、Bの値がただ1つに決まる」関係で、**A → B** と書きます。例：社員番号 → 氏名（社員番号が決まれば氏名が1つに決まる）。正規化はこの関数従属を手がかりに行います。

### 第7章② 正規化のステップ

| 段階 | 条件 | やること |
|---|---|---|
| **非正規形** | 1つのセルに複数の値（繰返し項目）がある | — |
| **第1正規形** | すべての項目が単一の値（**繰返し項目がない**） | 繰返し項目を別の行に分ける |
| **第2正規形** | 第1正規形 ＋ **部分関数従属がない** | 主キーの **一部** だけで決まる項目を別表に分ける |
| **第3正規形** | 第2正規形 ＋ **推移的関数従属がない** | 主キー以外の項目で決まる項目を別表に分ける |

**計算例　正規化の具体例**

受注表（**受注番号, 商品コード**, 受注日, 顧客コード, 顧客名, 商品名, 数量）　※太字が主キー

**①部分関数従属を見つける**：受注番号 → 受注日, 顧客コード, 顧客名 ／ 商品コード → 商品名。主キーの一部だけで決まるので、別表に分ける（第2正規形）。

　受注（**受注番号**, 受注日, 顧客コード, 顧客名）／商品（**商品コード**, 商品名）／受注明細（**受注番号, 商品コード**, 数量）

**②推移的関数従属を見つける**：受注番号 → 顧客コード → 顧客名。主キー以外の顧客コードで顧客名が決まるので、別表に分ける（第3正規形）。

　受注（**受注番号**, 受注日, 顧客コード）／顧客（**顧客コード**, 顧客名）

**用語メモ　ボイス・コッド正規形（BCNF）**

第3正規形よりさらに厳しく、「すべての関数従属 A → B で、A が候補キーを含む」状態です。応用情報ではまれに出題されます。

## 4. SQL

**SQL** は、関係データベースを操作するための言語です。

### ■ データ定義（DDL）とデータ操作（DML）

| 種類 | 主な命令 | 用途 |
|---|---|---|
| DDL（定義） | CREATE TABLE / CREATE VIEW / DROP / ALTER | 表やビューを作る・削除する・変更する |
| DML（操作） | SELECT / INSERT / UPDATE / DELETE | データを検索・追加・更新・削除する |
| DCL（制御） | GRANT / REVOKE / COMMIT / ROLLBACK | 権限の付与・取消し、トランザクションの確定・取消し |

## ■ SELECT 文の基本

```
SELECT 部署コード, AVG(給与) AS 平均給与      -- ④取り出す列
  FROM 社員                                   -- ①どの表から
 WHERE 給与 >= 200                            -- ②行を絞り込む
 GROUP BY 部署コード                           -- ③グループにまとめる
HAVING AVG(給与) > 300                        -- ③'グループを絞り込む
 ORDER BY 平均給与 DESC                       -- ⑤並べ替え（DESC:降順）
```

> **ポイント　WHERE と HAVING の違い**
>
> **WHERE** は「グループ化する前の **行** 」を絞り込み、**HAVING** は「GROUP BY でまとめた後の **グループ** 」を絞り込みます。集約関数（SUM, AVG, COUNT, MAX, MIN）を条件に使うときは HAVING です。

## ■ よく使う述語・関数

| 書き方 | 意味 |
|---|---|
| BETWEEN 100 AND 200 | 100以上200以下 |
| IN ('D01', 'D02') | いずれかに一致 |
| LIKE '山%' | 「山」で始まる（% は0文字以上の任意の文字列、_ は任意の1文字） |
| IS NULL | 値が NULL |
| COUNT(*) | 行数（NULL を含む）。COUNT(列) は NULL を数えない |
| DISTINCT | 重複を除く |

## ■ 表の結合

```
SELECT 社員.氏名, 部署.部署名
  FROM 社員 INNER JOIN 部署
    ON 社員.部署コード = 部署.部署コード
```

- **内部結合（INNER JOIN）**：両方の表で結合条件が一致する行だけを取り出す。
- **左外部結合（LEFT OUTER JOIN）**：左の表のすべての行を残し、右の表に一致する行がなければ NULL で埋める。「受注がない顧客も一覧に出したい」ときに使う。

## ■ 副問合せ（サブクエリ）

SELECT 文の中に、さらに SELECT 文を入れる書き方です。

```
-- 平均より給与が高い社員
SELECT 氏名 FROM 社員
 WHERE 給与 > (SELECT AVG(給与) FROM 社員)

-- 受注が1件もない顧客（相関副問合せ）
SELECT 顧客名 FROM 顧客 C
 WHERE NOT EXISTS (SELECT * FROM 受注 J WHERE J.顧客コード = C.顧客コード)
```

- **EXISTS**：副問合せの結果が1行以上あれば真。**相関副問合せ** は外側の行ごとに内側を実行する。

### ■ 関係代数（集合演算）

| 演算 | 内容 |
|---|---|
| **選択** | 条件に合う **行** を取り出す（WHERE） |
| **射影** | 指定した **列** を取り出す（SELECT 列名） |
| **結合** | 共通の列で2つの表をつなぐ（JOIN） |
| 和・差・積 | 2つの表の行の和集合（UNION）・差集合（EXCEPT）・積集合（INTERSECT） |
| 直積 | 2つの表のすべての行の組合せ |

### ■ ビューとインデックス

- **ビュー**：SELECT 文の結果を、仮想的な表として定義したもの。利用者に必要な列だけ見せる（セキュリティ）、複雑な問合せを簡単にするなどに使う。
- **インデックス（索引）**：検索を速くするための仕組み。$B^+$木がよく使われる。検索は速くなるが、更新時にインデックスも更新するので **更新は遅くなる**。

## 5. トランザクション管理

### ■ トランザクションと ACID 特性

**トランザクション** とは、「これ以上分けられない一連の処理のまとまり」です。例えば銀行振込の「A口座から引く」と「B口座に足す」は、両方成功するか両方取り消すかのどちらかでなければなりません。トランザクションは次の **ACID特性** を満たす必要があります。

**第7章③ ACID 特性**

**Atomicity 原子性**
全部成功（コミット）か、全部取消し（ロールバック）のどちらか

**Consistency 一貫性**
処理の前後でデータの整合性が保たれる

**Isolation 独立性**
同時に実行しても、互いに影響しない（順に実行したのと同じ結果）

**Durability 永続性**
コミットした結果は、障害が起きても失われない

### ■ 排他制御（ロック）

複数のトランザクションが同じデータを同時に更新すると、片方の更新が消える（**ロストアップデート**）などの問題が起きます。これを防ぐのが **ロック** です。

| ロック | 意味 | 他のトランザクションは |
|---|---|---|
| **共有ロック**（読取り用） | データを読む間かける | 読取りは可、更新は不可 |
| **専有ロック**（排他ロック・更新用） | データを更新する間かける | 読取りも更新も不可 |

- **2相ロック方式**：必要なロックをすべてかけ終わるまでロックを解除しない（ロックをかける相 → 解除する相）。これで直列化可能性が保証される。

- **デッドロック**：トランザクションどうしが互いのロック解除を待ち合う状態（第5章と同じ）。DBMS は検出して、一方をロールバックする。
- **ロックの粒度**：ロックをかける単位（表・ページ・行）。粒度が **細かい** ほど同時実行性は高いが、ロック管理の負荷が大きい。
- **楽観的ロック**：ロックをかけずに処理し、更新時に他者が変更していないかを確認する方式。

### ■ 障害回復

DBMS は、更新の前後のデータを **ログ（ジャーナル）** に記録しておき、障害時に使います。

| 回復処理 | 使うもの | 場面 |
|---|---|---|
| **ロールバック（後退復帰）** | **更新前** ログ | トランザクションの途中で障害が起きたとき、処理を **取り消して** 開始前の状態に戻す |
| **ロールフォワード（前進復帰）** | バックアップ ＋ **更新後** ログ | ディスク障害のとき、バックアップを復元してから更新後ログで **やり直して** 障害直前の状態にする |

- **チェックポイント**：一定間隔で、メモリ上の更新内容をディスクに書き出す時点。システム障害時は、チェックポイント以降のログだけ処理すればよいので回復が速い。
- **2相コミット**：分散データベースで、複数のサイトの更新を整合させる方式。第1相で全サイトに「コミットできるか」を確認し、全員OKなら第2相でコミットを指示する。

## 6. データベースの応用

- **データウェアハウス**：業務データを時系列に蓄積し、意思決定の分析に使う大規模DB。
- **データマイニング**：大量のデータから、統計やAIで有用なパターンや規則性を見つけ出す。
- **OLAP**：データを多次元的に集計・分析する（ドリルダウン、スライスなど）。
- **NoSQL**：関係モデル以外のDB。**キーバリュー型**、**ドキュメント型**（JSONなど）、**列指向型**、**グラフ型**。大量データの高速処理や柔軟な構造に強い。
- **BASE特性**：NoSQL が重視する特性。ACID のような厳密な一貫性より、可用性を優先し「最終的に一貫すればよい（結果整合性）」と考える。
- **ビッグデータ**：量（Volume）・速度（Velocity）・多様性（Variety）が大きいデータ。
- **データレイク**：構造化・非構造化を問わず、生データをそのまま大量に蓄積する保存場所。
- **ブロックチェーン**：取引記録をハッシュ値でつないだブロックの鎖として、多数の参加者で分散管理する技術。改ざんが極めて困難。

> **この章のまとめ**
>
> - 主キーは重複不可・NULL不可。外部キーは参照制約。NULL の判定は IS NULL。
> - 第1正規形＝繰返しなし、第2＝部分関数従属なし、第3＝推移的関数従属なし。
> - WHERE は行、HAVING はグループを絞る。LEFT OUTER JOIN は左表を全件残す。
> - ACID：原子性・一貫性・独立性・永続性。共有ロックは読取り可、専有ロックは一切不可。
> - ロールバックは更新前ログで取消し、ロールフォワードはバックアップ＋更新後ログでやり直し。
> - NoSQL は BASE（結果整合性）。インデックスは検索が速く更新が遅くなる。

# 第8章 ネットワーク

**ネットワーク** とは、コンピュータどうしを通信回線でつないで、データをやり取りできるようにしたものです。インターネットも、世界中のネットワークをつないだ巨大なネットワークです。この章では、データが相手に届くまでの仕組みを「階層（レイヤ）」に分けて理解し、IPアドレスの計算や代表的なプロトコルを学びます。午後の問5（ネットワーク）と問1（セキュリティ）の土台です。

**第8章① OSI参照モデルと TCP/IP の対応**

通信の機能を7つの層に分け、各層が決められた役割だけを担当する

| | |
|---|---|
| **第7層 アプリケーション層** | アプリごとの通信ルール ― HTTP, SMTP, POP3, IMAP, DNS, FTP, SSH |
| **第6層 プレゼンテーション層** | データの表現形式（文字コード・圧縮・暗号化） ※TCP/IPではアプリケーション層にまとめる |
| **第5層 セション層** | 通信の開始・終了の管理 |
| **第4層 トランスポート層** | 端末間の信頼性（再送・順序制御）― TCP, UDP ／ ポート番号 |
| **第3層 ネットワーク層** | 異なるネットワーク間の中継・経路選択 ― IP, ICMP ／ IPアドレス ／ ルータ |
| **第2層 データリンク層** | 同じネットワーク内の隣接機器間の転送 ― イーサネット ／ MACアドレス ／ スイッチ |
| **第1層 物理層** | 電気信号・光・電波への変換、ケーブル・コネクタ ／ リピータ・ハブ |

## 1. 階層モデルとプロトコル

### ■ プロトコルとは

**プロトコル** とは、通信するための約束事（ルール）です。「どんな形式でデータを送るか」「届いたら返事をするか」などを決めておかないと、機器どうしが通信できません。

### ■ カプセル化

送信側では、上の層から下の層へデータを渡すたびに、各層が自分の制御情報（**ヘッダ**）を付け加えます。これを **カプセル化** といいます。受信側では逆に、下の層から順にヘッダを外していきます。

- データリンク層の単位：**フレーム**、ネットワーク層：**パケット**、トランスポート層（TCP）：**セグメント**。

# 2. LAN と接続機器

## ■ 層ごとの接続機器

| 機器 | 層 | 働き |
|---|---|---|
| **リピータ（リピータハブ）** | 物理層 | 電気信号を増幅して中継する。宛先は見ない |
| **ブリッジ・L2スイッチ（スイッチングハブ）** | データリンク層 | **MACアドレス** を見て、宛先の機器がつながるポートにだけ転送する |
| **ルータ・L3スイッチ** | ネットワーク層 | **IPアドレス** を見て、異なるネットワークへ最適な経路で中継する |
| **ゲートウェイ** | トランスポート層以上 | プロトコルが異なるネットワークどうしを接続・変換する |

## ■ MAC アドレスと ARP

- **MACアドレス**：ネットワーク機器（NIC）に製造時に割り当てられる、48ビットの固有の番号。
- **ARP**：IPアドレスから、対応するMACアドレスを調べるプロトコル。同じネットワーク内の全員に「このIPアドレスの人は誰？」と問い合わせる（ブロードキャスト）。

## ■ VLAN

**VLAN** とは、物理的な配線とは関係なく、スイッチの設定によって論理的にネットワークをグループ分けする技術です。部署ごとに分けるなど、レイアウト変更に柔軟に対応できます。

# 3. TCP/IP の主要プロトコル

## ■ TCP と UDP

| | TCP | UDP |
|---|---|---|
| 接続 | **コネクション型**（通信前に接続を確立：3ウェイハンドシェイク） | コネクションレス型（いきなり送る） |
| 信頼性 | 高い（確認応答・再送・順序制御） | 低い（届いたかを確認しない） |
| 速度 | 遅め | 速い・オーバーヘッド小 |
| 用途 | Web、メール、ファイル転送 | 音声通話、動画配信、DNS の問合せ、DHCP |

## ■ ポート番号

**ポート番号** は、1台のコンピュータの中で、どのアプリケーション（サービス）宛ての通信かを区別する番号です。IPアドレスが「建物の住所」なら、ポート番号は「部屋番号」です。

| ポート | プロトコル | 用途 |
|---|---|---|
| 20/21 | FTP | ファイル転送 |
| 22 | SSH | 暗号化された遠隔ログイン |
| 25 | SMTP | メールの送信・転送 |
| 53 | DNS | ドメイン名の名前解決 |
| 80 | HTTP | Web |
| 110 | POP3 | メールの受信（サーバからダウンロード） |
| 143 | IMAP | メールの受信（サーバ上で管理） |
| 443 | HTTPS | 暗号化されたWeb |
| 587 | サブミッションポート | メールの送信（認証つき） |

### ■ アプリケーション層の主要プロトコル

- **HTTP**：WebブラウザとWebサーバの間の通信。**HTTPS** は TLS で暗号化したもの。
- **SMTP**：メールの送信・サーバ間の転送。**POP3**：メールを端末にダウンロードして読む。**IMAP**：メールをサーバ上に置いたまま管理する（複数端末で同じメールを見られる）。
- **DNS**：「www.example.jp」のような **ドメイン名** と IPアドレスを対応付ける（**名前解決**）。
- **DHCP**：ネットワークに接続した端末に、IPアドレスなどの設定を **自動的に割り当てる**。
- **NTP**：機器の時刻を合わせる。
- **SNMP**：ネットワーク機器を監視・管理する。
- **MIME**：メールで、テキスト以外（画像・添付ファイル）や日本語を扱えるようにする規格。暗号化・署名を加えたものが **S/MIME**。

## 4. IP アドレスとサブネット

### ■ IPv4 アドレスの構造

**IPアドレス** は、ネットワーク上の機器を識別する番号で、IPv4 では **32ビット** です。8ビットずつ区切って10進数で「192.168.1.10」のように書きます。IPアドレスは、どのネットワークかを表す **ネットワーク部** と、そのネットワーク内のどの機器かを表す **ホスト部** に分かれます。

### ■ サブネットマスクと CIDR

**サブネットマスク** は、IPアドレスのどこまでがネットワーク部かを示す32ビットの値で、ネットワーク部を1、ホスト部を0にします。「/24」のように **ネットワーク部のビット数** で書く方法を **CIDR表記（プレフィックス長）** といいます。

## 第8章② サブネットの計算（192.168.10.77/26 の例）

| | 第1オクテット | 第2 | 第3 | 第4オクテット（2進数） |
|---|---|---|---|---|
| IPアドレス | 192 | 168 | 10 | **01**001101（＝77） |
| サブネットマスク /26 | 255 | 255 | 255 | **11**000000（＝192） |
| ネットワークアドレス | 192 | 168 | 10 | **01**000000（＝**64**）← ホスト部をすべて0 |
| ブロードキャストアドレス | 192 | 168 | 10 | **01**111111（＝**127**）← ホスト部をすべて1 |

**結果**　ネットワーク部 26ビット、ホスト部 6ビット。割当て可能なホスト数 ＝ $2^6 - 2$ ＝ **62台**（192.168.10.65 ～ 192.168.10.126）。「−2」はネットワークアドレスとブロードキャストアドレスの分です。

**計算例**　**サブネットマスクとホスト数の早見表**

| プレフィックス | サブネットマスク | ホスト部 | 割当て可能なホスト数 |
|---|---|---|---|
| /24 | 255.255.255.0 | 8ビット | 254 |
| /25 | 255.255.255.128 | 7ビット | 126 |
| /26 | 255.255.255.192 | 6ビット | 62 |
| /27 | 255.255.255.224 | 5ビット | 30 |
| /28 | 255.255.255.240 | 4ビット | 14 |
| /30 | 255.255.255.252 | 2ビット | 2 |

### ■ プライベートアドレスと NAT

- **グローバルアドレス**：インターネット上で一意の IPアドレス。
- **プライベートアドレス**：社内・家庭内などで自由に使えるアドレス。インターネットには直接出られない。範囲は **10.0.0.0/8、172.16.0.0/12、192.168.0.0/16**。
- **NAT**：プライベートアドレスとグローバルアドレスを変換する。
- **NAPT（IPマスカレード）**：IPアドレスに加えて **ポート番号も変換** し、1つのグローバルアドレスを複数の端末で共有する。
- **ループバックアドレス**：127.0.0.1。自分自身を表す。

### ■ IPv6

IPv4 のアドレス不足を解消するため、アドレスを **128ビット** に拡張したもの。16ビットずつ16進数で区切り、「2001:db8::1」のように書きます（連続する0は「::」で1回だけ省略可）。

### ■ ルーティング

**ルーティング** とは、ルータがパケットを宛先に届けるための最適な経路を選ぶことです。ルータは **ルーティングテーブル**（宛先ネットワークと次の転送先の対応表）に従って転送します。

- **静的ルーティング**：管理者が手作業で経路を設定する。
- **動的ルーティング**：ルータどうしが経路情報を交換して自動で更新する。**RIP**（ホップ数＝経由するルータ数で判断）、**OSPF**（回線速度などのコストで判断）、**BGP**（組織（AS）間の経路制御）。
- **デフォルトゲートウェイ**：宛先がテーブルにない場合に送る先のルータ。
- **ICMP**：通信の異常を通知したり、到達確認（ping）を行うためのプロトコル。

## 5. 伝送時間の計算

**計算例　伝送時間を求める**

1Mバイトのファイルを、100Mビット/秒の回線で送る。回線の伝送効率は80%。何秒かかる？

**①単位をビットにそろえる**：1Mバイト ＝ 8Mビット

**②実効速度**：100M × 0.8 ＝ 80Mビット/秒

**③伝送時間**：8 ÷ 80 ＝ **0.1秒**

「バイトとビットの変換（×8）」と「伝送効率を掛ける」を忘れないことがポイントです。

- **回線利用率** ＝ 実際に流れているデータ量 ÷ 回線速度。
- **bps**：bits per second（1秒間に送れるビット数）。

## 6. 無線 LAN・移動体通信・新しいネットワーク技術

### ■ 無線 LAN

| 項目 | 内容 |
|---|---|
| 規格 | IEEE 802.11 シリーズ。Wi-Fi 6（802.11ax）、Wi-Fi 7（802.11be）など |
| **SSID（ESSID）** | 無線LANのネットワークを識別する名前 |
| **アクセス制御方式** | **CSMA/CA**：送信前に電波が空いているか確認し、さらにランダムな時間待ってから送る（衝突を **回避**）。有線のCSMA/CDは衝突を **検出** |
| **暗号化** | **WPA3**（最新）、WPA2。WEP は脆弱なので使ってはいけない |
| メッシュWi-Fi | 複数のアクセスポイントが網目状に連携し、広い範囲をカバーする |

### ■ 移動体通信・IoT 向けネットワーク

- **5G**：第5世代移動通信システム。**超高速・大容量**、**超低遅延**、**多数同時接続** が特徴。
- **LPWA**：Low Power Wide Area。**低消費電力で広い範囲（数km〜）** をカバーする、IoT向けの無線通信。通信速度は低い（LoRaWAN、Sigfox、NB-IoT）。
- **BLE（Bluetooth Low Energy）**：省電力の近距離無線。
- **テザリング**：スマートフォンを中継器にして、PCなどをインターネットに接続する。
- **MVNO**：自前の回線設備を持たず、他社から借りて通信サービスを提供する事業者（格安SIM）。

## ■ ネットワークの仮想化

- **SDN**（Software-Defined Networking）：ネットワークの制御機能（経路の判断）と転送機能を分離し、ソフトウェアで集中的に制御する。**OpenFlow** が代表的な規格。
- **CDN**：コンテンツを世界中のキャッシュサーバに分散配置し、利用者に近いサーバから配信する。
- **PoE**：LANケーブルで電力も供給する（監視カメラ、無線LANアクセスポイント）。

### この章のまとめ

- OSI 7層：物理・データリンク（MAC・スイッチ）・ネットワーク（IP・ルータ）・トランスポート（TCP/UDP）…。
- TCP はコネクション型で高信頼、UDP は高速・軽量。代表的なポート番号は暗記する。
- サブネット：ホスト部全0＝ネットワーク、全1＝ブロードキャスト。ホスト数 ＝ $2^{\text{ホスト部ビット数}} - 2$。
- プライベートアドレス3範囲、NAPT はポート番号も変換。DHCP は自動割当て、DNS は名前解決。
- 伝送時間 ＝ データ量（ビット）÷（回線速度 × 伝送効率）。
- 無線LANは CSMA/CA、暗号化は WPA3。LPWA は低消費電力・広域、5G は超高速・低遅延・多数接続。

# 第9章 情報セキュリティ

**情報セキュリティ** は、午前で毎回10問前後出題され、午後では問1として **全員必須** の最重要分野です。「どんな攻撃があり、それぞれ何で防ぐのか」を、攻撃と対策のセットで理解することが合格への近道です。暗号やデジタル署名は最初は難しく感じますが、「どの鍵で暗号化し、どの鍵で復号するか」さえ押さえれば一気にわかるようになります。

### 第9章① 情報セキュリティの3要素（CIA）

この3つを維持することが情報セキュリティの目的

**機密性（Confidentiality）**
許可された人だけがアクセスできる
対策例：アクセス制御、暗号化

**完全性（Integrity）**
情報が正確で、改ざんされていない
対策例：デジタル署名、ハッシュ値

**可用性（Availability）**
必要なときに使える
対策例：冗長化、バックアップ、DDoS対策

**追加の4要素**　真正性（本物であること）・責任追跡性（誰がやったか追える）・否認防止（後から「やっていない」と言わせない）・信頼性（意図どおりに動く）。JIS Q 27000 で定義されています。

## 1. リスクと脅威

### ■ 情報セキュリティリスクの考え方

- **情報資産**：守るべき価値のある情報やシステム（顧客データ、サーバなど）。
- **脅威**：情報資産に損害を与える可能性のある原因（攻撃者、ウイルス、災害、誤操作）。
- **脆弱性**：脅威につけ込まれる弱点（ソフトウェアのバグ、設定ミス、パスワードの使い回し）。
- **リスク**：脅威が脆弱性を突いて、損害が発生する可能性。**リスク ＝ 資産価値 × 脅威 × 脆弱性** のように評価する。

### ■ リスクマネジメント

**リスクアセスメント** ＝ リスク特定 → リスク分析 → リスク評価。その後、評価結果に応じて **リスク対応** を選びます。

| リスク対応 | 内容 | 例 |
|---|---|---|
| **リスク回避** | リスクの原因そのものをなくす | 危険なサービスを廃止する |
| **リスク低減** | 対策を講じて発生確率や影響を下げる | ウイルス対策ソフトの導入 |
| **リスク移転（共有）** | 他者に移す | 保険に入る、外部委託する |
| **リスク保有（受容）** | 影響が小さいので、そのまま受け入れる | 対策費用の方が高い場合 |

# 2. 攻撃手法

## ■ マルウェア

**マルウェア** とは、悪意のあるソフトウェアの総称です。

| 種類 | 特徴 |
|---|---|
| **ウイルス** | 他のプログラムに寄生して感染を広げる |
| **ワーム** | 単独で動作し、ネットワーク経由で自己増殖する |
| **トロイの木馬** | 有用なソフトを装って侵入し、裏で不正な動作をする |
| **ランサムウェア** | ファイルを暗号化して使えなくし、元に戻すための **身代金** を要求する |
| **ボット** | 感染した端末を、外部の指令サーバ（**C&Cサーバ**）から遠隔操作できるようにする |
| **スパイウェア／キーロガー** | 利用者の情報やキー入力を盗み、外部に送る |
| **ルートキット** | 侵入の痕跡を隠すためのツール群 |
| **ファイルレスマルウェア** | ファイルをディスクに保存せず、メモリ上やOS標準のツールで動作するため検知しにくい |

## ■ パスワードに対する攻撃

| 攻撃 | 内容 | 対策 |
|---|---|---|
| **ブルートフォース攻撃（総当たり）** | 考えられるすべての文字の組合せを試す | 試行回数制限（アカウントロック） |
| **辞書攻撃** | よく使われる単語のリストで試す | 推測されにくいパスワード |
| **リバースブルートフォース攻撃** | パスワードを1つ固定し、利用者IDを次々に変えて試す（ロックされにくい） | 多要素認証 |
| **パスワードリスト攻撃** | 他のサイトから漏えいしたIDとパスワードの組を使って試す | **パスワードの使い回しをしない** |
| **レインボーテーブル攻撃** | ハッシュ値から元のパスワードを逆引きする表を使う | **ソルト**（ランダムな値）を付けてからハッシュ化 |

## ■ Web アプリケーションへの攻撃

### 第9章② 代表的な Web 攻撃と対策（午後問1で超頻出）

| 攻撃 | 何をされるか | 主な対策 |
|---|---|---|
| **SQLインジェクション** | 入力欄に SQL の断片を入れ、DBを不正に操作（情報漏えい・改ざん）する | **プレースホルダ**（バインド機構）を使う、入力値のエスケープ |
| **クロスサイトスクリプティング（XSS）** | 悪意のあるスクリプトを Web ページに埋め込み、閲覧者のブラウザで実行させる（Cookie の盗取など） | 出力時の **エスケープ（サニタイジング）**、Cookie に HttpOnly 属性 |
| **クロスサイトリクエストフォージェリ（CSRF）** | ログイン中の利用者に、罠のページ経由で意図しない操作（送金・書込み）をさせる | **トークン**（推測困難な値）をフォームに埋め込んで確認、再認証 |
| **ディレクトリトラバーサル** | 「../」を使ってファイル名を指定し、公開していないファイルにアクセスする | ファイル名の直接指定をさせない、パスの正規化 |
| **OSコマンドインジェクション** | 入力値に OS のコマンドを混ぜ、サーバ上で実行させる | シェルを起動する関数を使わない |
| **セッションハイジャック** | セッションIDを盗んで、正規の利用者になりすます | セッションIDを推測困難にし、ログイン後に再発行、HTTPS |
| **クリックジャッキング** | 透明なページを重ねて、意図しないボタンを押させる | X-Frame-Options ヘッダ |

## ■ ネットワークへの攻撃

| 攻撃 | 内容 |
|---|---|
| **DoS / DDoS 攻撃** | 大量の通信を送りつけて、サービスを停止させる。多数の端末（ボット）から行うのが DDoS |
| **DNS キャッシュポイズニング** | DNS サーバに偽の情報を覚えさせ、利用者を偽サイトに誘導する |
| **中間者攻撃（MITM）** | 通信の間に割り込み、盗聴や改ざんを行う |
| **ポートスキャン** | どのポートが開いているかを調べ、攻撃の糸口を探す（攻撃の下調べ） |
| **IP スプーフィング** | 送信元 IP アドレスを偽装する |
| **リフレクション攻撃（反射型 DDoS）** | 送信元を攻撃対象に偽装して問合せを送り、大きな応答を攻撃対象に送りつけさせる（DNSアンプ攻撃など） |

## ■ 人や組織を狙う攻撃

- **ソーシャルエンジニアリング**：技術ではなく、人の心理的な隙を突いて情報を盗む（電話で上司になりすましてパスワードを聞き出す、肩越しにのぞく **ショルダーハッキング**、ゴミ箱をあさる **トラッシング**）。
- **フィッシング**：実在の組織を装ったメールで偽サイトに誘導し、IDやパスワードを入力させる。
- **標的型攻撃**：特定の組織を狙い、業務に関係ありそうなメールでマルウェアを送り込む。長期間潜伏するものは **APT（持続的標的型攻撃）**。
- **水飲み場型攻撃**：標的がよく見るWebサイトを改ざんし、閲覧しただけで感染させる。

- **ビジネスメール詐欺（BEC）**：取引先や経営者になりすましたメールで、偽の口座に送金させる。
- **ゼロデイ攻撃**：脆弱性が発見されてから、修正プログラムが提供される **前に** 行われる攻撃。
- **サプライチェーン攻撃**：セキュリティの弱い取引先や、ソフトウェアの更新の仕組みを経由して標的に侵入する。

## 3. 暗号技術

### ■ 暗号の基本用語

- **平文**：暗号化する前のデータ。**暗号文**：暗号化した後のデータ。
- **暗号化**：平文を、鍵を使って第三者に読めない暗号文に変換すること。**復号**：暗号文を元の平文に戻すこと。
- **鍵**：暗号化・復号に使う秘密の値。

### ■ 共通鍵暗号方式と公開鍵暗号方式

**第9章③ 2つの暗号方式の比較**

| | 共通鍵暗号方式 | 公開鍵暗号方式 |
|---|---|---|
| 鍵 | 暗号化と復号で **同じ鍵** を使う | **公開鍵** と **秘密鍵** のペアを使う |
| 暗号化 → 復号 | 共通鍵 → 共通鍵 | **受信者の公開鍵** で暗号化 → **受信者の秘密鍵** で復号 |
| 処理速度 | **速い** | 遅い |
| 鍵の数（n人で通信） | n(n−1)/2 個（多い） | 2n 個（1人1ペア） |
| 課題 | 鍵を安全に相手に渡す方法（鍵配送問題） | 公開鍵が本人のものかの証明（→ PKI） |
| 代表例 | **AES**（DESの後継） | **RSA**（素因数分解の難しさを利用）、**楕円曲線暗号** |

**計算例　必要な鍵の数**

10人が互いに暗号通信する。共通鍵暗号方式では、ペアごとに別の鍵が必要 → 10 × 9 ÷ 2 ＝ **45個**。公開鍵暗号方式では1人1ペア → 10 × 2 ＝ **20個**。

**用語メモ　ハイブリッド暗号方式**

実際の通信（TLS など）では、両方のいいとこ取りをします。データ本体は **速い共通鍵** で暗号化し、その共通鍵を **公開鍵暗号** で暗号化して相手に送ります。これで「速さ」と「鍵配送問題の解決」を両立できます。

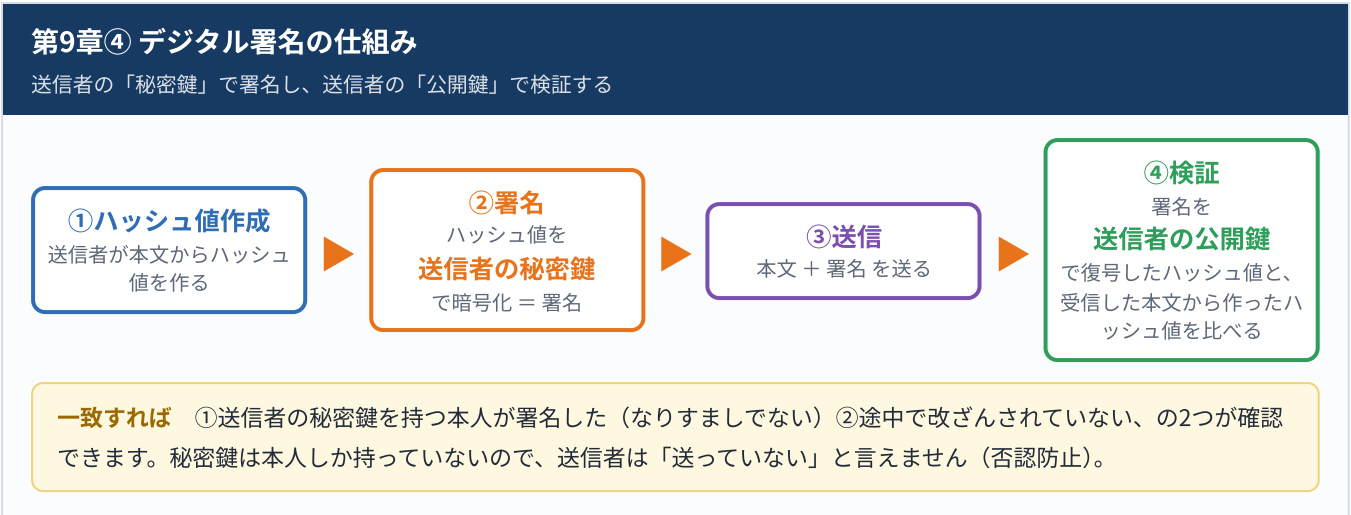
### ■ ハッシュ関数

**ハッシュ関数** は、任意の長さのデータから、**固定長の値（ハッシュ値・メッセージダイジェスト）** を作る関数です。

- 同じデータからは必ず同じハッシュ値ができる。データが1ビットでも変わると、まったく違う値になる → **改ざん検知** に使える。
- ハッシュ値から元のデータを復元できない（**一方向性**）→ パスワードの保存に使える。
- 同じハッシュ値になる別のデータを見つけるのが極めて困難（**衝突発見困難性**）。
- 代表例：**SHA-256**（SHA-2 ファミリ）、SHA-3。MD5、SHA-1 は安全性が低下しているので非推奨。
- **HMAC**：共通鍵とハッシュ関数を組み合わせて、メッセージ認証コード（MAC）を作る。改ざん検知と送信者の確認ができる。

## ■ デジタル署名

**デジタル署名** は、「**送信者本人が作成したこと（真正性）**」と「**改ざんされていないこと（完全性）**」を証明する仕組みです。暗号化とは鍵の使い方が **逆** になります。




> **ポイント　鍵の使い分けを一言で**
>
> **暗号化（秘密にしたい）**：受信者の公開鍵で暗号化 → 受信者の秘密鍵で復号。
> **署名（本人証明したい）**：送信者の秘密鍵で署名 → 送信者の公開鍵で検証。
> 「誰の」「どちらの」鍵かを必ずセットで答えるのが、午後問1の記述でも重要です。

## ■ PKI（公開鍵基盤）とデジタル証明書

公開鍵暗号には「その公開鍵は本当に本人のものか？」という問題があります。これを解決するのが **PKI** です。

- **認証局（CA）**：公開鍵とその持ち主の対応を証明する、信頼できる第三者機関。
- **デジタル証明書（公開鍵証明書）**：認証局が、利用者の公開鍵と身元情報に **認証局のデジタル署名** を付けて発行したもの。標準規格は **X.509**。
- **CRL（証明書失効リスト）**：有効期限前に失効した証明書の一覧。秘密鍵の漏えい時などに失効させる。リアルタイムに失効状態を確認するプロトコルは **OCSP**。
- **ルート証明書**：最上位の認証局が自分自身で署名した証明書。ブラウザやOSにあらかじめ組み込まれている。

### ■ TLS（SSL）

**TLS** は、Web などの通信を暗号化するプロトコルです（HTTPS は HTTP ＋ TLS）。サーバ証明書でサーバの正当性を確認し、ハイブリッド暗号方式で通信を暗号化し、MAC で改ざんを検知します。

## 4. 認証技術

### ■ 利用者認証の3要素

| 要素 | 例 |
|---|---|
| **知識情報**（知っているもの） | パスワード、暗証番号、秘密の質問 |
| **所持情報**（持っているもの） | ICカード、スマートフォン、ワンタイムパスワード生成器 |
| **生体情報**（本人の身体的特徴） | 指紋、静脈、顔、虹彩、声紋 |

- **多要素認証**：異なる要素を **2つ以上** 組み合わせる認証。パスワード＋スマホへのコードなど。（パスワード＋秘密の質問は、同じ知識情報なので多要素ではない）
- **ワンタイムパスワード**：1回限り有効なパスワード。盗まれても再利用できない。
- **チャレンジレスポンス認証**：サーバが送るランダムな値（チャレンジ）とパスワードからハッシュ値（レスポンス）を作って返す。パスワードそのものがネットワークを流れない。
- **シングルサインオン（SSO）**：1回の認証で、複数のシステムを利用できる。**SAML**、**OpenID Connect** などの仕組みがある。
- **生体認証の誤り**：本人を拒否してしまう **本人拒否率（FRR）** と、他人を受け入れてしまう **他人受入率（FAR）** はトレードオフの関係。
- **FIDO（パスキー）**：パスワードを使わず、端末内の秘密鍵と生体認証などで認証する。サーバには公開鍵だけを登録するので、サーバから漏えいしても安全。
- **CAPTCHA**：ゆがんだ文字の読取りなどで、操作しているのが人間かプログラムかを判別する。

## 5. セキュリティ対策技術

### ■ ネットワークのセキュリティ

**第9章⑤ ファイアウォールと DMZ の構成**




**ポイント**　公開サーバが乗っ取られても、DMZ から社内LANへの通信は制限されているので、被害が社内に広がりにくい構成です。

| 技術 | 内容 |
| --- | --- |
| **ファイアウォール（パケットフィルタリング）** | IPアドレス・ポート番号などのルールで通信を許可・遮断する |
| **WAF** | Web アプリケーションへの通信の **中身** を検査し、SQLインジェクションや XSS を防ぐ |
| **IDS（侵入検知システム）** | 不正な通信を **検知して通知** する |
| **IPS（侵入防止システム）** | 不正な通信を検知して **遮断** する |
| **VPN** | インターネット上に、暗号化された仮想的な専用線を作る（**IPsec**、TLS） |
| **プロキシサーバ** | 内部の端末に代わってインターネットにアクセスする。通信の記録・フィルタリング |
| **検疫ネットワーク** | 社内LANに接続する端末を一時的に隔離し、セキュリティ状態（パッチ適用など）を検査してから接続を許可する |
| **UTM** | ファイアウォール・IPS・ウイルス対策などを1台にまとめた機器 |
| **EDR** | 端末（エンドポイント）の挙動を監視し、侵入後の不審な動きを検知・対応する |
| **SIEM** | 様々な機器のログを集めて相関分析し、攻撃の兆候を見つける |
| **ゼロトラスト** | 「社内なら安全」という境界防御の考えを捨て、**すべてのアクセスを信頼せずに毎回検証** する考え方 |

## ■ メールのセキュリティ

- **SPF**：送信元メールサーバの IPアドレスを DNS に登録し、受信側で **送信元ドメインが詐称されていないか** を確認する。
- **DKIM**：送信側のメールサーバがメールに **デジタル署名** を付け、受信側が DNS の公開鍵で検証する。
- **DMARC**：SPF・DKIM の認証に失敗したメールをどう扱うか（受信拒否など）を、送信ドメインの管理者が宣言する。
- **OP25B（Outbound Port 25 Blocking）**：ISP が、自社ネットワーク内から外部のメールサーバの25番ポートへの直接の通信を遮断し、迷惑メールの送信を防ぐ。

## ■ 無線 LAN・物理的な対策

- 無線LANの暗号化は **WPA3**（WPA2）を使い、WEPは使わない。
- **入退室管理**：ICカード、生体認証。**アンチパスバック**（入室記録がないと退室できない）、**共連れ防止**（1人の認証で複数人が通るのを防ぐ）。
- **クリアデスク・クリアスクリーン**：離席時に書類を机に放置しない、画面をロックする。

## ■ 脆弱性への対策

- **セキュアプログラミング**：入力値の検証、エスケープ処理など、脆弱性を作らない書き方をする。
- **ペネトレーションテスト**：攻撃者の視点で実際にシステムに侵入を試み、脆弱性を確認する。
- **ファジング**：大量の異常なデータを入力して、未知の脆弱性を発見する。
- **CVSS**：脆弱性の深刻度を、ベンダーに依存しない共通の基準で0.0〜10.0の数値で評価する。

- **ディジタルフォレンジックス**：不正アクセスなどの事件が起きたとき、証拠となるデータを保全・解析する技術。
- **サンドボックス**：隔離された仮想環境で不審なプログラムを実行し、挙動を観察する。

## 6. セキュリティ管理

- **ISMS（情報セキュリティマネジメントシステム）**：組織として情報セキュリティを管理する仕組み。**JIS Q 27001** で要求事項が定められ、**PDCAサイクル**（Plan・Do・Check・Act）で継続的に改善する。
- **情報セキュリティポリシー**：組織のセキュリティに関する方針。**基本方針**（経営者の宣言）→ **対策基準**（何をすべきか）→ **実施手順**（具体的な手順）の3階層。
- **CSIRT**：セキュリティインシデント（事故）に対応する組織内のチーム。
- **SOC**：ネットワークやシステムを24時間監視し、攻撃を検知・分析する組織。
- **サイバーセキュリティ経営ガイドライン**：経済産業省とIPAが策定した、経営者が主導してサイバーセキュリティ対策を進めるための指針。
- **NOTICE、J-CRAT、J-CSIP** など：国や IPA によるサイバー攻撃対策の取組み。

> **過去問ではこう出る　午後問1（必須）でよく問われること**
>
> 令和7年度も、午後の問1は情報セキュリティでした。典型的な設問は「この攻撃の名称は？」「この対策で防げる理由は？」「（図の構成で）どの機器のどんな設定を変更すべきか？」「証明書の検証で確認すべきことは？」などです。本章の **攻撃と対策の対応表** と **鍵の使い分け** を説明できる状態にしておきましょう（解き方は第17章）。

### この章のまとめ

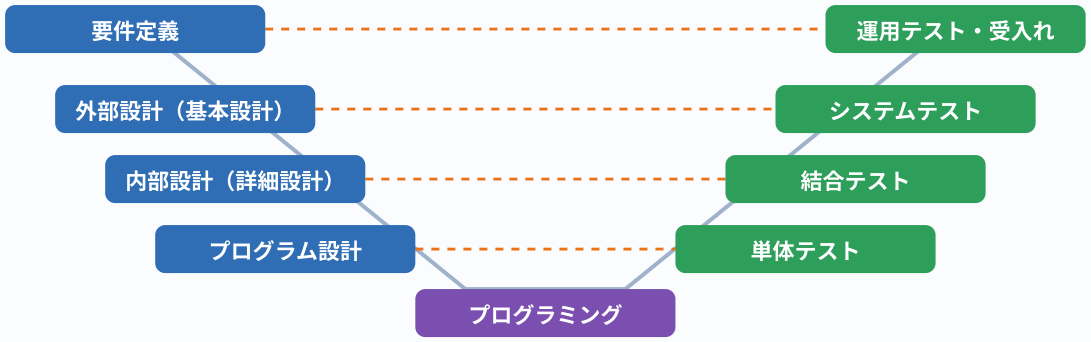
- CIA：機密性・完全性・可用性。リスク対応は回避・低減・移転・保有。
- SQLインジェクション→プレースホルダ、XSS→エスケープ、CSRF→トークン。
- 共通鍵は速いが鍵配送が課題、公開鍵は受信者の公開鍵で暗号化。実際はハイブリッド。
- デジタル署名は送信者の秘密鍵で署名・公開鍵で検証。真正性と完全性を保証。
- 証明書は認証局の署名つき公開鍵。失効は CRL／OCSP。
- 多要素認証は知識・所持・生体のうち異なる2つ以上。パスワードリスト攻撃には使い回し禁止。
- WAF は Web の中身を検査、IDS は検知、IPS は遮断。SPF・DKIM・DMARC で送信元詐称対策。

# 第10章 システム開発技術

システム開発は、「何を作るか決める（要件定義）」→「どう作るか決める（設計）」→「作る（プログラミング）」→「確かめる（テスト）」→「使い始める（導入）」という流れで進みます。この章では各工程で使う手法と、品質を高めるための考え方を学びます。午後の問8（情報システム開発）の中心テーマで、問7（組込み）や問9（PM）とも関係が深い分野です。

**第10章① 開発プロセスの全体像と V字モデル**

左側で決めたことを、右側の同じ高さのテストで確認する




## 1. 開発プロセスとモデル

### ■ 共通フレーム（SLCP）

**共通フレーム** とは、ソフトウェアの企画から開発・運用・廃棄までの作業内容や用語を標準化した枠組みです。発注者と受注者が「要件定義」などの言葉の意味を共通に理解できるようにするのが目的です（日本では「共通フレーム2013」）。

### ■ 開発モデル

| モデル | 特徴 |
|---|---|
| **ウォーターフォールモデル** | 工程を上流から下流へ順に進め、原則として前の工程に戻らない。計画や進捗管理がしやすいが、仕様変更に弱い |
| **プロトタイピングモデル** | 早い段階で試作品（プロトタイプ）を作り、利用者に確認してもらいながら要求を明確にする |
| **スパイラルモデル** | システムをいくつかの部分に分け、設計・開発・評価のサイクルを繰り返して徐々に完成させる |
| **アジャイル** | 短い期間の反復（イテレーション）で、動くソフトウェアを少しずつ開発する。変化への対応を重視 |

## 2. 要件定義

- **機能要件**：システムが何をするか（受注を登録できる、売上を集計できる）。

- **非機能要件**：性能、可用性、セキュリティ、使いやすさなど、機能以外の品質に関する要件（応答時間3秒以内、稼働率99.9%）。
- **要求の分析手法**：インタビュー、アンケート、業務の観察、**ユースケース**（利用者とシステムのやり取り）、**ユーザストーリー**（「○○として、△△したい、なぜなら□□だから」という形で要求を書く）。

## 3. 設計の手法

### ■ 構造化分析と DFD

**DFD（データフロー図）** は、業務やシステムの中で **データがどこからどこへ流れ、どう処理されるか** を表す図です。処理の順序や条件は表しません。

| 記号 | 意味 |
|---|---|
| ○（円・バブル） | プロセス（処理） |
| →（矢印） | データフロー（データの流れ） |
| ＝（二重線） | データストア（ファイル・DB） |
| □（四角） | 外部実体（データの発生源・行き先：顧客など） |

### ■ オブジェクト指向

**オブジェクト指向** は、データと、そのデータを操作する手続（メソッド）をひとまとめにした **オブジェクト** を単位としてシステムを組み立てる考え方です。

| 用語 | 意味 |
|---|---|
| **クラス** | オブジェクトの設計図（ひな形）。そこから作られた実体が **インスタンス** |
| **カプセル化** | データと手続をまとめ、内部の詳細を隠す。外部からはメソッド経由でしか操作させない（情報隠蔽） |
| **継承（インヘリタンス）** | 上位クラス（スーパクラス）の性質を、下位クラス（サブクラス）が引き継ぐ |
| **汎化と特化** | 共通部分をまとめて上位クラスを作ること（汎化）。その逆が特化。「is-a」の関係 |
| **集約と分解** | 全体と部分の関係（車とタイヤ）。「part-of」の関係 |
| **ポリモーフィズム（多相性）** | 同じメッセージを送っても、受け取ったオブジェクトによって異なる動作をする |
| **オーバーライド** | サブクラスで、スーパクラスのメソッドを再定義する |

### ■ UML

**UML** は、オブジェクト指向の設計で使う図の表記法を統一したものです。

| 図 | 表すもの |
|---|---|
| **ユースケース図** | 利用者（アクター）とシステムの機能（ユースケース）の関係 |
| **クラス図** | クラスの属性・操作と、クラス間の関係（関連・汎化・集約）、多重度 |
| **シーケンス図** | オブジェクト間のメッセージのやり取りを **時間の順序** に沿って表す |
| **コミュニケーション図** | オブジェクト間のメッセージのやり取りを、**つながり（構造）** を中心に表す |
| **アクティビティ図** | 処理の流れ（フローチャートに近い）。並行処理も表せる |
| **ステートマシン図（状態遷移図）** | オブジェクトの状態と、イベントによる状態の変化 |

## ■ モジュール設計 — 強度と結合度

プログラムを機能ごとの部品（**モジュール**）に分けるとき、良い分け方の基準が **モジュール強度** と **モジュール結合度** です。

### 第10章② 良いモジュールは「強度が高く、結合度が低い」

モジュールの中はまとまりが強く、モジュール間のつながりは弱いほど、修正の影響が広がらない

**モジュール強度（高いほど良い）**

低 ← 暗合的 ＜ 論理的 ＜ 時間的 ＜ 手順的 ＜ 連絡的 ＜ 情報的 ＜ **機能的** → 高

機能的強度：1つの機能だけを実行する

情報的強度：同じデータを扱う複数の機能をまとめ、入口を分けたもの

**モジュール結合度（低いほど良い）**

強 ← 内容 ＞ 共通 ＞ 外部 ＞ 制御 ＞ スタンプ ＞ **データ** → 弱

データ結合：必要なデータだけを引数で渡す（最良）

共通結合：グローバル変数（共通領域）でデータを共有する

内容結合：他のモジュールの内部を直接参照する（最悪）

## ■ デザインパターン・その他の設計技法

- **デザインパターン**：オブジェクト指向設計でよく使われる設計の型。Singleton（インスタンスを1つだけにする）、Observer（状態変化を複数に通知する）、Factory など。
- **MVC モデル**：アプリケーションを Model（データと処理）、View（表示）、Controller（入力の受付と制御）に分ける。
- **共通処理の部品化・再利用**：同じ処理を何度も書かず、ライブラリやフレームワークとして再利用する。

# 4. テスト

## ■ テストの種類（V字モデルの右側）

| テスト | 内容 |
|---|---|
| **単体テスト** | モジュール（部品）単位で、プログラム設計どおり動くか確認する。**ホワイトボックステスト** が中心 |
| **結合テスト** | モジュールを組み合わせて、インタフェースが正しいか確認する |
| **システムテスト** | システム全体が要件（性能・セキュリティなどの非機能要件も含む）を満たすか確認する |
| **運用テスト（受入れテスト）** | 利用者が、実際の業務の環境で使えるか確認する |
| **回帰テスト（リグレッションテスト）** | 修正や変更によって、**それまで正常だった部分に悪影響が出ていないか** を確認する |

## ■ ホワイトボックステストとブラックボックステスト

### 第10章③ テストケースの設計技法

**ホワイトボックステスト**

プログラムの **内部構造**（命令や分岐）に着目する

- **命令網羅**：すべての命令を1回以上実行
- **判定条件網羅（分岐網羅）**：すべての分岐の真・偽を1回以上実行
- **条件網羅**：判定中の個々の条件の真・偽を1回以上
- **複数条件網羅**：条件の真偽のすべての組合せ（最も厳しい）

**ブラックボックステスト**

内部構造は見ずに、**入力と出力（仕様）** に着目する

- **同値分割**：同じ扱いになる入力をグループに分け、代表値を選ぶ
- **限界値分析（境界値分析）**：グループの境目の値を選ぶ（誤りが多い）
- **原因結果グラフ・デシジョンテーブル**：入力条件の組合せと結果を整理
- **エラー推測**：経験から誤りやすい箇所を推測

**計算例　同値分割と限界値分析**

「年齢が 18以上65未満なら会員登録できる」という仕様のテストデータを選ぶ。

**同値分割**：無効（18未満）・有効（18〜64）・無効（65以上）の3グループから代表値 → 例：10, 30, 70

**限界値分析**：境界の両側 → **17, 18, 64, 65**

## ■ テストの支援と品質の評価

- **スタブ**：上位モジュールから先にテストする（トップダウンテスト）とき、未完成の **下位** モジュールの代わりをする仮のモジュール。
- **ドライバ**：下位モジュールから先にテストする（ボトムアップテスト）とき、未完成の **上位** モジュールの代わりに呼び出す仮のモジュール。
- **バグ曲線（信頼度成長曲線）**：テスト時間と発見したバグの累積数のグラフ。S字型で、終盤に増加が鈍って **収束** していれば品質が安定してきたと判断できる。

- **エラー埋込み法**：わざと既知のエラーを埋め込んでテストし、その発見率から、残っている未知のエラー数を推定する。
- **カバレッジ（網羅率）**：テストで実行された命令や分岐の割合。

## 5. レビュー

テストは「作ったものを動かして」確認しますが、**レビュー** は設計書やソースコードを「人が読んで」誤りを早期に発見する活動です。上流工程の誤りほど、後で直すコストが大きいので、早期発見が重要です。

| レビュー | 特徴 |
|---|---|
| **ウォークスルー** | 作成者が主催し、成果物を順に説明しながら参加者が誤りを指摘する |
| **インスペクション** | **モデレータ**（進行役）が主催し、役割と手順を明確に定めて行う公式なレビュー |
| **ラウンドロビン** | 参加者が持ち回りでレビュー責任者を務める |
| **ピアレビュー** | 同僚どうしで行うレビュー |
| **パスアラウンド** | 成果物を回覧して、各自がコメントする |

> **ひっかけ注意　レビューは「人」ではなく「成果物」を評価する**
>
> レビューで見つけた誤りの数を、作成者の人事評価に使ってはいけません。誤りを隠すようになり、レビューの効果がなくなるからです。選択肢で狙われやすいポイントです。

## 6. アジャイル開発と DevOps

### ■ アジャイルの代表的な手法

**アジャイル開発** は、1〜4週間程度の短い反復期間で、「計画・設計・実装・テスト」を繰り返し、動くソフトウェアを少しずつリリースしていく開発手法の総称です。

#### ▶ スクラム

| 用語 | 意味 |
|---|---|
| **プロダクトオーナー** | プロダクトの価値を最大化する責任者。プロダクトバックログの優先順位を決める |
| **スクラムマスター** | スクラムが正しく実践されるよう支援する。チームの障害を取り除く |
| **開発者（開発チーム）** | 実際にプロダクトを作る。自己組織化されたチーム |
| **スプリント** | 1か月以内の固定された反復期間 |
| **プロダクトバックログ** | プロダクトに必要な機能などを優先順位順に並べた一覧 |
| **スプリントバックログ** | 今回のスプリントで実施する項目と作業の計画 |
| **デイリースクラム** | 毎日15分程度で行う短い打合せ |
| **スプリントレビュー** | スプリントの成果物を関係者に見せ、フィードバックを得る |
| **スプリントレトロスペクティブ** | チームの進め方を振り返り、改善策を決める |

### ▶ XP（エクストリームプログラミング）のプラクティス

- **ペアプログラミング**：2人1組で、1人がコードを書き、もう1人がその場でレビューする。
- **テスト駆動開発（TDD）**：**先にテストケースを書き**、そのテストに合格するようにコードを書く。
- **リファクタリング**：外部から見た動作を変えずに、プログラムの内部構造を改善する。
- **継続的インテグレーション（CI）**：コードの変更を頻繁に統合し、自動でビルド・テストする。

### ■ DevOps と CI/CD

- **DevOps**：開発（Development）と運用（Operations）が協力し、ツールを活用して、迅速かつ安全にリリースを繰り返す考え方。
- **CI/CD**：継続的インテグレーション／継続的デリバリ（デプロイ）。ビルド・テスト・リリースを自動化するパイプラインを作る。
- **バージョン管理**：Git などで、ソースコードの変更履歴を管理する。
- **マイクロサービスアーキテクチャ**：システムを小さな独立したサービスの集まりとして構築し、API で連携させる。サービスごとに独立して開発・リリースできる。

## 7. 見積り・保守・知的財産の活用

### ■ ソフトウェアの規模・工数見積り

| 手法 | 内容 |
|---|---|
| **ファンクションポイント法（FP法）** | 外部入力・外部出力・内部論理ファイル・外部インタフェースファイル・外部照会の **機能の数と複雑さ** から規模を見積もる。利用者から見た機能で測るので、開発言語に依存しない |
| **COCOMO** | 見積もったプログラムの規模（ステップ数）に、補正係数を掛けて工数を算出する |
| **類推見積法** | 過去の類似プロジェクトの実績を基に見積もる |
| **ボトムアップ見積法** | 作業を細かく分け、それぞれの見積りを積み上げる |
| **プログラムステップ法（LOC法）** | ソースコードの行数で規模を見積もる |

### ■ 保守・リバースエンジニアリング

- **保守の種類**：是正保守（障害の修正）、適応保守（環境変化への対応）、完全化保守（性能や保守性の改善）、予防保守（潜在的な障害を事前に修正）。
- **リバースエンジニアリング**：既存のソフトウェアを解析して、仕様や設計を明らかにする。
- **リエンジニアリング**：リバースエンジニアリングで得た仕様を基に、新しいシステムを作り直す。

### この章のまとめ

- V字モデル：要件定義↔運用テスト、外部設計↔システムテスト、内部設計↔結合テスト、プログラム設計↔単体テスト。
- 非機能要件＝性能・可用性・セキュリティなど。DFD はデータの流れ（処理順序は表さない）。
- シーケンス図は時間順、ステートマシン図は状態変化。強度は機能的、結合度はデータ結合が最良。
- 判定条件網羅＝分岐の真偽、複数条件網羅が最も厳しい。限界値分析は境界の両側を選ぶ。
- スタブは下位の代役、ドライバは上位の代役。回帰テストは変更の悪影響の確認。
- スクラム：PO が優先順位、スクラムマスターが支援。TDD はテストが先。FP法は機能の数で規模を見積もる。

# 第11章 プロジェクトマネジメント

**プロジェクト** とは、「独自の成果物を作るために、**期間を区切って** 行う活動」です。システム開発はその代表例です。期間・予算・人員が限られた中で、目標を達成するための管理手法を **プロジェクトマネジメント** といいます。この章からマネジメント系です。計算問題（クリティカルパス、EVM）は確実に得点しましょう。午後の問9の土台です。

**第11章① プロジェクトマネジメントの全体像（PMBOK の考え方）**

立上げ → 計画 → 実行 → 監視・コントロール → 終結




**管理する対象（知識エリア）** 統合・スコープ・スケジュール・コスト・品質・資源・コミュニケーション・リスク・調達・ステークホルダー の10分野。PMBOK 第7版では「原理・原則」と「パフォーマンス領域」を中心とした考え方に変わりましたが、試験では両方の用語が出ます。

## 1. プロジェクトの基本

- **プロジェクト憲章**：プロジェクトを正式に認可する文書。目的、概要、プロジェクトマネージャの任命と権限などを記載する。
- **ステークホルダー**：プロジェクトに利害関係を持つ人や組織（発注者、利用者、開発メンバ、経営者など）。
- **PMO**：組織内の複数のプロジェクトを横断的に支援・管理する部門。
- **プロジェクトの組織形態**：機能型組織（部門に所属したまま参加）、プロジェクト型組織（専任のチームを作る）、**マトリックス型組織**（部門とプロジェクトの両方に所属）。

## 2. スコープマネジメント — WBS

**スコープ** とは、プロジェクトで作る成果物と、そのために必要な作業の範囲です。

**WBS（Work Breakdown Structure）** は、プロジェクトの成果物や作業を、**階層的に細かく分解** した図です。最下位の要素を **ワークパッケージ** といい、これがコストや日程を見積もり、担当者を割り当てる単位になります。

- WBS に含まれない作業は、プロジェクトのスコープ外。
- **スコープクリープ**：正式な手続を経ずに、作業範囲がなし崩しに広がってしまうこと。変更は **変更管理** の手続で行う。

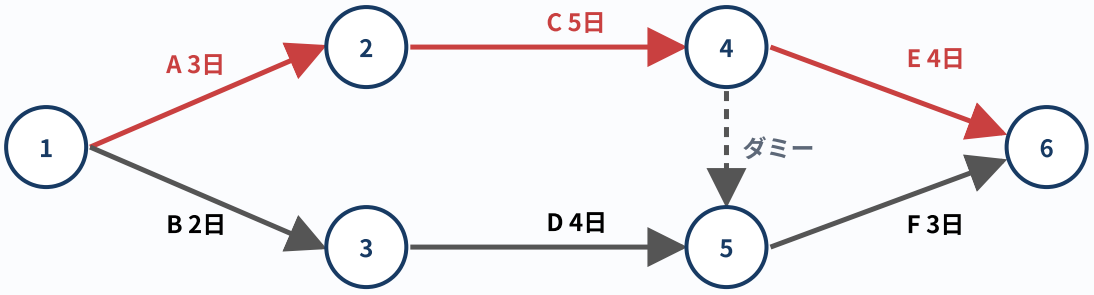
# 3. スケジュールマネジメント

## ■ アローダイアグラム（PERT図）

**アローダイアグラム** は、作業の順序関係と所要日数を矢印で表した図です。

- 矢印（→）：**作業**（アクティビティ）。矢印に作業名と所要日数を書く。
- 丸（◯）：**結合点**（作業の開始・終了のタイミング）。
- 点線の矢印：**ダミー作業**（所要日数0。順序関係だけを表す）。

### 第11章② アローダイアグラムとクリティカルパス




**読み方**　作業 F は D と（ダミー経由で）C の両方が終わらないと始められません。最も時間がかかる経路 A→C→E（3＋5＋4＝**12日**）がクリティカルパスです。

**計算例**　**クリティカルパスと余裕日数**

上の図ですべての経路を列挙して所要日数を足す。

・A→C→E：3＋5＋4 ＝ **12日**　・A→C→(ダミー)→F：3＋5＋3 ＝ 11日　・B→D→F：2＋4＋3 ＝ 9日

最長の **A→C→E（12日）** がクリティカルパス ＝ プロジェクト全体の最短所要日数です。

**最早開始日**（最も早く始められる日）：F は C 終了（8日目）と D 終了（6日目）の **遅い方** → 8日目から。

**最遅開始日**（全体を遅らせずに済む最も遅い開始日）：F は 12 − 3 ＝ 9日目まで。余裕日数（フロート）＝ 9 − 8 ＝ **1日**。

- **クリティカルパス**：開始から終了までの経路のうち、**所要日数が最も長い経路**。この上の作業が1日遅れると、プロジェクト全体が1日遅れる。クリティカルパス上の作業の余裕日数は0。
- **ガントチャート**：横軸に時間をとり、作業ごとの予定と実績を横棒で表す。進捗が見やすいが、作業間の依存関係はわかりにくい。

## ■ スケジュール短縮の技法

| 技法 | 内容 | 注意点 |
|---|---|---|
| **クラッシング** | クリティカルパス上の作業に **資源（人・費用）を追加投入** して期間を短縮する | コストが増える |
| **ファストトラッキング** | 本来は順番に行う作業を **並行して** 行う | 手戻りのリスクが増える |

**ポイント　短縮するならクリティカルパス上の作業**

クリティカルパス以外の作業を短縮しても、全体の期間は短くなりません。また、短縮した結果、別の経路が新しいクリティカルパスになることがあるので注意します。

## 4. コストマネジメント — EVM

**EVM（アーンドバリューマネジメント）** は、プロジェクトの進捗とコストを、**金額という共通の尺度** で同時に管理する手法です。

### 第11章③ EVM の3つの指標と4つの評価値

| 指標 | 意味 | 覚え方 |
|---|---|---|
| **PV**（Planned Value：計画価値） | 現時点までに **完了している予定だった** 作業の予算 | 予定 |
| **EV**（Earned Value：出来高） | 現時点までに **実際に完了した** 作業の予算 | 出来た分 |
| **AC**（Actual Cost：実コスト） | 現時点までに **実際にかかった** 費用 | 使ったお金 |

| 評価値 | 式 | 判断 |
|---|---|---|
| **SV**（スケジュール差異） | EV − PV | マイナスなら **遅れ** |
| **CV**（コスト差異） | EV − AC | マイナスなら **予算超過** |
| **SPI**（スケジュール効率指数） | EV ÷ PV | 1未満なら遅れ |
| **CPI**（コスト効率指数） | EV ÷ AC | 1未満なら予算超過 |

**計算例　EVM の計算**

予算総額（BAC）1,000万円のプロジェクト。現時点で PV＝500万円、EV＝400万円、AC＝450万円。

SV ＝ 400 − 500 ＝ **−100万円**（遅れている）／ CV ＝ 400 − 450 ＝ **−50万円**（予算超過）

CPI ＝ 400 ÷ 450 ≒ 0.89

**完了時総コスト見積り（EAC）**：このままの効率が続くとすると、EAC ＝ BAC ÷ CPI ＝ 1,000 ÷ 0.89 ≒ **約1,125万円**

**残作業のコスト見積り（ETC）** ＝ EAC − AC ≒ 1,125 − 450 ＝ 675万円

**ポイント　EV が主役**

SV も CV も「EV から引く」形です。EV（出来高）を基準に、「予定（PV）より進んでいるか」「使ったお金（AC）より価値を生んでいるか」を見ると覚えましょう。

### ■ 工数と生産性の計算

**計算例　要員計画**

全体で 60人月の作業を、6か月で完了したい。必要な要員数は？
60人月 ÷ 6か月 ＝ **10人**
途中から要員を追加すると、教育やコミュニケーションの負荷が増えて生産性が下がることも考慮します（「遅れているプロジェクトに要員を追加すると、さらに遅れる」＝ブルックスの法則）。

## 5. 品質・資源・コミュニケーション

### ■ 品質マネジメント

- **品質計画**：品質の基準と、それを満たす方法を決める。**品質保証**：計画どおりのプロセスで作業しているかを監査する。**品質コントロール**：成果物を検査し、基準を満たしているか確認する。
- **品質管理の手法（QC七つ道具）**は第16章で詳しく扱う（パレート図、特性要因図、管理図など）。

### ■ 資源マネジメント

- **責任分担マトリックス（RAM）**：作業と担当者の対応を表にしたもの。**RACIチャート** では、実行責任者（Responsible）、説明責任者（Accountable）、相談先（Consulted）、報告先（Informed）を明示する。
- **タックマンモデル**：チームは「形成期 → 混乱期 → 統一期 → 機能期 → 散会期」の段階を経て成長する。

### ■ コミュニケーションマネジメント

**計算例　コミュニケーション経路の数**

メンバが n 人のとき、2人の組合せでできるコミュニケーション経路の数は $n(n-1)/2$。
5人 → 5×4÷2 ＝ 10本。10人 → 10×9÷2 ＝ **45本**。人数が増えると経路は急激に増えます。

## 6. リスクマネジメント

**リスク** とは、発生すればプロジェクトの目標に影響を与える **不確実な事象** です。マイナスの影響（脅威）だけでなく、プラスの影響（好機）も含みます。

### ■ リスクの分析

- **定性的リスク分析**：発生確率と影響度を「高・中・低」などで評価し、優先順位を付ける（発生確率・影響度マトリックス）。
- **定量的リスク分析**：数値で評価する。**期待金額価値（EMV）** ＝ 発生確率 × 影響額。**デシジョンツリー**、**モンテカルロシミュレーション**、**感度分析** など。

**計算例　期待金額価値**

発生確率 20%、発生時の損失 500万円のリスクの期待金額価値 ＝ 0.2 × 500 ＝ **100万円**。対策費用が100万円未満なら、対策する価値があると判断できます。

### ■ リスク対応戦略

| 脅威（マイナスのリスク）への対応 | 内容 |
|---|---|
| **回避** | 計画を変更して、リスクそのものを取り除く |
| **転嫁（移転）** | 保険や契約によって、影響を第三者に移す |
| **軽減** | 発生確率や影響を小さくする |
| **受容** | 対策をとらずに受け入れる（予備費を確保するなど） |
| **エスカレーション** | プロジェクトの権限を超えるので、上位の組織に委ねる |

好機（プラスのリスク）への対応は、**活用・共有・強化・受容** です。

## 7. 調達マネジメントと契約

- **RFI（情報提供依頼書）**：ベンダーに、技術や製品の情報提供を依頼する。
- **RFP（提案依頼書）**：ベンダーに、システムの要件を示して **具体的な提案** を依頼する。
- **契約形態**：**定額契約**（金額を事前に固定。コスト超過のリスクは受注者が負う）、**実費償還契約**（実際にかかった費用＋報酬を支払う。リスクは発注者が負う）、**タイムアンドマテリアル契約**（作業時間と資材の単価で支払う）。
- 請負契約・準委任契約の違いは第16章（法務）で扱います。

> **この章のまとめ**
>
> - WBS の最下位はワークパッケージ。スコープ外の作業は WBS に含まれない。
> - クリティカルパスは最長経路で余裕日数0。最早開始日は前作業の遅い方、最遅開始日は後ろから逆算。
> - クラッシング＝資源追加（コスト増）、ファストトラッキング＝並行作業（手戻りリスク増）。
> - EVM：SV＝EV−PV、CV＝EV−AC、SPI＝EV/PV、CPI＝EV/AC。EAC＝BAC/CPI。
> - コミュニケーション経路 n(n−1)/2。期待金額価値＝確率×影響額。
> - 脅威への対応：回避・転嫁・軽減・受容（＋エスカレーション）。

# 第12章 サービスマネジメント

システムは作って終わりではありません。完成したシステムを使って、利用者に **ITサービス** を安定して提供し続けることが必要です。そのための管理の仕組みが **ITサービスマネジメント** です。「インシデント管理」と「問題管理」の違いのような、似た用語の区別が頻出です。午後の問10の土台です。




## 1. ITサービスマネジメントの枠組み

- **ITIL**：ITサービスマネジメントの **ベストプラクティス（成功事例）** をまとめた書籍群。英国政府が作成した。最新の ITIL 4 では「サービスバリューシステム（SVS）」として、価値の共創を重視している。
- **JIS Q 20000（ISO/IEC 20000）**：ITサービスマネジメントシステム（SMS）の **要求事項** を定めた規格。認証の基準になる。
- **PDCAサイクル**：計画（Plan）→実行（Do）→点検（Check）→処置（Act）を繰り返して、サービスを継続的に改善する。

## 2. サービスレベル管理

- **SLA（Service Level Agreement：サービスレベル合意書）**：サービス提供者と利用者（顧客）の間で、**サービスの内容と品質の目標値** を合意した文書。稼働率、応答時間、障害時の復旧時間、問合せ受付時間などを数値で定める。
- **SLM（サービスレベル管理）**：SLA を合意し、実績を監視・報告し、改善を続けるプロセス。
- **OLA（運用レベル合意書）**：サービス提供者の **組織内部の部門間** で結ぶ合意。
- **UC（Underpinning Contract：裏付け契約）**：サービス提供者が **外部の供給者** と結ぶ契約。
- **SLO**：SLA の中の個々のサービスレベル目標値。

> **計算例　SLA の稼働率から許容停止時間を求める**
>
> 「月間稼働率 99.9%以上」を合意した。1か月を30日とすると、許容される停止時間は？
>
> 30日 × 24時間 ＝ 720時間。停止してよい割合 ＝ 0.1% → 720 × 0.001 ＝ **0.72時間 ＝ 約43分**
>
> 計画停止（メンテナンス）を稼働率の計算に含めるかどうかも、SLA で決めておく必要があります。

## 3. インシデント管理と問題管理

### 第12章② インシデント管理と問題管理の違い（最頻出）

**インシデント管理**

目的：**サービスを可能な限り迅速に回復**させ、業務への影響を最小限にする

・原因がわからなくても、回避策（ワークアラウンド）で暫定的に復旧してよい

・記録 → 分類・優先度付け → 診断 → 解決 → 終了

・解決できなければ専門部署へ **エスカレーション**

**問題管理**

目的：インシデントの **根本原因** を特定し、**再発を防止**する

・複数のインシデントの傾向を分析し、未知の原因を突き止める

・根本原因と回避策がわかったものを **既知の誤り** として記録する

・恒久的な解決には、変更管理に変更要求（RFC）を出す

- **インシデント**：サービスの計画外の中断や、品質の低下（サーバが止まった、画面の表示が遅い）。
- **サービス要求**：パスワードの再設定やソフトウェアの使い方の質問など、障害ではない定型的な依頼。**サービス要求管理** で扱う。
- **優先度**：**影響度**（業務への影響の大きさ）と **緊急度**（解決までに許される時間）を組み合わせて決める。
- **エスカレーション**：自分たちで解決できない場合に、より専門的な担当（**機能的エスカレーション**）や上位の管理者（**階層的エスカレーション**）に引き継ぐこと。

## 4. 変更管理・リリース管理・構成管理

- **変更管理**：サービスやシステムへの変更を、**リスクと影響を評価してから承認** し、計画的に実施するプロセス。変更の申請は **RFC（変更要求）** で行い、**CAB（変更諮問委員会）** が評価する。緊急の変更は ECAB（緊急変更諮問委員会）が扱う。
- **リリース管理及び展開管理**：承認された変更を、本番環境に確実に展開（リリース）する。展開前のテスト、展開の計画、失敗時の **切り戻し（ロールバック）** 計画を立てる。
- **構成管理**：サービスを構成する要素（**構成品目：CI**）――ハードウェア、ソフトウェア、ドキュメントなど――の情報と相互関係を **CMDB（構成管理データベース）** に記録し、常に最新の状態に保つ。

## 5. サービスデスク

**サービスデスク** は、利用者からの問合せやインシデントの報告を受け付ける **単一の窓口（SPOC：Single Point Of Contact）** です。

| 形態 | 内容 |
|---|---|
| **ローカルサービスデスク** | 利用者の拠点ごとに設置する |
| **中央サービスデスク** | 1か所に集約する。効率がよい |
| **バーチャルサービスデスク** | 担当者は各地に分散しているが、ネットワークで1つの窓口に見せる |
| **フォロー・ザ・サン** | 時差のある複数の拠点で引き継ぎながら、24時間対応する |

- **FAQ・チャットボット**：利用者自身が解決できるようにして（セルフヘルプ）、問合せを減らす。

- **一次解決率**：最初の窓口で解決できた割合。サービスデスクの評価指標。

## 6. 可用性管理・容量管理・サービス継続管理

- **可用性管理**：合意した可用性（稼働率）を達成するために、冗長化や保守を計画する。MTBF・MTTR の考え方は第4章を参照。
- **容量・能力管理（キャパシティ管理）**：CPU・メモリ・ネットワークの使用状況を監視し、**将来の需要を予測**して、必要な容量を計画する。
- **サービス継続管理**：災害などの重大な事態でも、合意したレベルでサービスを継続・復旧できるようにする。
- **BCP（事業継続計画）**：災害やシステム障害が起きても、重要な業務を継続・早期復旧するための計画。
- **RTO（目標復旧時間）**：障害発生から **いつまでに** 復旧させるか。**RPO（目標復旧時点）**：障害発生時に、**どの時点まで** のデータを復旧させるか（＝どれだけのデータ損失を許容するか）。

> **ポイント　RTO と RPO の区別**
>
> RTO は「Time（どれだけ早く戻すか）」、RPO は「Point（どの時点のデータまで戻すか）」。RPO を「直前」にするなら、リアルタイムのレプリケーション（複製）が必要になります。

## 7. サービスの運用

- **ジョブ管理**：バッチ処理などのジョブの実行スケジュールを管理し、自動実行・監視する。
- **バックアップ**：フル・差分・増分（第5章）。バックアップの媒体を **遠隔地に保管** して災害に備える。
- **監視**：死活監視（ping などで生きているか確認）、性能監視、ログ監視。
- **SRE（サイトリライアビリティエンジニアリング）**：ソフトウェアエンジニアリングの手法でシステム運用の信頼性を高める考え方。**エラーバジェット**（SLO で許容される失敗の量）の範囲内で新機能のリリースを進める。

## 8. ファシリティマネジメント

**ファシリティマネジメント** とは、コンピュータを設置する建物や設備を、最適な状態に保つ管理活動です。

| 設備 | 内容 |
|---|---|
| **UPS（無停電電源装置）** | 停電時にバッテリーから **一時的に** 電力を供給する。その間にシステムを安全に停止させる |
| **自家発電装置** | 長時間の停電に備えて発電する（UPS とセットで使う） |
| **サージ保護** | 落雷などによる異常な電圧（サージ）から機器を守る |
| **免震・耐震** | 地震の揺れを伝えにくくする／揺れに耐える |
| **データセンタ** | サーバを安全に運用するための専用施設。空調、電源、セキュリティが整っている |

- **PUE**：データセンタ全体の消費電力 ÷ IT機器の消費電力。**1に近いほど省エネ**。
- **グリーンIT**：IT機器の省エネや、ITを使った社会全体の環境負荷低減。

### この章のまとめ

- SLA は提供者と顧客の合意、OLA は内部部門間、UC は外部供給者との契約。
- インシデント管理＝迅速な回復（暫定対応可）、問題管理＝根本原因の究明と再発防止。
- 変更は RFC で申請し CAB が評価。構成情報は CMDB。サービスデスクは単一窓口（SPOC）。
- RTO は復旧までの時間、RPO はどの時点のデータまで戻すか。
- UPS は一時的な電力供給、長時間は自家発電。PUE は1に近いほど良い。

# 第13章 システム監査

**システム監査** とは、情報システムが適切に作られ、運用されているかを、**独立した立場の監査人** が点検・評価し、経営者に報告することです。健康診断のようなもので、問題点を見つけて改善を促します。マネジメント系の中でも独特の考え方（監査人の立場、監査証拠）が問われます。午後の問11の土台です。

**第13章① システム監査の流れ**

監査は「計画 → 実施 → 報告 → フォローアップ」の順に進む




## 1. システム監査の基本

### ■ システム監査の目的と基準

- **目的**：情報システムにまつわるリスクに適切に対処しているかを、独立かつ専門的な立場で点検・評価・検証し、**ガバナンス・マネジメント・コントロールが適切であるという保証を与える**、または **改善のための助言を行う**。
- **システム監査基準**：監査人の行動規範（監査をどう行うか）を定めたもの。
- **システム管理基準**：監査の際の **判断の尺度**（システムがどうあるべきか）を定めたもの。
- **保証型監査**：適切であることを保証する。**助言型監査**：改善点を助言する。

### ■ システム監査人の独立性

監査人は、公正な判断をするために、監査対象から独立していなければなりません。

- **外観上の独立性**：監査対象の部門に所属していない、利害関係がないなど、**外から見ても** 独立していること。例えば、開発を担当した人が、そのシステムを監査してはいけない。
- **精神上の独立性**：偏りのない、**公正かつ客観的な** 姿勢で判断すること。
- 監査人は **改善を指示・命令する権限を持たない**。あくまで改善を **勧告（提案）** する立場。改善を実施するのは監査対象の部門（被監査部門）であり、改善を指示するのは経営者。

> **ひっかけ注意　監査人は「改善の実施」をしない**
>
> 「監査人が問題点を修正した」「監査人が改善を指示した」という選択肢は誤りです。監査人は、指摘事項と改善提案を監査報告書にまとめて **依頼者（経営者など）に報告** し、改善の状況を **フォローアップで確認** するまでが役割です。

## 2. 監査証拠と監査技法

### ■ 監査証拠と監査調書

- **監査証拠**：監査人が結論（監査意見）を出すための根拠となる事実。文書、記録、ログ、ヒアリング結果など。
- **監査調書**：監査人が実施した監査手続の内容や、入手した監査証拠、結論に至った過程を記録した文書。**監査の結論の裏付け** となる。
- 監査証拠は、十分かつ適切でなければならない。

### ■ 監査技法

| 技法 | 内容 |
|---|---|
| **チェックリスト法** | あらかじめ作成した質問のリストに回答してもらう |
| **ドキュメントレビュー法** | 関連する文書（規程、設計書、議事録）を閲覧して確認する |
| **インタビュー法** | 関係者に直接質問して回答を得る |
| **ウォークスルー法** | データの生成から出力までの流れを、記録をたどって追跡する |
| **突合・照合法** | 関連する複数の記録を突き合わせて、一致しているか確認する |
| **現地調査法** | 実際に現場に出向いて、作業の状況を観察する |
| **CAAT（コンピュータ支援監査技法）** | 監査人がコンピュータを使って監査する。**テストデータ法**（監査人が用意したデータを処理させ、結果を予想と比べる）、**ITF法**（本番システムに監査用の架空口座を設けて処理を確認する）、**並行シミュレーション法**（監査人が作ったプログラムと本番システムに同じデータを処理させ、結果を比べる）など |

> **ポイント　監査の問題の解き方**
> 「〇〇を確かめるために、監査人が確認すべき監査証拠（記録）はどれか」という問題が多く出ます。「何を確かめたいのか」と「その事実を **客観的に** 示す記録は何か」を結び付けて考えます。例：アクセス権限の見直しが定期的に行われているか → 権限の見直しの **実施記録**（承認の記録）を確認する。

## 3. 内部統制

### ■ 内部統制とは

**内部統制** とは、企業が健全かつ効率的に事業を行うために、**組織内部で整備・運用するルールや仕組み** です。経営者が整備の責任を持ちます。

**第13章② 内部統制の4つの目的と6つの基本的要素（金融庁の基準）**

**4つの目的**

- 業務の有効性及び効率性
- 報告の信頼性（財務報告など）
- 事業活動に関わる法令等の遵守
- 資産の保全

**6つの基本的要素**

- 統制環境（組織の気風、経営者の姿勢）
- リスクの評価と対応
- 統制活動（承認、職務分掌など）
- 情報と伝達
- モニタリング（監視活動）
- **IT（情報技術）への対応**

- **職務分掌**：1人に権限が集中しないように、担当業務を分けること。例えば、「発注の担当者」と「その発注を承認する人」を別にする。「プログラムを開発する人」と「本番環境に移行する人（運用者）」を分ける。不正やミスの防止に重要。
- **J-SOX（内部統制報告制度）**：上場企業に、財務報告に関する内部統制を評価した **内部統制報告書** の提出を義務付ける制度（金融商品取引法）。
- **IT全般統制**：ITを利用した業務全体を支える基盤の統制（システムの開発・変更管理、運用管理、アクセス管理など）。
- **IT業務処理統制**：個々の業務処理が正しく行われることを確保する統制（入力データのチェック、エラー処理など）。

## 4. ITガバナンスとコンプライアンス

- **ITガバナンス**：**経営者が主導** して、ITを活用するための戦略を立て、IT投資の効果やリスクを適切に管理・統制する仕組み。JIS Q 38500 では「**評価（Evaluate）・指示（Direct）・モニタ（Monitor）**」の3つのタスクを示している。
- **コーポレートガバナンス（企業統治）**：経営者を監視し、企業が健全に経営されるようにする仕組み（社外取締役の設置など）。
- **コンプライアンス（法令遵守）**：法律だけでなく、社会規範や企業倫理を守ること。
- **COBIT**：ITガバナンスのフレームワーク。

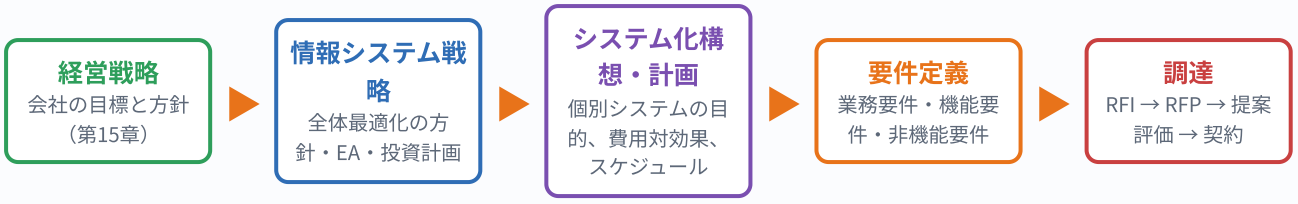
**この章のまとめ**

- 監査は 計画 → 予備調査 → 本調査 → 報告 → フォローアップ。報告先は依頼者（経営者など）。
- 監査人は外観上・精神上の独立性が必要。改善は勧告するだけで、指示や実施はしない。
- 監査調書は結論の裏付け。CAAT：テストデータ法、ITF法、並行シミュレーション法。
- 内部統制の要素に「ITへの対応」。職務分掌で権限を分ける。
- ITガバナンスは経営者が主導（評価・指示・モニタ）。

# 第14章 システム戦略とシステム企画

ここからはストラテジ系です。技術そのものではなく、「**会社の目的を達成するために、ITをどう活用するか**」を考える分野です。この章では、経営戦略に沿って情報システムの全体計画を立て（システム戦略）、個々のシステム化を企画し、外部のベンダーから調達するまでの流れを学びます。午後の問2（経営戦略）や問8（情報システム開発）の背景知識にもなります。

**第14章① 経営戦略からシステム調達までの流れ**




**大原則**　情報システム戦略は、必ず **経営戦略に基づいて** 立てます。「ITありき」ではなく「経営目標を達成する手段としてのIT」です。

## 1. 情報システム戦略

### ■ 全体最適化

部門ごとにバラバラにシステムを作ると、データが重複したり連携できなかったりします（部分最適）。会社全体として最も効果が上がるように、システムを計画・統合することを **全体最適化** といいます。

- **CIO（最高情報責任者）**：経営的な視点から、情報システム戦略の立案と実行に責任を持つ役員。
- **情報システム化委員会（IT戦略委員会）**：経営層や各部門の代表が参加し、全社的なIT投資の優先順位などを決める。

### ■ EA（エンタープライズアーキテクチャ）

**EA** は、組織全体の業務とシステムを統一的な手法でモデル化し、**現状（As-Is）と理想（To-Be）を描いて**、全体最適化を進めるための手法です。次の4つの体系で整理します。

| 体系 | 内容 |
|---|---|
| **ビジネスアーキテクチャ** | 業務の内容・流れ（業務体系） |
| **データアーキテクチャ** | 業務で使うデータの構造（データ体系） |
| **アプリケーションアーキテクチャ** | 業務を実現するシステムの構成（処理体系） |
| **テクノロジアーキテクチャ** | ハードウェア・ソフトウェア・ネットワークなどの技術基盤（技術体系） |

### ■ 業務プロセスの改善

- **BPR（ビジネスプロセスリエンジニアリング）**：既存の業務プロセスを **根本的に見直し、抜本的に再設計** すること。

- **BPM（ビジネスプロセスマネジメント）**：業務プロセスを、PDCA サイクルで **継続的に** 改善していく管理手法。
- **BPMN**：業務プロセスを図で表すための標準的な表記法。
- **BPO（ビジネスプロセスアウトソーシング）**：自社の業務プロセスの一部（経理、人事、コールセンタなど）を、外部の専門業者に委託すること。
- **ワークフローシステム**：申請・承認などの業務の流れを電子化して管理する。

### ■ 業務モデリングの手法

DFD（第10章）、E-R図（第7章）、UML（第10章）のほかに、業務の分析では次のような手法を使います。

- **As-Is / To-Be 分析**：現状の姿と、あるべき姿を比べて、ギャップ（差）を明確にする。
- **ペルソナ・カスタマージャーニーマップ**：顧客がサービスを知り、利用し、評価するまでの行動や感情の流れを可視化する。

## 2. ソリューションビジネスと IT 活用

### ■ 代表的なソリューション

| ソリューション | 内容 |
|---|---|
| **ERP（統合基幹業務システム）** | 販売・生産・会計・人事など、企業の基幹業務を統合し、データを一元管理するパッケージ |
| **SFA（営業支援システム）** | 商談の進捗や顧客との接触履歴を管理し、営業活動を効率化する |
| **CRM（顧客関係管理）** | 顧客情報を一元管理し、顧客との長期的な関係を築いて、顧客満足度と収益を高める |
| **SCM（サプライチェーンマネジメント）** | 調達・生産・物流・販売の一連の流れ（供給連鎖）を、**企業の枠を越えて** 最適化する |
| **RPA** | パソコン上で人が行う **定型的な操作** を、ソフトウェアのロボットで自動化する |
| **グループウェア** | スケジュール共有、電子会議室など、組織の共同作業を支援する |

### ■ DX（デジタルトランスフォーメーション）

**DX** とは、企業がデータとデジタル技術を活用して、製品・サービス・ビジネスモデルを変革するとともに、業務・組織・企業文化も変革して、競争上の優位性を確立することです（経済産業省の定義）。単なるIT化（デジタイゼーション：アナログをデジタルにする、デジタライゼーション：業務プロセスをデジタル化する）とは区別されます。

- **DX推進指標**：経済産業省が示した、企業が DX の取組み状況を自己診断するための指標。
- **2025年の崖**：複雑化・老朽化した既存システム（**レガシーシステム**）が DX の足かせとなり、大きな経済損失が生じるおそれがあるという、経済産業省の警告。
- **デジタルツイン**：現実世界の機器や設備から集めたデータを基に、仮想空間に「双子」を再現して、シミュレーションや予測に使う技術。

### ■ システム活用促進

- **情報リテラシー**：情報を収集・活用する能力。
- **データ利活用**：社内外のデータを分析して意思決定に活かす（**データドリブン経営**）。
- **ディジタルディバイド**：情報技術を使える人と使えない人の間に生じる格差。

## 3. システム企画

### ■ システム化構想・システム化計画

- **システム化構想**：経営上の課題を整理し、どのようなシステムを作るか、その **目的と概要** を明確にする。
- **システム化計画**：システム化構想を具体化し、**費用対効果、スケジュール、体制、リスク** などを計画する。
- **投資効果の評価**：**ROI（投資利益率）** ＝ 利益（効果）÷ 投資額 × 100%。NPV（正味現在価値）、回収期間法なども使う（第16章）。

> **計算例　ROI の計算**
>
> システム投資額 2,000万円で、5年間の累計で 3,000万円の利益増加が見込める。
> ROI ＝（3,000 − 2,000）÷ 2,000 × 100 ＝ **50%**（利益を「効果 − 投資」とする場合）
> 問題によって「利益」の定義が異なるので、問題文の式に従って計算します。

### ■ 要件定義

- **業務要件**：業務上、何を実現したいか（業務の流れ、業務ルール）。
- **機能要件・非機能要件**（第10章参照）。非機能要件の整理には、IPA の **非機能要求グレード**（可用性・性能・拡張性・運用・保守性・移行性・セキュリティなどの項目）が使われる。
- **要件の合意**：利害関係者の間で要件の優先順位を調整し、合意を得て文書化する。

## 4. 調達

### ■ 調達の流れ

**第14章② 調達の流れ**




- **RFI（Request For Information）**：調達の前段階で、ベンダーに技術動向や製品・サービスの情報の提供を依頼する文書。
- **RFP（Request For Proposal）**：システムの目的、要件、契約条件などを示して、ベンダーに **提案書の提出を依頼する** 文書。全ベンダーに **同じ条件** を示し、公平に比較できるようにする。

- **RFQ（Request For Quotation）**：見積りを依頼する文書。
- **提案の評価**：評価項目と配点（重み）を **事前に決めて** おき、各社の提案を点数化して比較する。価格だけでなく、技術力、実績、体制なども評価する。

> **計算例　重み付け評価**
>
> 評価項目の重みが「機能 0.5、価格 0.3、実績 0.2」、A社の評点が「機能 8、価格 6、実績 9」のとき、A社の総合点は？
>
> $8 \times 0.5 + 6 \times 0.3 + 9 \times 0.2 = 4.0 + 1.8 + 1.8 =$ **7.6点**

## ■ グリーン調達・その他

- **グリーン調達**：環境への負荷が小さい製品やサービスを優先して購入すること。
- **CSR調達**：環境だけでなく、人権・労働・法令遵守などにも配慮した取引先から調達すること。
- **オフショア開発**：開発の一部を、人件費の安い海外の企業に委託すること。**ニアショア** は国内の地方に委託すること。

### この章のまとめ

- 情報システム戦略は経営戦略に基づく。全体最適化の責任者は CIO。
- EA の4体系：ビジネス・データ・アプリケーション・テクノロジ。As-Is と To-Be を描く。
- BPR は抜本的な再設計、BPM は継続的改善、BPO は業務の外部委託。
- ERP は基幹業務の統合、CRM は顧客関係、SCM は供給連鎖全体、RPA は定型操作の自動化。
- RFI は情報提供依頼、RFP は提案依頼（同条件で公平に）。評価基準は事前に決める。

# 第15章 経営戦略・技術戦略・ビジネスインダストリ

**経営戦略** とは、企業が競争に勝ち、目標を達成するための長期的な方針です。この章では、自社や市場を分析するフレームワーク（考え方の枠組み）、マーケティング、技術開発の戦略、そして IT を活用したビジネスの具体例を学びます。用語の意味を問う問題が中心で、午後の問2（経営戦略）では、これらのフレームワークを事例に当てはめる力が問われます。

**第15章① 経営戦略の主なフレームワーク**

「分析する」→「戦略を決める」→「実行・評価する」

| 環境を分析する | 戦略を決める | 実行・評価する |
|---|---|---|
| SWOT分析／3C分析／PEST分析／5フォース分析／バリューチェーン分析 | PPM／アンゾフの成長マトリクス／競争の基本戦略／コアコンピタンス／ブルーオーシャン戦略 | バランススコアカード／CSF・KGI・KPI／マーケティングミックス（4P） |

## 1. 経営環境の分析

### ■ SWOT 分析

自社の **内部環境** を強み（Strengths）・弱み（Weaknesses）、**外部環境** を機会（Opportunities）・脅威（Threats）に分けて整理する手法です。

| | プラスの要因 | マイナスの要因 |
|---|---|---|
| **内部環境**（自社でコントロールできる） | **S**：強み（技術力、ブランド） | **W**：弱み（資金不足、人材不足） |
| **外部環境**（自社ではコントロールできない） | **O**：機会（市場の拡大、規制緩和） | **T**：脅威（競合の参入、景気後退） |

**ポイント　内部か外部かで判断する**

「競合他社が新製品を出した」→ 外部の脅威。「自社の営業力が高い」→ 内部の強み。迷ったら「自社の努力で変えられるか」で判断しましょう。SWOT を組み合わせて「強みを活かして機会をつかむ」などの戦略を考える手法を **クロスSWOT分析** といいます。

### ■ その他の分析手法

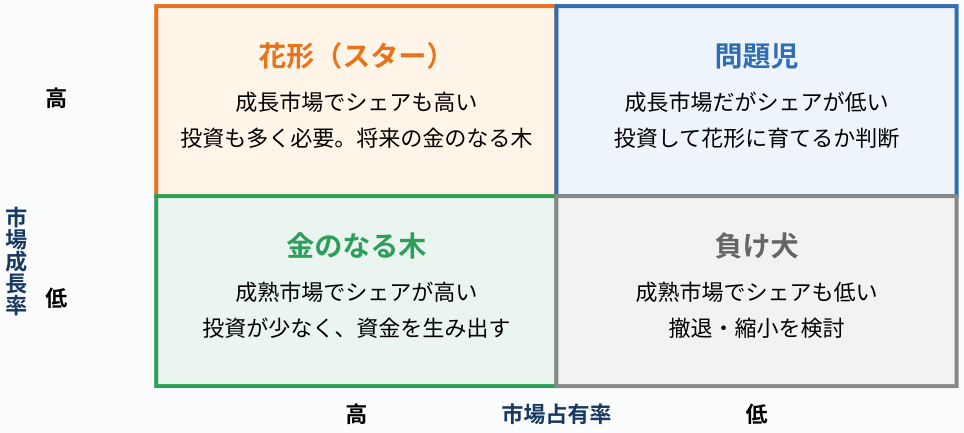
| 手法 | 内容 |
|---|---|
| **3C分析** | 顧客（Customer）・競合（Competitor）・自社（Company）の3つの視点で分析する |
| **PEST分析** | 政治（Politics）・経済（Economy）・社会（Society）・技術（Technology）のマクロ環境を分析する |
| **5フォース分析**（ファイブフォース） | 業界の収益性を決める5つの競争要因：①業界内の競合、②新規参入の脅威、③代替品の脅威、④買い手の交渉力、⑤売り手（供給業者）の交渉力 |
| **バリューチェーン分析** | 企業活動を、主活動（購買物流・製造・出荷物流・販売・サービス）と支援活動（人事・技術開発など）に分け、どこで価値（利益）が生まれているかを分析する |
| **VRIO分析** | 経営資源を、価値（Value）・希少性（Rarity）・模倣困難性（Imitability）・組織（Organization）で評価する |

## 2. 戦略の立案

### ■ PPM（プロダクトポートフォリオマネジメント）

**PPM** は、自社の事業や製品を、**市場成長率** と **市場占有率（シェア）** の2つの軸で4つに分類し、経営資源（資金）の配分を決める手法です。

**第15章② PPM の4象限**




**資金の流れ** 「金のなる木」で稼いだ資金を、「問題児」や「花形」に投資するのが基本です。

### ■ 成長戦略と競争戦略

**アンゾフの成長マトリクス**：「製品（既存・新規）」と「市場（既存・新規）」の組合せで、成長の方向性を4つに分けます。

| | 既存製品 | 新製品 |
|---|---|---|
| **既存市場** | 市場浸透（今の客にもっと売る） | 新製品開発 |
| **新市場** | 新市場開拓（新しい客に売る） | **多角化** |

**ポーターの競争の基本戦略**：

- **コストリーダーシップ戦略**：業界で最も低いコストを実現して、価格競争で優位に立つ。
- **差別化戦略**：品質・機能・ブランドなどで他社と違う価値を提供する。
- **集中戦略**：特定の顧客層や地域などに絞って、経営資源を集中する。

**コトラーの競争地位別戦略**：

| 地位 | 戦略 |
|---|---|
| **リーダー**（業界1位） | 市場全体の拡大、全方位戦略、同質化（他社の差別化を真似して打ち消す） |
| **チャレンジャー**（2位以下で1位を狙う） | リーダーとの **差別化** |
| **フォロワー** | リーダーの **模倣**。開発コストを抑える |
| **ニッチャー** | 大手が狙わない **隙間市場** に特化する |

### ■ その他の戦略用語

- **コアコンピタンス**：他社が真似できない、自社の **核となる強み**（技術やノウハウ）。
- **アウトソーシング**：自社の業務を外部に委託する。コアコンピタンスに集中するために使う。
- **アライアンス**：他社との提携。**M&A**：企業の合併・買収。**TOB**：株式の公開買付け。**MBO**：経営陣による自社の買収。
- **ブルーオーシャン戦略**：競争の激しい既存市場（レッドオーシャン）ではなく、競争のない **新しい市場** を創造する。
- **ベンチマーキング**：優良企業の優れたやり方（ベストプラクティス）と比較し、自社の改善に取り入れる。

## 3. 戦略の実行と評価

### ■ バランススコアカード（BSC）

**バランススコアカード** は、企業の業績を財務面だけでなく、**4つの視点** からバランスよく評価・管理する手法です。

**第15章③ バランススコアカードの4つの視点**

| 財務の視点 | 顧客の視点 | 業務プロセスの視点 | 学習と成長の視点 |
|---|---|---|---|
| 売上高、利益率、ROE（株主から見て） | 顧客満足度、市場シェア、リピート率 | 生産性、不良率、リードタイム | 従業員のスキル、資格取得数、提案件数 |

**関係**　学習と成長 → 業務プロセスの改善 → 顧客満足 → 財務成果、という因果関係を **戦略マップ** で表します。

### ■ CSF・KGI・KPI

- **KGI（重要目標達成指標）**：最終的な目標の達成度を測る指標（例：売上高を前年比10%増）。
- **CSF（重要成功要因）**：目標を達成するために、特に重要な要因（例：新規顧客の獲得）。
- **KPI（重要業績評価指標）**：CSF の達成度を測るための、途中経過の指標（例：月間の新規訪問件数）。

- **OKR**：野心的な目標（Objectives）と、その達成度を測る主要な成果（Key Results）を組み合わせた目標管理手法。

## 4. マーケティング

### ■ マーケティングの基本

| 用語 | 内容 |
|---|---|
| **STP** | **セグメンテーション**（市場を細分化）→ **ターゲティング**（狙う市場を決める）→ **ポジショニング**（競合と比べた自社の立ち位置を決める） |
| **4P（売り手の視点）** | **Product**（製品）、**Price**（価格）、**Place**（流通）、**Promotion**（販売促進）。この組合せを **マーケティングミックス** という |
| **4C（買い手の視点）** | Customer Value（顧客価値）、Cost（顧客の負担）、Convenience（入手の容易さ）、Communication（対話） |
| **プロダクトライフサイクル** | 製品の一生を **導入期 → 成長期 → 成熟期 → 衰退期** に分ける。成長期は競合が増え、成熟期は差別化や価格競争が重要 |
| **AIDMA** | 消費者の購買行動：Attention（注意）→ Interest（関心）→ Desire（欲求）→ Memory（記憶）→ Action（行動）。ネット時代は **AISAS**（Search・Share が入る） |

### ■ 価格戦略・販売の手法

- **スキミングプライシング（上澄み吸収価格）**：新製品を **高い価格** で発売し、早く開発費を回収する。
- **ペネトレーションプライシング（市場浸透価格）**：**低い価格** で発売し、一気にシェアを獲得する。
- **ダイナミックプライシング**：需要と供給の状況に応じて、価格を変動させる（航空券、ホテル）。
- **オムニチャネル**：店舗、通販サイト、SNS などのあらゆる販売チャネルを連携させ、顧客がどこからでも同じように購入できるようにする。
- **アフィリエイト**：自分のサイトに企業の広告を載せ、そこから商品が購入されると報酬を得る。
- **ロングテール**：ネット販売では、販売数の少ない多数のニッチ商品の売上合計が、大きな割合を占めるという現象。
- **インバウンドマーケティング**：有益な情報を発信して、顧客の方から見つけてもらう手法。
- **RFM分析**：顧客を **R**ecency（最終購買日）・**F**requency（購買頻度）・**M**onetary（購買金額）で分析し、優良顧客を見つける。

## 5. 技術戦略（MOT）

**MOT（技術経営）** とは、技術開発の成果を、事業の成功と利益に結び付けるための経営の考え方です。

- **技術ロードマップ**：将来の技術の進化や市場の変化を予測し、どの技術をいつまでに開発するかを、時間軸に沿って示した図。
- **魔の川・死の谷・ダーウィンの海**：研究開発が事業化するまでの3つの障壁。基礎研究→製品開発（魔の川）、製品開発→事業化（死の谷：資金不足など）、事業化→産業化（ダーウィンの海：市場での競争）。

- **イノベーションのジレンマ**：優良企業が、既存の顧客の要望に応えて製品を改良し続けた結果、新しい技術（破壊的イノベーション）を軽視して、市場を奪われてしまう現象。
- **オープンイノベーション**：自社だけでなく、大学や他社などの外部の技術やアイデアを積極的に取り入れて、新しい価値を生み出す。
- **ハイプサイクル**：新しい技術への期待が「黎明期 → 過度な期待のピーク期 → 幻滅期 → 啓蒙活動期 → 生産性の安定期」と推移することを示した図（ガートナー社）。
- **キャズム**：新しい製品が、初期の利用者（アーリーアダプター）から多数派（アーリーマジョリティ）に普及する間にある、大きな溝。
- **デザイン思考**：利用者の視点に立ち、「共感 → 問題定義 → 創造 → 試作 → テスト」を繰り返して、課題解決の方法を見つける。
- **PoC（概念実証）**：新しいアイデアや技術が実現可能か、効果があるかを、小規模に試して検証すること。

## 6. ビジネスインダストリ（ITの活用分野）

### ■ e-ビジネス

| 用語 | 内容 |
|---|---|
| **EC（電子商取引）** | **BtoB**（企業間）、**BtoC**（企業と消費者）、**CtoC**（消費者間：フリマアプリ）、**GtoC**（行政と市民） |
| **EDI（電子データ交換）** | 企業間で、受発注などの取引データを、標準化された形式で電子的に交換する |
| **O2O**（Online to Offline） | ネット上でクーポンを配るなどして、実店舗への来店を促す |
| **シェアリングエコノミー** | 個人が持つ車や部屋などを、インターネットを介して他者と共有・貸し借りする |
| **サブスクリプション** | 製品やサービスを所有せず、一定期間の利用権に対して料金を払う |
| **クラウドファンディング** | インターネットを通じて、不特定多数の人から資金を集める |
| **フィンテック（FinTech）** | 金融（Finance）と技術（Technology）を融合した新しいサービス（スマホ決済、ロボアドバイザ） |
| **暗号資産** | ブロックチェーン技術を使った、インターネット上の財産的価値 |

### ■ 産業での IT 活用

- **IoT**：あらゆるモノがインターネットにつながり、データを収集・活用する。
- **インダストリー4.0**：IoT・AI で工場の生産をスマート化する、ドイツ発の構想。**スマートファクトリー**。
- **CAD / CAM**：コンピュータを使った設計 / 製造。
- **MRP（資材所要量計画）**：生産計画と部品表（BOM）から、必要な部品の量と時期を計算する。
- **JIT（ジャストインタイム）**：必要なものを、必要なときに、必要な量だけ生産する。トヨタの **かんばん方式** が代表。
- **POS システム**：販売時点で、商品の販売情報を収集・管理する。売れ筋分析に使う。
- **GPS・GIS**：位置情報の取得 / 地図と情報を重ねて表示・分析する。
- **MaaS**：電車・バス・タクシーなどの複数の交通手段を、1つのサービスとして検索・予約・決済できるようにする。

- **スマートグリッド**：IT を活用して、電力の需要と供給を効率的に制御する次世代の送電網。
- **Society 5.0**：サイバー空間と現実空間を高度に融合させたシステムにより、経済発展と社会的課題の解決を両立する、人間中心の社会（日本政府が提唱）。

## ■ 組込みシステムと IoT の構成

- **組込みシステム**：家電、自動車、産業機器などに組み込まれ、特定の機能を実現するコンピュータシステム。リアルタイム性・省電力・小型化が求められる（第3章、第5章）。
- **IoT のシステム構成**：デバイス（センサー）→ ゲートウェイ（エッジ）→ ネットワーク → クラウド（データ分析）。

### この章のまとめ

- SWOT：内部（強み・弱み）と外部（機会・脅威）。5フォースは業界の5つの競争要因。
- PPM：花形・金のなる木・問題児・負け犬。金のなる木の資金を問題児・花形へ。
- アンゾフ：新製品×新市場＝多角化。ポーター：コストリーダーシップ・差別化・集中。
- BSC：財務・顧客・業務プロセス・学習と成長。KGI（最終目標）、CSF（成功要因）、KPI（途中指標）。
- STP → 4P。スキミングは高価格、ペネトレーションは低価格。
- イノベーションのジレンマ、死の谷、キャズム、ハイプサイクル、PoC。

# 第16章 企業活動と法務

この章では、企業の基本的な仕組み（組織・会計）と、業務の問題を数学的に解く手法（OR・IE）、そしてITに関わる法律を学びます。午前の最後の方（問75〜問80前後）で毎回出題されます。会計は計算パターンが決まっているので得点源にしやすく、法律は「何が違反になるか」の具体例で覚えるのがコツです。

### 第16章① この章の全体像

**企業と組織**
経営理念、組織形態、CSR、ステークホルダー

**OR・IE**
線形計画法、在庫管理、需要予測、QC七つ道具、意思決定

**会計・財務**
損益分岐点、財務諸表、財務指標、減価償却、投資評価

**法務・標準化**
知的財産権、個人情報保護、不正アクセス禁止法、労働・取引の法律、JIS/ISO

## 1. 企業と組織

### ■ 企業活動の基本

- **経営理念（ミッション）**：企業の存在意義や価値観。**ビジョン**：将来のありたい姿。
- **CSR（企業の社会的責任）**：利益の追求だけでなく、環境保護・法令遵守・社会貢献など、社会に対して責任ある行動をとること。
- **SDGs**：国連が定めた「持続可能な開発目標」。17の目標がある。
- **ESG投資**：環境（Environment）・社会（Social）・ガバナンス（Governance）への取組みを重視して投資先を選ぶこと。
- **ステークホルダー**：株主、顧客、従業員、取引先、地域社会など、企業の利害関係者。

### ■ 組織形態

| 組織 | 特徴 |
|---|---|
| **職能別（機能別）組織** | 営業部・製造部・経理部など、**仕事の機能** ごとに分ける。専門性が高まる |
| **事業部制組織** | 製品や地域ごとに事業部を作り、各事業部が **利益責任** を持って独立して運営する |
| **マトリックス組織** | 職能と事業（プロジェクト）の **2つの指揮系統** を持つ。命令系統が複雑になりやすい |
| **カンパニー制** | 事業部の独立性をさらに高め、社内の独立した会社のように扱う |
| **持株会社（ホールディングス）** | 他の会社の株式を保有し、グループ全体を支配・管理することを主な目的とする会社 |
| **ネットワーク組織・ホラクラシー** | 階層を減らし、役割に基づいて自律的に動く組織 |

### ■ 人事・人材の用語

- **OJT**：職場で実際の仕事を通じて教育する。**Off-JT**：職場を離れて研修などで教育する。
- **リスキリング**：新しい業務や職種に対応するために、必要なスキルを学び直すこと。

- **HRテック**：AI などを活用した人事業務の効率化。**タレントマネジメント**：従業員のスキルや経験を一元管理し、最適な配置や育成に活かす。
- **ダイバーシティ**：性別・年齢・国籍などの多様性を受け入れ、活かすこと。
- **ワークエンゲージメント**：仕事に対する熱意・没頭・活力に満ちた心の状態。

## 2. OR・IE（業務分析と意思決定の手法）

**OR（オペレーションズリサーチ）** は、数学的なモデルを使って、最適な意思決定を支援する手法です。**IE（インダストリアルエンジニアリング）** は、作業の方法や時間を分析して、生産性を高める手法です。

### ■ 線形計画法

**線形計画法** は、いくつかの制約条件（材料の量、作業時間など）のもとで、利益を最大にする（またはコストを最小にする）生産量などを求める手法です。

> **計算例　線形計画法**
>
> 製品 X は1個あたり利益3万円で、材料 A を 1kg、材料 B を 2kg 使う。製品 Y は1個あたり利益2万円で、材料 A を 1kg、材料 B を 1kg 使う。材料 A は 40kg、材料 B は 60kg まで使える。利益が最大になる生産量は？
> **①制約条件を式にする**：X ＋ Y ≦ 40（材料A）、2X ＋ Y ≦ 60（材料B）、X, Y ≧ 0
> **②2つの制約の交点を求める**：X ＋ Y ＝ 40 と 2X ＋ Y ＝ 60 を連立 → X ＝ 20、Y ＝ 20
> **③頂点で利益を比べる**：(X,Y) ＝ (0,40) → 80万円、(20,20) → 3×20＋2×20 ＝ **100万円**、(30,0) → 90万円
> 最大利益は X ＝ 20個、Y ＝ 20個 のとき **100万円**。最適解は、制約条件が作る領域の **頂点** のどれかになります。

### ■ 在庫管理と発注方式

| 方式 | 内容 |
|---|---|
| **定量発注方式** | 在庫が決められた量（**発注点**）まで減ったら、**決まった量** を発注する。発注間隔は変わる |
| **定期発注方式** | **決まった間隔**（毎月1日など）で、そのときに必要な量を計算して発注する。重要な商品向け |
| **ABC分析** | 売上や在庫金額の多い順に A・B・C の3グループに分け、重要度に応じて管理方法を変える（**パレート図** を使う） |

> **計算例　発注点**
>
> 1日の平均使用量が 20個、発注してから届くまで（**調達期間**）が 5日、安全在庫が 30個 のとき、発注点は？
> 発注点 ＝ 1日の使用量 × 調達期間 ＋ 安全在庫 ＝ 20 × 5 ＋ 30 ＝ **130個**

- **経済的発注量（EOQ）**：発注費用（発注1回ごとにかかる費用）と在庫維持費用（在庫を持つための費用）の合計が最小になる1回あたりの発注量。

### ■ 需要予測と意思決定

- **移動平均法**：直近の一定期間の平均値で予測する。
- **指数平滑法**：前回の予測値 ＋ α ×（前回の実績値 − 前回の予測値）。α（平滑化係数）は0〜1。

- **デシジョンツリー**：選択肢と、起こりうる結果とその確率を木の形で表し、**期待値** が最大の選択肢を選ぶ。
- **マクシミン原理**：各案の **最悪の結果** を比べ、その中で最も良い案を選ぶ（悲観的な判断）。**マクシマックス原理**：各案の最良の結果を比べ、最も良い案を選ぶ（楽観的な判断）。**ミニマックスリグレット原理**：機会損失（後悔）の最大値が最小になる案を選ぶ。

> **計算例　期待値による意思決定**
>
> 新製品の販売で、好況（確率0.6）なら利益500万円、不況（確率0.4）なら損失200万円。期待値は？
> $500 \times 0.6 + (-200) \times 0.4 = 300 - 80 =$ **220万円**

## ■ QC七つ道具と新QC七つ道具

### 第16章② QC七つ道具（主に数値データを分析する）

| 手法 | 内容・用途 |
|---|---|
| **パレート図** | 項目を件数の多い順に並べた棒グラフと、その **累積比率** の折れ線グラフを組み合わせる。**重点的に対策すべき項目** を見つける（ABC分析） |
| **特性要因図**（フィッシュボーン） | 結果（特性）と、それに影響する **原因（要因）** の関係を、魚の骨の形で体系的に整理する |
| **管理図** | 品質のデータを時系列に描き、**上方管理限界・下方管理限界** を設けて、工程が安定しているか（異常がないか）を判断する |
| **散布図** | 2つの変数を縦軸・横軸にとって点を打ち、**相関関係** を見る |
| **ヒストグラム** | データの範囲をいくつかの区間に分け、各区間に入るデータの数を棒グラフにして、**ばらつき（分布）** を見る |
| **チェックシート** | データを漏れなく集計するための記録用紙 |
| **層別** | データを、機械別・担当者別などのグループに分けて分析する |

**新QC七つ道具**（主に言葉のデータを整理する）：**親和図法**（似た意見をグループにまとめる）、**連関図法**（原因と結果が複雑に絡んだ関係を矢印で整理する）、**系統図法**（目的と手段を多段階に展開する）、**マトリックス図法**、**アローダイアグラム法**、**PDPC法**（事前に起こりうる事態を予測して計画を立てる）、**マトリックスデータ解析法**。

> **用語メモ　ブレーンストーミングの4原則**
>
> 多くのアイデアを出すための会議手法です。①**批判禁止**、②**自由奔放**（突飛な意見も歓迎）、③**質より量**、④**結合・便乗**（他人のアイデアに乗っかる）。

# 3. 会計・財務

## ■ 損益分岐点

**損益分岐点** とは、売上高と費用がちょうど等しくなり、**利益がゼロ** になる売上高のことです。これを超えれば黒字、下回れば赤字です。

- **固定費**：売上に関係なく一定にかかる費用（家賃、正社員の給料、減価償却費）。
- **変動費**：売上に比例して増える費用（材料費、外注費）。
- **変動費率** ＝ 変動費 ÷ 売上高。

### 第16章③ 損益分岐点の公式

**損益分岐点売上高**

**固定費 ÷（1 − 変動費率）**

「1 − 変動費率」は売上1円あたりの儲け（限界利益率）。これで固定費を回収するには、いくら売ればよいか

**目標利益を達成する売上高**

**（固定費 ＋ 目標利益）÷（1 − 変動費率）**

固定費に目標利益を上乗せして考える

**計算例　損益分岐点**

売上高 1,000万円、変動費 600万円、固定費 300万円の会社。

変動費率 ＝ 600 ÷ 1,000 ＝ 0.6

損益分岐点売上高 ＝ 300 ÷ (1 − 0.6) ＝ 300 ÷ 0.4 ＝ **750万円**

利益 200万円を出すための売上高 ＝ (300 ＋ 200) ÷ 0.4 ＝ **1,250万円**

## ■ 財務諸表

| 財務諸表 | 内容 |
|---|---|
| **貸借対照表（B/S）** | 決算日時点の **財政状態**。左側（借方）に **資産**、右側（貸方）に **負債** と **純資産**。資産 ＝ 負債 ＋ 純資産 |
| **損益計算書（P/L）** | 一定期間の **経営成績**（いくら儲かったか）。収益 − 費用 ＝ 利益 |
| **キャッシュフロー計算書** | 一定期間の **現金の流れ**。営業活動・投資活動・財務活動の3つに分ける |
| **株主資本等変動計算書** | 純資産の変動を示す |

**損益計算書の利益の段階：**

### 損益計算書の5つの利益

**売上総利益**
売上高 − 売上原価（粗利）
▶
**営業利益**
− 販売費及び一般管理費（本業の儲け）
▶
**経常利益**
＋ 営業外収益 − 営業外費用（利息など）
▶
**税引前当期純利益**
＋ 特別利益 − 特別損失（臨時のもの）
▶
**当期純利益**
− 法人税等

### ■ 財務指標

| 指標 | 式 | 意味 |
|---|---|---|
| **ROE**（自己資本利益率） | 当期純利益 ÷ 自己資本 | 株主のお金でどれだけ稼いだか |
| **ROA**（総資産利益率） | 利益 ÷ 総資産 | すべての資産でどれだけ稼いだか |
| **流動比率** | 流動資産 ÷ 流動負債 | 短期的な支払能力。高いほど安全 |
| **自己資本比率** | 自己資本 ÷ 総資本 | 返済不要な資本の割合。高いほど安全 |
| **総資本回転率** | 売上高 ÷ 総資本 | 資本を効率よく使っているか |

### ■ 減価償却

パソコンや建物などの固定資産は、使っていくうちに価値が下がります。その **取得にかかった費用を、使用できる期間（耐用年数）にわたって少しずつ費用として配分** することを **減価償却** といいます。

- **定額法**：毎年 **同じ金額** を償却する。償却額 ＝ 取得価額 ÷ 耐用年数。
- **定率法**：毎年、未償却残高に **一定の率** を掛けて償却する。初めの年ほど償却額が大きい。

> **計算例　定額法の減価償却**
> 取得価額 300万円のサーバ、耐用年数5年、定額法（残存価額0）。
> 毎年の償却額 ＝ 300 ÷ 5 ＝ 60万円。3年経過後の帳簿価額 ＝ 300 − 60 × 3 ＝ **120万円**

- **ソフトウェアの会計処理**：自社利用のソフトウェアで将来の収益獲得や費用削減が確実なものは **無形固定資産** として計上し、原則5年以内で償却する。

### ■ 投資の評価

- **回収期間法**：投資額を何年で回収できるかで評価する。短いほど良い。
- **NPV（正味現在価値）**：将来得られるお金を、**現在の価値に割り引いて** 合計し、投資額を引いたもの。NPV がプラスなら投資する価値がある。
- **割引の考え方**：金利が5%なら、今の100万円は1年後に105万円になる。逆に言えば、1年後の105万円の現在価値は100万円。1年後の金額 ÷ 1.05 で現在価値に直す。
- **IRR（内部収益率）**：NPV がちょうど0になる割引率。IRR が資本コストより高ければ投資する価値がある。

## 4. 知的財産権

**知的財産権** とは、人間の知的な創作活動から生まれたものを保護する権利です。

## 第16章④ 知的財産権の種類

| 権利 | 保護するもの | 手続 | 保護期間 |
|---|---|---|---|
| **著作権** | 思想や感情を創作的に表現したもの（文章、音楽、**プログラム**） | **不要**（創作した時点で発生） | 原則、著作者の死後70年 |
| **特許権** | 自然法則を利用した高度な技術的アイデア（**発明**） | 特許庁に出願・登録 | 出願から20年 |
| **実用新案権** | 物品の形状などの考案（小発明） | 出願・登録 | 出願から10年 |
| **意匠権** | 物品・建築物・画像の **デザイン** | 出願・登録 | 出願から25年 |
| **商標権** | 商品やサービスに付ける **マーク・名称** | 出願・登録 | 登録から10年（更新可） |

### ■ 著作権のポイント（IT関連）

- **プログラムは著作物として保護される**。ただし、**プログラム言語、規約（プロトコル）、アルゴリズム（解法）は保護されない**。アルゴリズムは特許の対象になりうる。
- **職務著作**：会社の発意に基づき、従業員が職務上作成し、会社名義で公表するものは、契約などで別の定めがない限り **会社（法人）が著作者** になる。
- **請負契約での著作権**：ソフトウェア開発を外部に請け負わせた場合、契約で定めがなければ、著作権は **実際に作成した受注者側** に帰属する。発注者が著作権を得たい場合は、契約で **著作権の譲渡** を定める。
- **著作者人格権**（公表権・氏名表示権・同一性保持権）は譲渡できない。

### ■ 営業秘密（不正競争防止法）

**不正競争防止法** は、営業秘密の不正な取得・使用や、他社の商品表示を真似るなどの不正な競争を防ぐ法律です。**営業秘密** として保護されるには、次の3要件をすべて満たす必要があります。

- ①**秘密管理性**：秘密として管理されている（アクセス制限、「社外秘」の表示）
- ②**有用性**：事業に役立つ情報である（顧客名簿、製造ノウハウ）
- ③**非公知性**：一般に知られていない

また、他人の商標などと同じ・類似のドメイン名を不正な目的で取得する行為（**ドメイン名の不正取得**）も、この法律で禁止されています。

### ■ ライセンス

- **ソフトウェアライセンス**：ソフトウェアの使用許諾。**サイトライセンス**（特定の場所での利用を一括許諾）、**ボリュームライセンス**（大量購入による一括契約）、**サブスクリプション**（期間ごとの利用権）。
- **クリエイティブ・コモンズ**：著作者が、作品の利用条件（表示、非営利、改変禁止など）を自ら示すための仕組み。

# 5. セキュリティ関連法規

| 法律・基準 | 内容 |
|---|---|
| **サイバーセキュリティ基本法** | 国や自治体、事業者などの責務を定めた、サイバーセキュリティ政策の基本となる法律 |
| **不正アクセス禁止法** | ①他人のIDとパスワードを無断で使ってログインする、②セキュリティホールを突いて侵入する、ことを禁止。**他人のIDやパスワードを、正当な理由なく第三者に提供する行為** や、**不正に取得・保管する行為**、**フィッシングサイトの作成** も処罰される |
| **刑法（不正指令電磁的記録に関する罪）** | 正当な理由なく、**コンピュータウイルスを作成・提供・保管** することを処罰する（いわゆるウイルス作成罪） |
| **刑法（電子計算機使用詐欺罪・電子計算機損壊等業務妨害罪）** | コンピュータに虚偽の情報を与えて不正に利益を得る／データを破壊して業務を妨害する |
| **個人情報保護法** | 個人情報を取り扱う事業者の義務を定める（下記） |
| **プロバイダ責任制限法（情報流通プラットフォーム対処法）** | インターネット上で権利侵害があったとき、プロバイダの損害賠償責任の範囲や、**発信者情報の開示請求** について定める |
| **特定電子メール法** | 広告宣伝メールは、原則として受信者の **事前の同意（オプトイン）** が必要 |

## ■ 個人情報保護法

- **個人情報**：生存する個人に関する情報で、**特定の個人を識別できる** もの（氏名、顔写真、メールアドレスで個人が特定できるもの）。**個人識別符号**（マイナンバー、指紋データ、パスポート番号など）を含むものも個人情報。
- **要配慮個人情報**：人種、信条、病歴、犯罪の経歴など、不当な差別につながりうる情報。取得には **原則として本人の同意** が必要。
- **利用目的の特定と通知・公表**：利用目的をできる限り特定し、本人に通知または公表する。目的外の利用は原則禁止。
- **第三者提供の制限**：個人データを第三者に提供するには、**原則として本人の同意** が必要。ただし、業務の委託先への提供、合併による承継、共同利用は「第三者」にあたらない。
- **オプトアウト**：本人の求めがあれば提供を停止することを条件に、同意なく第三者提供すること（事前に個人情報保護委員会への届出が必要。要配慮個人情報は不可）。
- **匿名加工情報**：特定の個人を識別できないように加工し、**元に戻せないようにした** 情報。本人の同意なく第三者に提供できる。**仮名加工情報**：他の情報と照合しない限り個人を識別できないように加工した情報（主に社内での分析用）。
- **漏えい時の報告**：要配慮個人情報の漏えいなど、一定の重大な漏えいが発生した場合は、**個人情報保護委員会への報告と本人への通知** が義務。
- **マイナンバー法**：マイナンバー（個人番号）の利用範囲は、社会保障・税・災害対策の手続に **限定** されている。

# 6. 労働・取引関連の法規

## ■ 請負・準委任・派遣の違い（超頻出）

### 第16章⑤ 契約形態の違い ― 誰が作業者に指揮命令するか

| | 請負契約 | 準委任契約 | 労働者派遣 |
|---|---|---|---|
| 目的 | **仕事の完成**（成果物の納品） | **事務処理の遂行**（作業すること自体） | 労働力の提供 |
| 完成責任 | **あり**（成果物に不適合があれば契約不適合責任） | なし（善管注意義務はある） | なし |
| 作業者への指揮命令 | **受注者** が行う | **受注者** が行う | **派遣先（発注者）** が行う |
| 雇用関係 | 受注者と作業者 | 受注者と作業者 | **派遣元** と作業者 |

**ひっかけ注意　偽装請負**

請負契約や準委任契約なのに、**発注者が受注者の作業者に直接指揮命令する** のは「偽装請負」として違法です。指揮命令したいなら、労働者派遣契約を結ぶ必要があります。また、派遣先が派遣労働者を、さらに別の会社に派遣すること（**二重派遣**）も禁止です。

## ■ その他の労働・取引の法規

- **労働基準法**：労働時間、休日、賃金などの最低基準を定める。法定労働時間を超えて働かせるには、労使協定（**36協定**）の締結と届出が必要。
- **フレックスタイム制**：一定期間の総労働時間を決めたうえで、始業・終業の時刻を労働者が自由に決められる制度。**裁量労働制**：実際の労働時間に関係なく、あらかじめ決めた時間働いたものとみなす制度。
- **労働者派遣法**：派遣の期間（同一の組織単位で原則3年まで）などを定める。
- **下請法（下請代金支払遅延等防止法）**：親事業者が下請事業者に対して、**代金の支払遅延**（受領日から **60日以内** に支払わなければならない）、不当な減額、受領拒否、買いたたきなどをすることを禁止する。発注時には、内容を記載した書面の交付が必要。（※2026年からは「中小受託取引適正化法（取適法）」に改正・改称）
- **公益通報者保護法**：組織の不正を通報した労働者を、解雇などの不利益な扱いから守る。
- **PL法（製造物責任法）**：製品の欠陥によって損害が生じた場合、製造業者は **過失がなくても** 賠償責任を負う。ソフトウェア単体は「製造物」にあたらないが、ソフトウェアを組み込んだ製品は対象。

## ■ その他の法規・ガイドライン

- **電子帳簿保存法**：税務関係の帳簿や書類を、電子データで保存することを認める法律。電子取引のデータは電子のまま保存することが必要。
- **e-文書法**：法律で保存が義務付けられた文書を、電子データで保存することを認める。

- **電子署名法**：本人による一定の電子署名がある電子文書は、本人の意思で作成されたものと推定される（手書きの署名や押印と同等の効力）。
- **独占禁止法**：私的独占、不当な取引制限（カルテル）、不公正な取引方法を禁止して、公正な競争を守る。
- **AI事業者ガイドライン**：AIの開発・提供・利用に関わる事業者が取り組むべき事項（人間中心、安全性、公平性、透明性など）を示した、日本政府のガイドライン。

## 7. 標準化

- **標準化**：製品の規格や仕様を統一すること。互換性が確保され、コストが下がる。
- **デジュレスタンダード**：公的な標準化機関が定めた標準（ISO、JIS）。**デファクトスタンダード**：市場での競争の結果、事実上の標準になったもの。
- **標準化団体**：**ISO**（国際標準化機構）、**IEC**（国際電気標準会議）、**IEEE**（米国電気電子学会：LANの規格など）、**W3C**（Web技術の標準）、**IETF**（インターネット技術：RFCを発行）、**JIS**（日本産業規格）。
- **主なマネジメントシステム規格**：ISO 9001（品質）、ISO 14001（環境）、ISO/IEC 27001（情報セキュリティ）、ISO/IEC 20000（ITサービス）、ISO 22301（事業継続）。
- **コード体系**：**JANコード**（商品の識別番号のバーコード）、**QRコード**（2次元コード）、**ISBN**（書籍）。

### この章のまとめ

- 線形計画法の最適解は領域の頂点。発注点 ＝ 1日使用量 × 調達期間 ＋ 安全在庫。
- パレート図は重点項目、特性要因図は原因と結果、管理図は工程の異常、散布図は相関。
- 損益分岐点売上高 ＝ 固定費 ÷ (1 − 変動費率)。営業利益は本業、経常利益は営業外を含む。
- 定額法は毎年同額。NPV は将来のお金を現在価値に割り引いて評価。
- プログラムは著作物だが、言語・規約・アルゴリズムは保護されない。請負の著作権は契約がなければ受注者。
- 営業秘密は秘密管理性・有用性・非公知性。ID・パスワードの無断提供も不正アクセス禁止法違反。
- 請負・準委任の指揮命令は受注者、派遣は派遣先。発注者が直接指示すると偽装請負。

# 第17章 午後試験の全体像と解き方

午後試験は、1問あたり数ページの長文の事例（ある会社の状況説明）を読み、設問に **記号選択・穴埋め・数値計算・記述（20〜40字程度）** で答える形式です。「知識を覚えているか」よりも、「**事例の中で、知識をどう使うか**」が問われます。この章では、全分野に共通する解き方と、分野ごとの攻略の型をまとめます。具体的な演習は、今後作成する「午後対策シート」で行います。

### 第17章① 午後試験の構成と時間配分

150分で5問 ＝ 1問あたり約30分

**問1 情報セキュリティ（必須）**
全員が解く。第9章の知識が直結。毎年必ず出るので、最優先で対策する

**問2〜問11（10問中4問を選択）**
技術系（プログラミング・アーキテクチャ・ネットワーク・DB・組込み・開発）と、マネジメント・ストラテジ系（経営戦略・PM・サービス・監査）

**時間配分の目安**
1問 25〜28分 × 5問 ＋ 見直し10分。1問に30分以上かけない。解けない設問は飛ばして後で戻る

**最大の注意**　選択した問題の番号を、解答用紙の選択欄で必ず○で囲む（○がない問題は採点されません。5問以上囲むと、はじめの4問だけが採点されます）。CBT方式の場合も、選択の操作手順を事前に確認しておきましょう。

## 1. 記述式問題の基本ルール

### ■ 答えは「問題文の中」にある

午後問題の最大のコツは、**答えの根拠は、ほぼ必ず問題文の中にある** ということです。自分の経験や一般論で答えるのではなく、「問題文のどこに書いてある事実を使えば、この設問に答えられるか」を探します。

**解き方の型　記述問題を解く5ステップ**

①**設問を先に読む**：何が問われているか（理由？対策？名称？）、字数、条件（「〜の観点から」など）を確認する。
②**本文を読む**：設問に関係しそうな箇所（下線部、〔〕の見出し、表や図）に印を付けながら読む。
③**根拠を探す**：下線部の前後、設問で指定された節を中心に、答えの根拠となる記述を見つける。
④**本文の言葉で書く**：根拠となる記述の **キーワードをそのまま使って** 答えを組み立てる。
⑤**問いに正対する**：「理由」を問われたら「〜ため」、「目的」なら「〜するため」、「問題点」なら「〜という問題がある」と、問いの形に合わせて文末を整える。

### ■ 記述のコツ

- **字数の8割以上は書く**：30字以内なら24〜30字が目安。短すぎると要素が足りないことが多い。
- **1つの解答に1つの論点**：「AとBとCのため」のように欲張らず、問われている核心を書く。
- **主語と目的語をはっきり**：「漏えいするため」ではなく「**顧客情報が** 漏えいするため」。
- **問題文にない用語や数値を勝手に作らない**。

- **わからなくても空欄にしない**：部分点がもらえることがある。

> **ポイント　「なぜ？」に強くなる練習**
> 午後の記述は「なぜその対策が必要か」「なぜその方式では問題があるのか」を問うものが中心です。午前の学習のときから、用語を覚えるだけでなく「**それは何を防ぐ（実現する）ためのものか**」を一言で説明する練習をしておくと、午後で大きな力になります。

## 2. 選択問題の選び方

### ■ 選択の考え方

- 本番の前に、**得意分野を5〜6分野** 決めておき、それを重点的に練習する。本番では問題を見て、その中から解きやすそうな4問を選ぶ（その年の難易度によって入れ替えられるよう、少し多めに準備）。
- 選ぶ基準は「その分野の午前知識がしっかりあるか」と「過去問の得点が安定しているか」。

**第17章② 分野ごとの特徴と向いている人**

| 分野 | 特徴 | 向いている人 |
|---|---|---|
| 問2 経営戦略 | 財務計算やフレームワーク（SWOT、BSC）を事例に当てはめる。専門知識が少なくても読解で解ける設問が多い | 文章読解が得意、ビジネスに関心がある |
| 問3 プログラミング | 擬似言語のアルゴリズムの空欄補充・トレース。得点が安定しやすいが、時間がかかる | プログラミング経験がある、手を動かすのが好き |
| 問4 システムアーキテクチャ | 構成・性能・可用性の計算（稼働率、処理時間）と設計の考え方 | 計算に強い、インフラに関心がある |
| 問5 ネットワーク | IPアドレス、ルーティング、DNS、ファイアウォールの設定など。知識の有無で差がつく | ネットワークの実務経験・知識がある |
| 問6 データベース | E-R図、正規化、SQLの穴埋め。型が決まっていて対策しやすい | SQLを書いたことがある、論理的思考が得意 |
| 問7 組込みシステム開発 | センサー・タスク制御・状態遷移・タイミング計算 | 組込み・電子回路の経験がある |
| 問8 情報システム開発 | 設計（UML、クラス図）、テスト、開発手法。設問がやや抽象的 | 開発の実務経験がある |
| 問9 プロジェクトマネジメント | 日程・コスト計算と、リスク・品質・要員などへの対応策。常識的な判断で答えられる設問が多い | 文章読解が得意、チーム作業の経験がある |
| 問10 サービスマネジメント | インシデント・変更・可用性・SLAなど。運用の考え方を事例に当てはめる | 文章読解が得意、運用・サポートの経験がある |
| 問11 システム監査 | 監査人の視点で、確認すべき事項や監査証拠を答える。本文の記述を使って答える問題が多い | 文章読解が得意、几帳面 |

**用語メモ　知識が少ない人へのおすすめ**

IT の実務経験が少ない場合は、**問1（必須）＋ 問2 経営戦略・問9 PM・問10 サービスマネジメント・問11 システム監査** のように、読解力で得点しやすい「文章系」を中心に選び、予備として問6 データベースを準備するのが定番の戦略です。ただし、文章系は年度によって難易度の差が大きいので、技術系も1〜2分野は準備しておくと安心です。

## 3. 分野別の解き方の型

### ■ 問1 情報セキュリティ（必須）

**解き方の型　情報セキュリティの解き方**

**①攻撃の流れを把握する**：「何が・どこから・どうやって侵入し・何が起きたか」を図や時系列で整理する。
**②攻撃名と対策を結び付ける**：第9章の対応表（SQLインジェクション→プレースホルダ、など）を即答できるようにする。
**③構成図の機器と通信経路を確認する**：ファイアウォールのルール（送信元・宛先・ポート・許可/拒否）の穴埋めがよく出る。
**④「なぜ」を答える**：「この対策で何が防げるか」「この設定のままだと何が起きるか」を、本文の事実を使って書く。

**頻出テーマ**：マルウェア感染と初動対応、ログの分析、ファイアウォールのルール設定、認証方式（多要素認証、パスワードの安全な保存＝ソルト＋ハッシュ）、Webアプリの脆弱性、メールのなりすまし対策（SPF・DKIM・DMARC）、TLSと証明書、ゼロトラスト、クラウドの設定ミス。

### ■ 問2 経営戦略

**解き方の型　経営戦略の解き方**

**①会社の状況を分類する**：本文の事実を SWOT（内部の強み・弱み、外部の機会・脅威）に振り分けながら読む。
**②フレームワークの用語を正確に**：PPM の位置付け、BSC の視点、マーケティングの4P などを問われたら、用語の定義どおりに答える。
**③計算は落とさない**：損益分岐点、利益率、投資回収などの計算は、第16章の公式で確実に得点する。
**④施策の理由は「会社の課題」と結び付ける**：「売上が伸び悩んでいる」「若年層の顧客が少ない」など、本文に書かれた課題を解決する方向で答える。

### ■ 問3 プログラミング（アルゴリズム）

**解き方の型　プログラミングの解き方**

**①プログラムの目的と変数の意味をつかむ**：問題文の説明と、プログラム中の変数名・コメントを対応させる。
**②小さな例でトレースする**：問題文の例（配列の値など）を使って、第2章のように変数の変化を表に書く。空欄は「この行の後でこの変数はこうなっているはず」から逆算する。
**③境界条件に注意**：ループの開始値・終了値（1からか0からか、n か n−1 か）、「＜」か「≦」かで答えが変わる。
**④計算量・実行回数の設問**：ループの入れ子と回数から考える。

**頻出テーマ**：探索・整列、再帰、木構造・グラフ（最短経路）、文字列処理、2進数・ビット演算、動的計画法、スタック・キュー。

## ■ 問4 システムアーキテクチャ

**解き方の型　システムアーキテクチャの解き方**

**①構成図を読み解く**：サーバの台数・役割、冗長化の方式、データの流れを確認する。
**②計算問題**：稼働率（直列・並列）、処理能力（1秒あたりの処理件数）、必要台数、CPU使用率などを第3・4章の公式で解く。単位（ミリ秒・秒、件/秒）の換算に注意。
**③方式の比較**：スケールアップとスケールアウト、ホットスタンバイとコールドスタンバイ、クラウドの利用形態の長所・短所を、本文の要件（コスト・可用性・性能）に照らして答える。

## ■ 問5 ネットワーク

**解き方の型　ネットワークの解き方**

**①構成図に情報を書き込む**：各機器の IPアドレス、サブネット、VLAN、通信経路を図に書き込みながら読む。
**②パケットの流れを追う**：「PC → スイッチ → ルータ → ファイアウォール → サーバ」と、通信がどの機器を通り、どこで何が変換されるか（NAT など）を追う。
**③プロトコルの役割を答える**：DNS（名前解決）、DHCP（アドレス割当て）、ARP（MACアドレスの解決）などを、本文の場面に当てはめる。

**頻出テーマ**：サブネットの計算、ルーティングテーブル、DNS の設定、冗長化（VRRP、リンクアグリゲーション）、無線LAN、VPN、プロキシ、負荷分散。

## ■ 問6 データベース

**解き方の型　データベースの解き方**

**①E-R図と表の定義を対応させる**：エンティティ、主キー（実線の下線）、外部キー（破線の下線）、多重度（1対多など）を確認する。
**②E-R図の空欄**：「1つの〇〇に対して複数の△△があるか」を本文の業務ルールから読み取り、リレーションシップの線を補う。
**③SQL の穴埋め**：SELECT・FROM・WHERE・GROUP BY・HAVING の役割（第7章）を思い出し、「何を数えるのか（COUNT）」「どの単位でまとめるのか（GROUP BY）」「結合条件は何か」を考える。
**④正規化・更新時異状**：「どの関数従属が問題か」「どの表に分けるべきか」を答える。

## ■ 問7 組込みシステム開発

**解き方の型　組込みシステムの解き方**

**①ハードウェア構成と動作の流れをつかむ**：センサー → 制御部（CPU）→ アクチュエータの流れと、各タスクの役割を整理する。

**②状態遷移表・タイミングチャートを丁寧に読む**：「どのイベントで、どの状態からどの状態に移るか」を表で確認する。

**③計算問題**：割込みの周期、タイマのカウント値、バッテリーの持続時間、データ量などを、単位に注意して計算する（第3章の計算例を参照）。

**④タスク間の制御**：排他制御（セマフォ）、優先度、イベントフラグなど、第5章の知識を使う。

## ■ 問8 情報システム開発

**解き方の型　情報システム開発の解き方**

**①設計図（クラス図・シーケンス図・状態遷移図）を読み解く**：第10章の UML の意味を確認しながら、本文の業務と対応させる。

**②テストの設問**：テストケースの設計（同値分割・限界値分析）や、どのテストで確認すべきかを答える。

**③開発手法の設問**：アジャイル・スクラム・CI/CD の考え方と、その事例での利点・注意点を答える。

## ■ 問9 プロジェクトマネジメント

**解き方の型　プロジェクトマネジメントの解き方**

**①プロジェクトの状況と課題を整理する**：目標（納期・予算・品質）、制約、ステークホルダー、発生している問題を書き出す。

**②計算は確実に**：工数（人月）、要員数、EVM、クリティカルパスの計算は第11章の公式で解く。

**③対応策は「PMの立場」で**：プロジェクトマネージャとして、リスクを早期に見つけ、関係者と合意を取り、計画を見直す、という行動を本文の事実に即して書く。

**④「なぜその対応をとったのか」を問われたら**：本文中の懸念点（「〜の可能性がある」「〜が不慣れ」など）とセットで答える。

## ■ 問10 サービスマネジメント

**解き方の型　サービスマネジメントの解き方**

**①サービスの内容と SLA を確認する**：サービス時間、稼働率の目標、問合せの対応時間など。

**②プロセスを正しく区別する**：インシデント管理（迅速な回復）と問題管理（根本原因の解決）、変更管理とリリース管理（第12章）。

**③計算問題**：稼働率、許容停止時間、対応件数、要員数などを計算する。

**④改善策は「再発防止」と「影響の最小化」の観点で**：本文の障害の経緯から、どこを改善すべきかを読み取る。

### ■ 問11 システム監査

**解き方の型　システム監査の解き方**

**①監査の目的（監査要点）を確認する**：「何が適切かどうか」を確かめる監査なのかを把握する。
**②リスクを見つける**：本文の業務の流れの中で、「職務分掌ができていない」「承認の記録がない」「定期的な見直しがされていない」などの弱点（リスク）を探す。
**③監査手続・監査証拠を答える**：「〇〇の記録を確認して、△△が行われているかを確かめる」という形で、**確認する証拠（記録・文書）** と **確認する内容** をセットで書く。
**④監査人の立場を守る**：監査人は改善を「実施」しない。「確認する」「評価する」「指摘する」という表現で答える。

## 4. 過去問演習の進め方

1. **まず午前知識を固める**：本書の第1〜16章を読み、午前の過去問で6割以上取れるようにする。午後の土台は午前の知識です。
2. **時間を気にせず1問解く**：最初は分野ごとに、時間無制限で1問をじっくり解く。
3. **解答例と採点講評を読む**：IPA が公開している **解答例** と **採点講評** を読み、「どの本文の記述を根拠にしているか」を確認する。採点講評には、受験者が間違えやすかった点が書かれています。
4. **同じ分野を3〜5年分解く**：出題パターンや頻出テーマに慣れる。
5. **時間を計って解く**：本番を想定して、1問25〜30分で解く練習をする。

**過去問ではこう出る　令和7年度の午後問題**

令和7年度（2025年度）の春期・秋期も、問1 情報セキュリティ（必須）と、問2〜問11 の10分野から4問選択という構成でした。IPA のサイトでは、問題冊子・解答例・採点講評が公開されているので、最新年度から順に解いていくことをおすすめします。

**この章のまとめ**

- 午後は問1必須＋4問選択、1問25〜30分。選択欄の問題番号を○で囲むのを忘れない。
- 答えの根拠は問題文にある。本文の言葉を使い、問いの形（理由・目的・問題点）に合わせて書く。
- 得意分野を5〜6つ準備し、本番で4つを選ぶ。知識が少ないなら文章系＋DBが定番。
- 各分野の「解き方の型」を使って、過去問で解答例と採点講評を確認しながら練習する。

# 総まとめ — 試験直前チェック

試験直前に見返すための総まとめです。各章の「この章のまとめ」とあわせて、ここに載っている公式と用語の区別がすべて説明できれば、午前の基礎は十分です。

## 1. 必ず覚える公式・計算パターン

| 分野 | 公式・計算 | 章 |
|---|---|---|
| 基数 | nビットで $2^n$ 通り。$2^{10} \fallingdotseq 10^3$。2進⇔16進は4桁区切り | 1 |
| 補数 | 2の補数 ＝ ビット反転 ＋ 1。nビットの範囲 $-2^{n-1}$ ～ $2^{n-1}-1$ | 1 |
| 待ち行列 | ρ ＝ 到着率 × サービス時間。待ち時間 ＝ ρ/(1−ρ) × サービス時間 | 1 |
| 二分探索 | 最大比較回数 ≒ $\log_2 n$ ＋ 1。線形探索の平均は n/2 | 2 |
| CPU性能 | MIPS ＝ クロック周波数 ÷ CPI ÷ $10^6$ | 3 |
| パイプライン | 所要サイクル ＝ ステージ数 ＋ (命令数 − 1) | 3 |
| キャッシュ | 実効アクセス時間 ＝ キャッシュ × ヒット率 ＋ 主記憶 × (1 − ヒット率) | 3 |
| HDD | 平均回転待ち時間 ＝ 1回転の時間 ÷ 2 | 3 |
| 稼働率 | MTBF ÷ (MTBF ＋ MTTR)。直列 a×b、並列 1−(1−a)(1−b) | 4 |
| データ量 | 画像 ＝ 画素数 × 色深度。音声 ＝ 周波数 × ビット数 × チャネル × 秒 | 6 |
| サブネット | ホスト数 ＝ $2^{\text{ホスト部ビット数}} - 2$ | 8 |
| 伝送時間 | データ量（ビット）÷（回線速度 × 伝送効率） | 8 |
| 鍵の数 | 共通鍵 n(n−1)/2、公開鍵 2n | 9 |
| EVM | SV＝EV−PV、CV＝EV−AC、SPI＝EV/PV、CPI＝EV/AC、EAC＝BAC/CPI | 11 |
| 経路の数 | コミュニケーション経路 ＝ n(n−1)/2 | 11 |
| 発注点 | 1日の使用量 × 調達期間 ＋ 安全在庫 | 16 |
| 損益分岐点 | 固定費 ÷ (1 − 変動費率) | 16 |
| 減価償却 | 定額法：取得価額 ÷ 耐用年数 | 16 |

## 2. まぎらわしい用語の区別

| これと | これの違い |
|---|---|
| 桁落ち（近い数の引き算） | 情報落ち（大きさが大きく違う数の足し算） |
| パリティ（1ビット誤りの検出のみ） | ハミング符号（1ビット訂正・2ビット検出） |
| スタック（LIFO） | キュー（FIFO） |
| DRAM（リフレッシュ必要・主記憶） | SRAM（フリップフロップ・キャッシュ） |
| ライトスルー（両方に同時書込み） | ライトバック（追い出し時に書き戻し） |
| フェールセーフ（安全側に止める） | フェールソフト（縮退して動き続ける） |
| スケールアップ（1台の性能を上げる） | スケールアウト（台数を増やす） |
| ファーストフィット（最初に見つかった領域） | ベストフィット（最小の十分な領域） |
| 差分バックアップ（前回フル以降） | 増分バックアップ（前回バックアップ以降） |
| WHERE（行を絞る） | HAVING（グループを絞る） |
| ロールバック（更新前ログで取消し） | ロールフォワード（更新後ログでやり直し） |
| TCP（コネクション型・高信頼） | UDP（コネクションレス・高速） |
| L2スイッチ（MACアドレス） | ルータ（IPアドレス） |
| 暗号化（受信者の公開鍵で暗号化） | デジタル署名（送信者の秘密鍵で署名） |
| IDS（検知・通知） | IPS（検知・遮断） |
| 同値分割（グループの代表値） | 限界値分析（境界の値） |
| スタブ（下位モジュールの代役） | ドライバ（上位モジュールの代役） |
| クラッシング（資源追加・コスト増） | ファストトラッキング（並行作業・手戻りリスク） |
| インシデント管理（迅速な回復） | 問題管理（根本原因の究明） |
| RTO（いつまでに復旧するか） | RPO（どの時点のデータまで戻すか） |
| RFI（情報提供依頼） | RFP（提案依頼） |
| BPR（抜本的な再設計） | BPM（継続的な改善） |
| スキミング（高価格で発売） | ペネトレーション（低価格で発売） |
| 請負（完成責任あり・指揮命令は受注者） | 派遣（完成責任なし・指揮命令は派遣先） |
| 著作権（登録不要・プログラムを保護） | 特許権（出願が必要・発明を保護） |

## 3. 試験当日のチェックリスト

- 受験票・身分証明書・筆記用具（鉛筆・消しゴム）・時計を準備したか（CBTの場合は案内に従う）
- 午前：1問に悩みすぎない。わからない問題は印を付けて後回しにし、最後に必ず全問マークする
- 午前：計算問題は問題用紙の余白で手順どおりに計算する。選択肢の数値から逆算するのも有効
- 午後：最初の5分で全問をざっと見て、選択する4問を決める。選択欄の問題番号を○で囲むのを忘れない
- 午後：1問30分を超えたら次の問題へ。空欄は作らない

- 午後：答えは問題文の言葉を使い、字数の8割以上を書く

**ポイント　最後に**

応用情報技術者試験は範囲が広いですが、問われる内容の多くは「基本的な用語の正確な理解」と「定番の計算」です。本書で全体像をつかんだら、午前クイズと過去問で繰り返し確認し、午後対策シートで記述の練習を重ねていきましょう。合格を心から応援しています。